SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-184458-126554

Urýchlenie smerovania s použitím CUDA

Bakalárska práca

2026 **Tomáš Dzivjak**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-184458-126554

Urýchlenie smerovania s použitím CUDA

Študijný program:	Informačné a komunikačné technológie
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav multimediálnych informačných a komunikačných technológií (FEI)
Vedúci záverečnej práce:	doc. Ing. Martin Medvecký, PhD.

Bratislava 2026 **Tomáš Dzivjak**

Slovenská technická univerzita v Bratislave
Ústav multimediálnych informačných a komunikačných technológií

Fakulta elektrotechniky a informatiky
Akademický rok: 2025/2026
Evidenčné číslo: FEI-184458-126554

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Tomáš Dzivjak**
ID študenta:	126554
Študijný program:	informačné a komunikačné technológie
Študijný odbor:	informatika
Vedúci práce:	doc. Ing. Martin Medvecký, PhD.
Vedúci pracoviska:	doc. Ing. Radoslav Vargic, PhD.
Miesto vypracovania:	Ústav multimediálnych informačných a komunikačných technológií

Názov práce: **Urýchlenie smerovania s použitím CUDA**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

1. Naštudujte technológiu CUDA.
2. Analyzujte možnosti použitia CUDA pre urýchlenie smerovania dátovej prevádzky v IP smerovačoch
3. Na vhodne zvolenom modeli overte možnosť urýchlenia smerovania dátovej prevádzky v IP smerovačoch
4. Dosiahnuté výsledky vyhodnoťte.

Rozsah práce: minimálne 1 AH a maximálne 2 AH okrem prípadnej ďalšej technickej dokumentácie

Termín odovzdania bakalárskej práce:	05. 06. 2026
Dátum schválenia zadania bakalárskej práce:	14. 05. 2026
Zadanie bakalárskej práce schválil:	doc. Ing. Radoslav Vargic, PhD. – garant študijného programu

Poďakovanie

Rád by som sa poďakoval vedúcemu mojej bakalárskej práce, doc. Ing. Martinovi Medveckému, PhD., za odborné vedenie, konzultácie, cenné pripomienky a usmernenie pri spracovaní témy. Jeho rady mi pomohli lepšie uchopiť problematiku urýchlenia smerovania pomocou technológie CUDA a vhodne zamerať experimentálnu časť práce.

Abstrakt

Bakalárska práca sa zaoberá možnosťami urýchlenia IPv4 smerovacieho rozhodovania pomocou technológie CUDA. Predmetom práce je operácia vyhľadania najdlhšieho zhodného prefixu, ktorá patrí medzi základné výpočtové úlohy dátovej roviny smerovača. V práci sú analyzované a implementované dátové štruktúry DIR24/8 a MultiBit Trie v procesorovej aj grafickej verzii. Procesorové implementácie využívajú viacvláknové spracovanie pomocou OpenMP, zatiaľ čo grafické implementácie sú realizované pomocou CUDA. Súčasťou práce je aj hybridný model pre algoritmus DIR24/8, ktorý umožňuje porovnať samostatné CPU a GPU spracovanie s rozhodovacou vrstvou vyberajúcou vhodnejšiu výpočtovú platformu.

Experimentálne overenie je založené na syntetických vstupných dátach a na dátach modelujúcich enterprise a hotspot typ prevádzky. Výsledky sú hodnotené najmä podľa času spracovania, priepustnosti a vplyvu prenosovej réžie medzi CPU a GPU. Práca ukazuje, že prínos GPU akcelerácie závisí od veľkosti dávky, použitej dátovej štruktúry, charakteru vstupnej prevádzky a započítania nákladov na prenos dát. Profilovanie pomocou nástroja Nsight Compute zároveň poskytuje detailnejší pohľad na využitie výpočtových a pamäťových prostriedkov grafického procesora.

Kľúčové slová

CUDA, IPv4 smerovanie, vyhľadanie najdlhšieho zhodného prefixu, DIR24/8, MultiBit Trie, GPU akcelerácia, OpenMP, výkonnostné meranie

Abstract

This bachelor thesis deals with the acceleration of IPv4 routing decisions using CUDA technology. The work focuses on the Longest Prefix Match operation, which represents one of the fundamental computational tasks in the data plane of a router. The thesis analyzes and implements the DIR24/8 and MultiBit Trie data structures in both CPU and GPU versions. The CPU implementations use multithreaded processing with OpenMP, while the GPU implementations are based on CUDA. The work also includes a hybrid DIR24/8 model that makes it possible to compare standalone CPU and GPU processing with a decision layer selecting the more suitable computing platform.

The experimental evaluation is based on synthetic input data and on datasets modelling enterprise and hotspot traffic patterns. The results are evaluated mainly according to processing time, throughput, and the impact of data transfer overhead between the CPU and GPU. The thesis shows that the benefit of GPU acceleration depends on the batch size, the selected data structure, the characteristics of the input traffic, and whether data transfer costs are included in the measurement. Profiling with Nsight Compute also provides a more detailed view of the utilization of computational and memory resources of the graphics processor.

Keywords

CUDA, IPv4 routing, Longest Prefix Match, DIR24/8, MultiBit Trie, GPU acceleration, OpenMP, benchmark

Obsah

Zoznam obrázkov 8
Zoznam tabuliek 11

Zoznam značiek a skratiek 12

Úvod 14

1 Teoretické východiská sieťového smerovania 16
1.1 Prepájanie paketov, FIB a operácia Longest Prefix Match 16
1.2 Dátové štruktúry DIR24/8 a MultiBit Trie 17
1.3 Výkonnostné hľadiská LPM vyhľadávania 18

2 Výkonnostné limity klasických riešení 20
2.1 Spracovanie paketov na CPU 20
2.2 Hardvérové riešenia – ASIC, FPGA 21
2.3 Softvérové akcelerátory – DPDK a netmap 21
2.4 Porovnanie CPU a GPU pri paralelnom spracovaní 22

3 Paralelizmus a GPGPU architektúry 24
3.1 Architektúra CPU a GPU 24
3.2 Model programovania CUDA 25
3.3 Typické úlohy vhodné pre GPU akceleráciu 26

4 Súčasný stav riešení GPU v sieťovom prostredí 27
4.1 Prehľad literatúry a výskumu 27
4.2 Existujúce implementácie spracovania paketov na GPU 28
4.3 Identifikované problémy a nevýhody GPU riešení 28

5 Zdôvodnenie výberu riešenia 29
5.1 Formulácia problému pre CUDA implementáciu 29
5.2 Hypotézy a očakávané prínosy riešenia 29
5.3 Riziká a obmedzenia navrhovaného prístupu 30

6 Metodika experimentálneho overenia 31
6.1 Rozsah experimentu a testované varianty 31
6.2 Testovacie vstupné dáta a typy vstupnej prevádzky 32
6.2.1 Testovacie vstupné dáta 32

6.2.2 Typy vstupnej prevádzky 32
6.2.3 Merané konfigurácie 33
6.3 Metodika CPU meraní 34
6.4 Metodika GPU meraní 35
6.5 Metodika hybridných meraní 36
6.6 Sledované metriky 37
6.7 Použitý hardvér a softvér 37

7 Návrh a implementácia riešenia 39
7.1 Architektúra a pracovný tok riešenia 39
7.2 Implementácia DIR24/8 40
7.3 MultiBit Trie 40
7.4 CPU implementácie a paralelizácia pomocou OpenMP 41
7.5 CUDA implementácie a konfigurácia CUDA jadier 41
7.6 Hybridný DIR24/8 model 43
7.7 Organizácia meracích programov a spracovanie výsledkov 44

8 Záverečné vyhodnotenie výsledkov 45
8.1 Porovnanie CPU a GPU implementácií 45
8.2 Vplyv typu prevádzky a prefixovej lokality 47
8.3 Profilovacia analýza pomocou Nsight Compute 48
8.4 Vyhodnotenie hybridného DIR24/8 modelu 49
8.5 Diskusia výsledkov a obmedzenia merania 51

Záver 53

Literatúra 55

Použitie nástrojov umelej inteligencie 57

Zoznam elektronických príloh 58

Dodatok A: Použité skripty a spôsob spustenia 59

Dodatok B: Doplnkové grafy experimentálneho vyhodnotenia 63

Dodatok C: Súhrnné CSV výstupy meracích skriptov 81

Dodatok D: Výstupy profilovania Nsight Compute 85

Zoznam obrázkov

Obr. 1 - Porovnanie CPU a GPU riešení pri IPv4 vyhľadávaní podľa štúdie APUNet [2]..23
Obr. 2 - Zjednodušené porovnanie architektúry CPU a GPU podľa NVIDIA [11]. 24
Obr. 3 - Blokový diagram celého implementovaného riešenia 39
Obr. 4 - Vplyv typu prevádzky na priepustnosť CPU implementácie DIR24/8 47
Obr. 5 - Vplyv typu prevádzky na čas spracovania dávky pri CPU implementácii DIR24/8 63
Obr. 6 - Vplyv typu prevádzky na priepustnosť CPU implementácie DIR24/8 63
Obr. 7 - Vplyv typu prevádzky na čas spracovania dávky pri CPU implementácii MultiBit Trie 64
Obr. 8 - Vplyv typu prevádzky na priepustnosť CPU implementácie MultiBit Trie 64
Obr. 9 - Porovnanie priepustnosti CPU implementácií DIR24/8 a MultiBit Trie pri rôznych typoch prevádzky 65
Obr. 10 - Vplyv typu prevádzky na čas spracovania dávky pri GPU implementácii DIR24/8 65
Obr. 11 - Vplyv typu prevádzky na priepustnosť GPU implementácie DIR24/8 66
Obr. 12 - Vplyv typu prevádzky na čas spracovania dávky pri GPU implementácii MultiBit Trie 66
Obr. 13 - Vplyv typu prevádzky na priepustnosť GPU implementácie MultiBit Trie 67
Obr. 14 - Porovnanie času spracovania dávky pri GPU implementáciách DIR24/8 a MultiBit Trie bez prenosovej réžie 67
Obr. 15 - Porovnanie priepustnosti GPU implementácií DIR24/8 a MultiBit Trie bez prenosovej réžie 68
Obr. 16 - Porovnanie času spracovania dávky pri GPU implementáciách DIR24/8 a MultiBit Trie vrátane prenosovej réžie 68
Obr. 17 - Porovnanie priepustnosti GPU implementácií DIR24/8 a MultiBit Trie vrátane prenosovej réžie 69
Obr. 18 - Porovnanie času spracovania dávky CPU a GPU implementácií pri syntetickej prevádzke bez samostatných meraní ČJ 69

Obr. 19 - Porovnanie priepustnosti CPU a GPU implementácií pri syntetickej prevádzke bez samostatných meraní ČJ 70
Obr. 20 - Porovnanie času spracovania dávky CPU a GPU implementácií pri syntetickej prevádzke vrátane samostatných meraní ČJ 70
Obr. 21 - Porovnanie priepustnosti CPU a GPU implementácií pri syntetickej prevádzke vrátane samostatných meraní ČJ 71
Obr. 22 - Porovnanie času spracovania dávky CPU a GPU implementácií pri enterprise prevádzke vrátane samostatných meraní ČJ 71
Obr. 23 - Porovnanie priepustnosti CPU a GPU implementácií pri enterprise prevádzke vrátane samostatných meraní ČJ 72
Obr. 24 - Realistické porovnanie času spracovania dávky CPU a GPU implementácií pri enterprise prevádzke 72
Obr. 25 - Realistické porovnanie priepustnosti CPU a GPU implementácií pri enterprise prevádzke 73
Obr. 26 - Porovnanie času spracovania dávky CPU a GPU implementácií pri hotspot prevádzke vrátane samostatných meraní ČJ 73
Obr. 27 - Porovnanie priepustnosti CPU a GPU implementácií pri hotspot prevádzke vrátane samostatných meraní ČJ 74
Obr. 28 - Realistické porovnanie času spracovania dávky CPU a GPU implementácií pri hotspot prevádzke 74
Obr. 29 - Realistické porovnanie priepustnosti CPU a GPU implementácií pri hotspot prevádzke 75
Obr. 30 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke 75
Obr. 31 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke 76
Obr. 32 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke vrátane merania samotného CUDA jadra 76
Obr. 33 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke 77
Obr. 34 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke 77

Obr. 35 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke vrátane merania samotného CUDA jadra 78
Obr. 36 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke 78
Obr. 37 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke 79
Obr. 38 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke vrátane merania samotného CUDA jadra 79
Obr. 39 (vľavo) - Súhrnné porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pre všetky typy prevádzky 80
Obr. 40 (vpravo) - Súhrnné porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pre všetky typy prevádzky vrátane merania samotného CUDA jadra 80

Zoznam tabuliek

Tab. 1 - Opis troch použitých typov prevádzky 33

Tab. 2 - Zoznam testovaných konfigurácií vstupných dát 33

Tab. 3 - Špecifikácia použitého hardvéru 38

Tab. 4 - Porovnanie priepustnosti implementácií pri reprezentatívnych konfiguráciách a typoch prevádzky 46

Tab. 5 - Nameraná prefixová lokalita testovaných typov prevádzky 47

Tab. 6 - Súhrn profilovania CUDA jadier pomocou Nsight Compute 48

Tab. 7 - Dopad rozhodovacej réžie na priepustnosť hybridného DIR24/8 modelu 50

Tab. 8 - Obsah elektronických príloh odovzdávaných spolu s prácou 58

Tab. 9 - Skripty na generovanie vstupných dát 59

Tab. 10 - Meracie programy pre LPM implementácie 60

Tab. 11 - Skripty na spracovanie výsledkov a tvorbu grafov 60

Tab. 12 - Súhrnné CSV výstupy CPU meraní algoritmu DIR24/8 81

Tab. 13 - Súhrnné CSV výstupy CPU meraní algoritmu MultiBit Trie 81

Tab. 14 - Súhrnné CSV výstupy GPU meraní algoritmu DIR24/8 82

Tab. 15 - Súhrnné CSV výstupy GPU meraní algoritmu MultiBit Trie 82

Tab. 16 - Súhrnné CSV výstupy hybridného DIR24/8 modelu 82

Tab. 17 - Doplnkové CSV výstupy a logy meraní 83

Tab. 18 - Hlavné typy údajov v CSV výstupoch 84

Tab. 19 - PDF výstupy profilovania CUDA jadier pomocou Nsight Compute 85

Tab. 20 - CSV výstupy profilovania CUDA jadier pomocou Nsight Compute 86

Tab. 21 - Vybrané metriky použité pri interpretácii profilovania 87

Zoznam značiek a skratiek

ASIC – Application-Specific Integrated Circuit; aplikačne špecifický integrovaný obvod
CUDA – Compute Unified Device Architecture; platforma spoločnosti NVIDIA na všeobecné výpočty na grafických procesoroch
CUDA jadro – výpočtová funkcia spustená z hostiteľskej časti programu a paralelne vykonávaná veľkým počtom vlákien na grafickom procesore; v práci nejde o fyzické hardvérové jadro GPU
CPU – Central Processing Unit; centrálny procesor
CSV – Comma-Separated Values; textový formát súboru s hodnotami oddelenými oddeľovačom
ČJ – čas samotného CUDA jadra; čas vykonania LPM výpočtu na GPU bez započítania prenosu vstupných a výstupných dát medzi operačnou pamäťou a pamäťou GPU
ČJP – čas GPU spracovania vrátane CUDA jadra a prenosovej réžie; zahŕňa prenos vstupnej dávky do pamäte GPU, vykonanie CUDA jadra a prenos výsledkov späť do operačnej pamäte
DIR24/8 – tabuľkový algoritmus na vyhľadanie najdlhšieho zhodného prefixu v IPv4 adresách
DPDK – Data Plane Development Kit; súbor knižníc a ovládačov na vysokovýkonné spracovanie paketov v používateľskom priestore
DPI – Deep Packet Inspection; hĺbková kontrola paketov
DRAM – Dynamic Random-Access Memory; dynamická operačná pamäť
FEI – Fakulta elektrotechniky a informatiky
FIB – Forwarding Information Base; prepájacia tabuľka používaná v dátovej rovine smerovača
FPGA – Field-Programmable Gate Array; programovateľné hradlové pole
GB – gigabajt
GDDR6 – Graphics Double Data Rate 6; typ grafickej pamäte
GPGPU – General-Purpose computing on Graphics Processing Units; všeobecné výpočty na grafických procesoroch
GPU – Graphics Processing Unit; grafický procesor
IP – Internet Protocol; internetový protokol
IPv4 – Internet Protocol version 4; štvrtá verzia internetového protokolu
L2 cache – vyrovnávacia pamäť druhej úrovne
LPM – Longest Prefix Match; vyhľadanie najdlhšieho zhodného prefixu
MB – megabajt

MIMD – Multiple Instructions, Multiple Data; výpočtový model s viacerými inštrukciami nad viacerými dátami

Mpps – Million packets per second; milión paketov, prípadne vyhľadávacích operácií, za sekundu

MultiBit Trie – viacbitový prefixový strom používaný na LPM vyhľadávanie

Nsight Compute – profilovací nástroj spoločnosti NVIDIA na analýzu CUDA jadier

OpenMP – Open Multi-Processing; rozhranie na viacvláknové programovanie so zdieľanou pamäťou

PCIe – Peripheral Component Interconnect Express; vysokorýchlostné rozhranie používané okrem iného na pripojenie grafických kariet

PDF – Portable Document Format; prenosný formát dokumentu

RAM – Random-Access Memory; operačná pamäť

RT – Ray Tracing; hardvérové jednotky GPU určené na výpočty súvisiace so sledovaním lúčov

SIMD – Single Instruction, Multiple Data; výpočtový model jednej inštrukcie nad viacerými dátami

SIMT – Single Instruction, Multiple Threads; výpočtový model jednej inštrukcie nad viacerými vláknami

SM – Streaming Multiprocessor; výpočtová jednotka grafického procesora NVIDIA

TCAM – Ternary Content Addressable Memory; ternárna obsahovo adresovateľná pamäť

TTL – Time To Live; hodnota určujúca životnosť paketu v sieti

Úvod

S rastúcimi požiadavkami na priepustnosť moderných počítačových sietí narastá aj význam efektívneho spracovania paketov v dátovej rovine sieťových zariadení. Jednou zo základných operácií pri IPv4 smerovaní je určenie výstupnej cesty paketu na základe jeho cieľovej adresy. Táto operácia sa v smerovači vykonáva opakovane pre každý spracovaný paket, a preto môže pri veľkom objeme prevádzky významne ovplyvňovať celkovú výkonnosť systému.

Pri IPv4 smerovaní sa na výber najvhodnejšieho záznamu zo smerovacej, respektíve prepájacej tabuľky používa operácia vyhľadania najdlhšieho zhodného prefixu (Longest Prefix Match, LPM). Vzhľadom na to, že jednotlivé vyhľadávacie operácie nad už zostavenou tabuľkou sú navzájom nezávislé, predstavuje LPM vhodný kandidát na paralelné spracovanie. Tradičné procesorové riešenia dokážu využívať viacjadrové spracovanie a vyrovnávacie pamäte, no pri veľkých dávkach vstupných adries môžu narážať na limity pamäťovej hierarchie a dostupného počtu výpočtových vlákien.

Jednou z možností urýchlenia takýchto výpočtovo opakovaných úloh je využitie grafických procesorov. Technológia CUDA umožňuje spúšťať veľké množstvo paralelných vlákien na grafickom procesore a využiť tak jeho dátový paralelizmus. Pri sieťových úlohách však samotná dostupnosť vysokého paralelného výkonu nemusí automaticky znamenať vyššiu praktickú priepustnosť. Výsledný výkon ovplyvňuje aj spôsob prístupu do pamäte, veľkosť spracúvanej dávky, charakter vstupných dát a réžia spojená s prenosom údajov medzi operačnou pamäťou a pamäťou grafického procesora.

Cieľom tejto bakalárskej práce je preskúmať možnosti urýchlenia IPv4 LPM vyhľadávania pomocou technológie CUDA a porovnať ich s viacvláknovým spracovaním na procesore. V práci sú implementované a experimentálne vyhodnotené dve dátové štruktúry používané pri LPM vyhľadávaní: DIR24/8 a MultiBit Trie. Pre obe štruktúry sú pripravené procesorové aj grafické varianty. Súčasťou práce je aj hybridný model pre algoritmus DIR24/8, ktorého cieľom je posúdiť, či môže rozhodovacia vrstva medzi CPU a GPU spracovaním priniesť praktický prínos.

Experimentálna časť práce je založená na dávkovom spracovaní IPv4 adries nad vopred zostavenými FIB tabuľkami. Práca sa nezameriava na kompletnú implementáciu smerovača ani na meranie kompletného reťazca spracovania paketov v smerovači vrátane príjmu a odosielania paketov. Predmetom merania je výpočtová časť smerovacieho rozhodnutia, teda LPM vyhľadávanie nad rôznymi veľkosťami vstupných dávok a rôznymi typmi prevádzky. V práci sa preto pojem urýchlenie smerovania používa v zúženom význame ako urýchlenie výpočtovej operácie LPM, ktorá tvorí jednu z kľúčových častí smerovacieho rozhodovania

v dátovej rovine. Práca nehodnotí priepustnosť kompletného smerovača. Okrem synteticky generovaných vstupov sú použité aj dátové modely reprezentujúce enterprise a hotspot prevádzku, ktoré umožňujú sledovať vplyv prefixovej lokality na výslednú priepustnosť.

Prvá časť práce sa venuje teoretickým východiskám IPv4 smerovania, operácii LPM a vlastnostiam dátových štruktúr DIR24/8 a MultiBit Trie. Následne sú opísané výkonnostné limity klasických riešení, princípy paralelného spracovania na GPU a existujúce prístupy k využitiu grafických procesorov v sieťovom prostredí. Praktická časť práce opisuje metodiku merania, návrh a implementáciu jednotlivých CPU, GPU a hybridných riešení. Záverečná experimentálna časť porovnáva dosiahnuté výsledky, analyzuje vplyv typu prevádzky a dopĺňa hodnotenie o profilovanie CUDA implementácií pomocou nástroja Nsight Compute.

1 Teoretické východiská sieťového smerovania

IP smerovanie zabezpečuje doručenie paketov medzi sieťami na základe ich cieľovej IP adresy. Z pohľadu fungovania smerovača je potrebné rozlišovať medzi smerovaním (routing) a prepájaním paketov (forwarding). Smerovanie predstavuje proces vytvárania a aktualizácie informácií o dostupných trasách v riadiacej rovine zariadenia. Prepájanie je naopak operácia vykonávaná v dátovej rovine, pri ktorej sa pre každý prijatý paket určí výstupná cesta podľa už existujúcich smerovacích informácií [1].

Táto práca sa nezaoberá vytváraním smerovacích trás ani fungovaním dynamických smerovacích protokolov. Predmetom skúmania je výpočtová operácia vykonávaná pri prepájaní paketov: vyhľadanie zodpovedajúceho záznamu pre cieľovú IPv4 adresu. Táto operácia sa vykonáva pre každý spracovaný paket a pri veľkom objeme prevádzky môže významne ovplyvňovať výkonnosť softvérového smerovania.

Cieľom tejto kapitoly je preto stručne vysvetliť význam prepájacej tabuľky, princíp operácie LPM a vlastnosti dátových štruktúr relevantných pre experimentálne porovnanie CPU a GPU spracovania.

1.1 Prepájanie paketov, FIB a operácia Longest Prefix Match

Pri prepájaní IPv4 paketov smerovač rozhoduje o ďalšom postupe paketu na základe jeho cieľovej IP adresy. Na tento účel používa prepájaciu tabuľku označovanú ako Forwarding Information Base (FIB). FIB obsahuje sieťové prefixy a im priradené výstupné hodnoty, napríklad identifikátory ďalšieho smerovača alebo výstupného rozhrania. Na rozdiel od riadiacej roviny, ktorej úlohou je získavať a aktualizovať informácie o trasách, dátová rovina využíva pripravenú FIB na rýchle rozhodovanie pri spracovaní paketov [1].

V smerovacej tabuľke sa môžu nachádzať prefixy s rôznou dĺžkou, pričom viacero z nich môže zodpovedať jednej cieľovej IP adrese. Pri prepájaní sa preto používa operácia vyhľadania najdlhšieho zhodného prefixu (Longest Prefix Match, LPM). Jej úlohou je vybrať zodpovedajúci prefix s najväčším počtom zhodných najvýznamnejších bitov cieľovej adresy. Ak napríklad FIB obsahuje prefixy 192.168.0.0/16 a 192.168.1.0/24, pre cieľovú adresu 192.168.1.25 sa použije špecifickejší prefix /24 [1].

V dokumentácii knižnice DPDK je LPM opísané ako vyhľadávanie nad 32-bitovým kľúčom, ktorého výsledkom je hodnota nasledujúceho kroku priradená k najlepšie zodpovedajúcemu prefixu [14]. Takto formulovaný model umožňuje hodnotiť výpočtovú náročnosť prepájacieho rozhodnutia oddelene od ostatných činností smerovača, napríklad od prijatia, úpravy alebo odoslania samotného paketu.

LPM vyhľadávanie je vhodné na paralelné spracovanie, pretože jednotlivé vstupné adresy možno spracovať nezávisle. Pri už zostavenej FIB jedna vyhľadávacia operácia nemení dátovú štruktúru ani výsledky iných operácií. Vstupnú dávku adries je preto možné rozdeliť medzi viaceré vlákna centrálneho procesora alebo mapovať jednotlivé adresy na samostatné vlákna grafického procesora.

1.2 Dátové štruktúry DIR24/8 a MultiBit Trie

Vyhľadanie najdlhšieho zhodného prefixu možno realizovať viacerými spôsobmi, ktoré sa líšia časovou zložitosťou, pamäťovou náročnosťou a spôsobom, akým sa pristupuje do pamäte. Najjednoduchším riešením je lineárne prehľadanie všetkých prefixov vo FIB, pri ktorom sa pre cieľovú adresu postupne overuje zhoda s jednotlivými záznamami. Takýto prístup je jednoduchý a vhodný na overovanie správnosti výsledkov, jeho výpočtová náročnosť však rastie priamo s počtom prefixov, preto nie je vhodný pre výkonné spracovanie rozsiahlych tabuliek.

Medzi optimalizované prístupy pre IPv4 vyhľadávanie patrí algoritmus DIR24/8. Je založený na rozdelení 32-bitovej IPv4 adresy na prvých 24 bitov a zostávajúcich 8 bitov. Prvých 24 bitov tvorí index do hlavnej tabuľky s 2^{24} položkami. Prefixy s dĺžkou najviac /24 možno vyhodnotiť priamym prístupom do tejto tabuľky. Ak je potrebné rozlíšiť prefix dlhší než /24, položka hlavnej tabuľky odkazuje na doplnkovú tabuľku pre daný prefix. Táto doplnková tabuľka obsahuje 2^8 položiek, indexovanú poslednými ôsmimi bitmi adresy [14].

Hlavnou výhodou algoritmu DIR24/8 je malý a predvídateľný počet pamäťových prístupov pri jednom vyhľadaní. Pre prefixy s dĺžkou najviac /24 postačuje jeden prístup do hlavnej tabuľky a pre dlhšie prefixy sa vykoná ešte jeden doplnkový prístup. Cenou za túto vlastnosť je vyššia pamäťová náročnosť spôsobená rozsiahlou hlavnou tabuľkou. Podrobnejší opis organizácie tabuliek a operácií vyhľadávania uvádza dokumentácia knižnice DPDK [14].

Alternatívny prístup predstavuje viacbitový prefixový strom, v ďalšom texte označovaný aj ako MultiBit Trie. Prefixový strom uchováva záznamy podľa postupne spracúvaných bitov adresy. Kým binárny strom vyhodnocuje v jednom kroku jeden bit, viacbitový strom spracúva skupinu bitov naraz. Tým sa znižuje počet úrovní, ktoré je potrebné počas vyhľadávania prejsť. Napríklad pri spracovaní štyroch bitov v jednom kroku môže jeden uzol rozlišovať 16 možností pokračovania a pre úplnú 32-bitovú IPv4 adresu postačuje najviac osem úrovní.

Použitie viacbitových stromov súvisí s postupom označovaným ako riadená expanzia prefixov (controlled prefix expansion). Pri ňom sa prefixy prispôsobujú hraniciam určeným zvolenou veľkosťou kroku, čím sa umožní spracovať viac bitov adresy v jednej operácii.

Výsledkom môže byť zníženie počtu krokov vyhľadávania, avšak za cenu zvýšených pamäťových nárokov spôsobených rozšírením niektorých prefixov [15]. Detailnú analýzu tohto kompromisu uvádzajú Srinivasan a Varghese [15].

DIR24/8 a MultiBit Trie predstavujú dva rozdielne kompromisy medzi rýchlosťou vyhľadávania a pamäťovým usporiadaním. DIR24/8 využíva rozsiahlu priamo indexovanú tabuľku a obmedzuje počet pamäťových prístupov. MultiBit Trie používa hierarchické usporiadanie, pri ktorom vyhľadávanie postupuje cez viacero uzlov podľa skupín bitov adresy. Pri posudzovaní možností paralelného spracovania je tento rozdiel významný, pretože výsledný výkon nezávisí iba od počtu vykonaných operácií, ale aj od pravidelnosti a lokality prístupov do pamäte.

1.3 Výkonnostné hľadiská LPM vyhľadávania

Výkonnosť prepájania paketov je výrazne ovplyvnená rýchlosťou vyhľadania príslušného záznamu vo FIB. Pri operácii vyhľadania najdlhšieho zhodného prefixu predstavuje každá cieľová IP adresa samostatnú vyhľadávaciu požiadavku. Pri dávkovom spracovaní možno preto výkon vyjadriť priepustnosťou, teda počtom vykonaných LPM vyhľadaní za jednotku času. V prostredí sieťového spracovania sa táto priepustnosť často uvádza v miliónoch spracovaných požiadaviek alebo paketov za sekundu (Mpps). [5]

Samotná priepustnosť nezávisí iba od počtu výpočtových jednotiek procesora. Pri LPM vyhľadávaní zohráva významnú úlohu aj organizácia použitej dátovej štruktúry a spôsob prístupov do pamäte. Algoritmus s malým a predvídateľným počtom pamäťových prístupov môže dosahovať odlišné výsledky než algoritmus založený na postupnom prechode viacerými uzlami stromovej štruktúry. Pri hodnotení možností urýchlenia LPM je preto potrebné prihliadať nielen na výpočtový paralelizmus, ale aj na pamäťové správanie zvoleného algoritmu [14], [15]. Prehľad optimalizovaných algoritmov pre rýchle LPM vyhľadávanie uvádza Waldvogel [6].

Centrálne procesory (CPU) disponujú vyrovnávacími pamäťami (cache). Ich prínos sa môže prejaviť najmä pri opakovanom prístupe k malej alebo dobre lokalizovanej časti dátovej štruktúry. Pri tabuľkovo orientovanom LPM algoritme, akým je DIR24/8, možno preto predpokladať, že koncentrácia cieľových adries do rovnakých alebo blízkych prefixových oblastí ovplyvní lokalitu prístupov do hlavnej tabuľky. Takýto charakter vstupných adries sa ďalej označuje ako prefixová lokalita. Pri rozptýlených adresách sa naopak prístupy môžu rozložiť do väčšieho rozsahu dátovej štruktúry, čím sa znižuje možnosť opakovaného využitia údajov vo vyrovnávacej pamäti [4], [14].

Grafické procesory (GPU) poskytujú vysoký stupeň dátového paralelizmu a umožňujú paralelne vykonávať veľké množstvo nezávislých vyhľadávacích operácií. To je priaznivé

pre dávkové LPM spracovanie, pri ktorom jednotlivé adresy nevyžadujú vzájomnú synchronizáciu. Výsledná výkonnosť GPU však závisí od toho, či majú paralelné vlákna efektívny prístup k dátam a či je spracúvaná dávka dostatočne rozsiahla na využitie dostupného paralelizmu. Pri nevhodnom pamäťovom správaní môže vysoký počet výpočtových jednotiek priniesť menší prínos, než by naznačoval samotný teoretický výkon grafického procesora [4].

Pri použití samostatnej grafickej karty je potrebné zohľadniť aj prenos údajov medzi operačnou pamäťou systému a pamäťou GPU. Pred vykonaním výpočtu musia byť vstupné údaje sprístupnené grafickému procesoru a po jeho dokončení sa výsledky spravidla prenášajú späť. Pri malých dávkach môže táto prenosová réžia tvoriť významnú časť celkového času spracovania, zatiaľ čo pri rozsiahlejších dávkach môže byť lepšie prekrytá výhodou paralelného výpočtu. Význam tohto obmedzenia pri GPU spracovaní sieťových úloh uvádzajú aj práce zaoberajúce sa architektúrami so spoločnou alebo oddelenou pamäťou CPU a GPU [2], [4].

Pri posudzovaní možností urýchlenia LPM vyhľadávania je teda potrebné zohľadniť tri základné hľadiská: vlastnosti dátovej štruktúry, charakter prístupov k prefixom a režijné náklady spojenia medzi CPU a GPU. Tieto faktory určujú, či je pre konkrétny typ spracovania výhodnejšia nízka latencia a vyrovnávacia pamäť CPU, alebo vysoký dátový paralelizmus GPU.

2 Výkonnostné limity klasických riešení

S rastúcimi požiadavkami na priepustnosť moderných sietí sa ukázalo, že tradičné prístupy k spracovaniu sieťovej prevádzky na všeobecných procesoroch (CPU) prestávajú postačovať. Klasické softvérové riešenia postavené na všeobecných procesoroch poskytujú vysokú flexibilitu, pri veľmi veľkom objeme sieťovej prevádzky však môžu narážať na limity spôsobené réžiou spracovania, pamäťovou hierarchiou a obmedzenou mierou paralelizmu.

Cieľom tejto kapitoly je stručne zasadiť skúmané LPM vyhľadávanie do širšieho kontextu vysokovýkonného spracovania paketov. Hardvérové riešenia typu ASIC a FPGA a softvérové rámce ako DPDK a netmap sú uvedené ako referenčný rámec, nie ako predmet experimentálneho porovnania tejto práce.

2.1 Spracovanie paketov na CPU

V tradičnom softvérovom smerovaní sú pakety spracúvané centrálnym procesorom prostredníctvom sieťového zásobníka operačného systému. Po prijatí paketu sieťovým rozhraním musí operačný systém zabezpečiť jeho presun do pamäte, spracovanie príslušných vrstiev protokolového zásobníka, kontrolu hlavičiek a rozhodnutie o ďalšom odoslaní. Tento postup je univerzálny a flexibilný, ale pri vysokých prenosových rýchlostiach prináša režijné náklady spojené s obsluhou paketov, kopírovaním údajov a prácou jadra operačného systému [8], [9].

Jednou z výpočtovo významných operácií v dátovej rovine smerovača je vyhľadanie zodpovedajúceho záznamu vo FIB na základe cieľovej IP adresy. Aj keď kompletné spracovanie paketu zahŕňa viacero ďalších krokov, ako kontrolu hlavičiek, výpočet kontrolných súčtov alebo odoslanie cez výstupné rozhranie, rýchlosť LPM vyhľadávania priamo ovplyvňuje schopnosť smerovača rozhodovať o veľkom množstve paketov. Z toho dôvodu sa pri analýze možností urýchlenia smerovania často pozornosť sústreďuje práve na optimalizáciu vyhľadávania v smerovacích tabuľkách.

CPU poskytuje pri takomto spracovaní nízku latenciu a viacúrovňovú vyrovnávaciu pamäť. Pri paralelnom spracovaní možno dávku cieľových adries rozdeliť medzi viacero vlákien, keďže jednotlivé LPM operácie nad už zostavenou FIB sú navzájom nezávislé. Výkon však nezávisí iba od počtu použitých jadier alebo vlákien, ale aj od pamäťového správania použitej dátovej štruktúry a od lokality prístupov k prefixom [4].

Napriek možnosti viacvláknového spracovania zostáva výkon CPU riešení závislý od pamäťovej hierarchie a charakteru prístupov do dátovej štruktúry. S rastúcim počtom vlákien

sa môže zvyšovať priepustnosť, no škálovanie nemusí byť lineárne, najmä ak jednotlivé vlákna pristupujú k rozsiahlym alebo rozptýleným častiam pamäte.

2.2 Hardvérové riešenia – ASIC, FPGA

Hardvérové riešenia predstavujú jednu z možností, ako dosiahnuť vysokú priepustnosť pri spracovaní sieťovej prevádzky. Aplikačne špecifické integrované obvody (ASIC) sú navrhnuté priamo na vykonávanie konkrétnych operácií dátovej roviny, napríklad prepájania paketov, vyhľadávania v tabuľkách alebo aplikovania jednoduchých pravidiel. Ich hlavnou výhodou je vysoká priepustnosť a nízka latencia, keďže výpočtová logika je realizovaná priamo v hardvéri. Nevýhodou je obmedzená flexibilita, pretože zmena funkcionality si môže vyžadovať nový návrh hardvéru. [1]

Rekonfigurovateľné hradlové polia (FPGA) predstavujú flexibilnejšiu hardvérovú alternatívu. Umožňujú vytvárať paralelné spracovateľské reťazce prispôsobené konkrétnej sieťovej úlohe. Po nasadení môžu byť opätovne prekonfigurované, čo je výhodou v systémoch, kde sa často menia pravidlá, formáty paketov alebo bezpečnostné mechanizmy. V porovnaní s ASIC riešeniami poskytujú vyššiu mieru prispôsobiteľnosti, avšak ich návrh je zložitejší než pri bežnom softvérovom programovaní a vyžaduje špecifické znalosti hardvérovej implementácie [7].

Z pohľadu tejto práce predstavujú ASIC a FPGA najmä referenčný rámec pre špecializované urýchlenie dátovej roviny. Ich podrobné hodnotenie ani implementácia však nie sú predmetom práce. Ďalšia časť sa preto sústreďuje na programovateľné riešenia a na otázku, do akej miery možno LPM vyhľadávanie urýchliť pomocou všeobecne dostupnejších CPU a GPU platforiem.

2.3 Softvérové akcelerátory – DPDK a netmap

Okrem špecializovaného hardvéru existujú aj softvérové prístupy, ktorých cieľom je znížiť réžiu tradičného sieťového zásobníka operačného systému. S rastúcou rýchlosťou sieťových rozhraní sa ukázalo, že zásadné obmedzenie pri spracovaní paketov je spôsobené najmä kopírovaním dát medzi jadrom operačného systému a používateľským priestorom a nadmerným počtom systémových volaní. S cieľom eliminovať tieto úzke hrdlá vznikli softvérové akcelerátory ako DPDK a netmap, ktoré využívajú princíp obídenia jadra operačného systému, označovaný aj ako kernel bypass [8], [9].

DPDK (Data Plane Development Kit) je sada knižníc a ovládačov určená na vysokovýkonné spracovanie sieťových paketov v používateľskom priestore. Jeho cieľom je minima-

lizovať počet kopírovaní údajov a obmedziť režijné náklady štandardnej cesty cez jadro operačného systému. Netmap rieši podobný problém prostredníctvom efektívneho prístupu k paketovým poliam a zdieľanej pamäte medzi sieťovým rozhraním a používateľským programom [9], [10].

Tieto riešenia sú dôležité pri návrhu kompletných softvérových smerovačov, pretože optimalizujú najmä príjem a odosielanie paketov. V tejto práci však nie sú predmetom experimentálneho porovnania, keďže merania sa zameriavajú na izolovanú výpočtovú operáciu LPM nad pripravenou dávkou IPv4 adries. DPDK a netmap preto slúžia len ako kontext k tomu, že vysokovýkonné smerovanie nezávisí iba od samotného vyhľadávania vo FIB, ale aj od celého reťazca spracovania paketov.

2.4 Porovnanie CPU a GPU pri paralelnom spracovaní

Centrálne procesory a grafické procesory predstavujú rozdielne výpočtové platformy. CPU je navrhnuté ako univerzálny procesor s dôrazom na nízku latenciu, efektívne riadenie programu a výkonné spracovanie menšieho počtu zložitejších vlákien. GPU je naopak orientované na vysokú priepustnosť pri veľkom počte jednoduchších paralelných operácií. Tento rozdiel je významný najmä pri úlohách, ktoré možno rozdeliť na veľké množstvo nezávislých výpočtov.

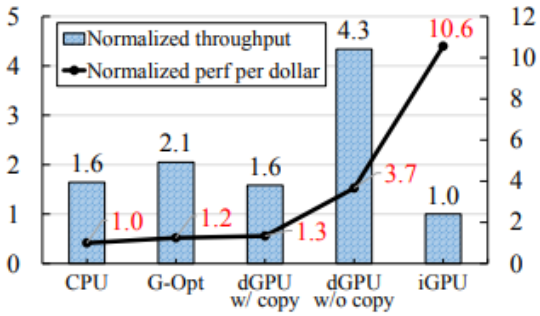
Pri sieťovom spracovaní sa grafické procesory skúmajú najmä ako akcelerátory výpočtovo náročných alebo dátovo paralelných častí dátovej roviny. Práce zamerané na GPU spracovanie paketov poukazujú na to, že prínos GPU nevychádza iba z výkonu jednotlivých jadier, ale najmä zo schopnosti skrývať pamäťovú latenciu pomocou veľkého počtu súbežne vykonávaných vlákien [4]. Takýto prístup môže byť výhodný pri dávkovom spracovaní, kde je k dispozícii dostatočne veľký počet nezávislých operácií.

Pri samostatnej grafickej karte však praktický prínos GPU neovplyvňuje iba čas samotného výpočtu, ale aj presun údajov medzi operačnou pamäťou systému a pamäťou grafického procesora. Tento problém dobre ilustruje štúdia APUNet, ktorej výsledky sú zobrazené na Obr. 1. Samostatná GPU bez započítania kopírovania údajov dosahuje pri IPv4 vyhľadávaní výrazne vyššiu normalizovanú priepustnosť než CPU riešenie. Po započítaní kopírovania údajov však jej priepustnosť výrazne klesá a približuje sa k optimalizovanej CPU konfigurácii [2].

Z hľadiska tejto práce je preto dôležité rozlišovať medzi výkonom samotného CUDA jadra a celkovým časom GPU spracovania vrátane prenosovej réžie. Pri LPM vyhľadávaní môže byť GPU výhodné najmä pri väčších dávkach, kde sa náklady na presun údajov rozložia medzi väčší počet nezávislých vyhľadávacích operácií. Pri menších dávkach môže naopak prenosová réžia výrazne znížiť alebo úplne prekryť výpočtový prínos GPU [2], [4].

CPU / Discrete GPU Platform	
CPU	Intel Xeon E5-2650 v2 (8 Core @ 2.6 GHz)
GPU	NVIDIA GTX980 (2048 Core @ 1.2 GHz)
RAM	64 GB (DIMM DDR3 @ 1333 MHz)
APU / Integrated GPU Platform	
CPU	AMD RX-421BD (4 Core @ 3.4 GHz)
GPU	AMD R7 Graphics (512 Core @ 800 MHz)
RAM	16 GB (DIMM DDR3 @ 2133 MHz)
OS	Ubuntu 14.04 LTS (Kernel 3.19.0-25-generic)

(a) CPU/GPU platforms for experiments



(b) IPv4 table lookup

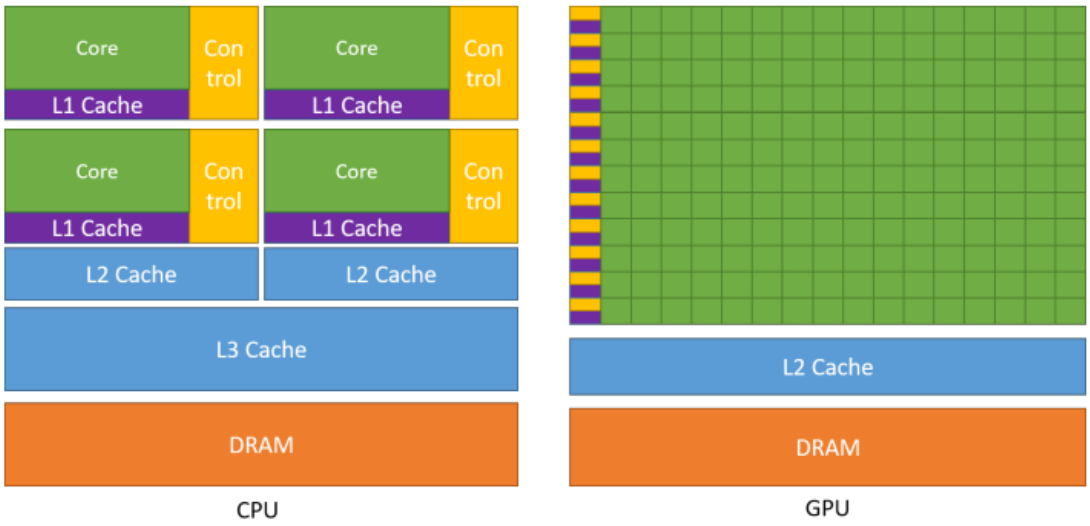
Obr. 1 - Porovnanie CPU a GPU riešení pri IPv4 vyhľadávaní podľa štúdie APUNet [2]

Z uvedeného prehľadu vyplýva, že urýchlenie smerovania možno dosahovať viacerými spôsobmi. Hardvérové riešenia typu ASIC a FPGA poskytujú špecializované spracovanie dátovej roviny, softvérové rámce DPDK a netmap znižujú réžiu tradičného sieťového zásobníka a GPU umožňuje presun dátovo paralelných výpočtov na samostatný akcelerátor. Pri GPU riešení však treba okrem samotného výpočtového výkonu zohľadniť aj pamäťové správanie algoritmu a náklady na presun údajov medzi CPU a GPU, ktoré môžu pri samostatnej grafickej karte významne ovplyvniť výsledný prínos [2], [4]. Tieto faktory vytvárajú základ pre nasledujúcu kapitolu zameranú na paralelný výpočtový model GPU a platformu CUDA.

3 Paralelizmus a GPGPU architektúry

Grafické procesory boli pôvodne určené predovšetkým na spracovanie grafických úloh, ktoré prirodzene obsahujú veľké množstvo podobných operácií vykonávaných nad rôznymi údajmi. Postupne sa však začali využívať aj na všeobecné výpočty, pri ktorých možno úlohu rozdeliť na veľký počet nezávislých častí. Tento prístup sa označuje ako všeobecné výpočty na grafických procesoroch (General-Purpose computing on Graphics Processing Units, GPGPU).

V oblasti sieťového spracovania je GPGPU zaujímavé najmä pri operáciách, ktoré možno vykonávať paralelne nad veľkou množinou vstupov. Ich efektívnosť však závisí od toho, či sú jednotlivé operácie dostatočne nezávislé, či majú podobný priebeh a či prístupy do pamäte zodpovedajú vlastnostiam grafickej architektúry [4]. Na tieto vlastnosti nadväzuje programovací model CUDA, ktorý umožňuje opisovať výpočty vykonávané veľkým počtom paralelných vlákien [11].



Obr. 2 - Zjednodušené porovnanie architektúry CPU a GPU podľa NVIDIA [11].

3.1 Architektúra CPU a GPU

CPU a GPU predstavujú rozdielne výpočtové architektúry. CPU je optimalizované na nízku latenciu, zložitejšie riadenie programu a efektívne spracovanie menšieho počtu vlákien. GPU je navrhnuté na vysokú priepustnosť pri veľkom počte jednoduchších paralelných operácií, preto väčšiu časť hardvérových prostriedkov venuje výpočtovým jednotkám [11]. Tento rozdiel je jasne viditeľný na porovnávacej schéme na Obr. 2.

Z pohľadu výpočtového modelu CPU spracúva menší počet zložitejších vlákien, ktoré môžu vykonávať rozdielne časti programu. GPU využíva model SIMT (Single Instruction, Multiple Threads), pri ktorom skupiny vlákien vykonávajú rovnakú inštrukciu nad rôznymi údajmi. V architektúrach NVIDIA sa základná skupina takto vykonávaných vlákien označuje ako warp [11].

Model SIMT je výhodný vtedy, keď jednotlivé vlákna vykonávajú podobný postup výpočtu. Ak sa však vlákna v rámci jedného warpu rozdelia do rozdielnych vetiev programu, vzniká divergencia vetvenia (warp divergence). V takom prípade sa jednotlivé vetvy vykonávajú postupne, čím sa znižuje efektívne využitie paralelizmu. Pri sieťových algoritmoch je preto vhodné navrhovať spracovanie tak, aby jednotlivé vlákna vykonávali čo najrovnomernejší postup a aby sa minimalizovali rozdiely v počte vykonaných operácií [4], [11].

Rozdiel medzi CPU a GPU sa prejavuje aj pri práci s pamäťou. CPU sa spolieha na rozsiahlu vyrovnávaciu pamäť a nízku latenciu prístupu k často používaným údajom. GPU sa naopak snaží vysokú latenciu prístupov do globálnej pamäte prekryť veľkým počtom aktívnych vlákien. Z tohto dôvodu môže byť GPU výhodné pri rozsiahlych dátovo paralelných úlohách, no výsledný výkon výrazne závisí od pravidelnosti pamäťových prístupov a od schopnosti udržať dostatočný počet súbežne vykonávaných vlákien [11].

3.2 Model programovania CUDA

Model programovania CUDA poskytuje rozhranie na využitie paralelnej architektúry GPU. Základným prvkom je CUDA jadro, čo je v anglickej terminológii označované ako CUDA kernel. V tejto práci sa pojem CUDA jadro používa na označenie výpočtovej funkcie spustenej z hostiteľskej časti programu, ktorá sa paralelne vykonáva veľkým počtom vlákien na grafickom procesore. Neoznačuje fyzické hardvérové jadro grafickej karty, jadro procesora ani jadro operačného systému. Hardvérové výpočtové jednotky grafickej karty sú v práci odlišované ako CUDA výpočtové jadrá alebo CUDA cores.

Vlákna CUDA jadra sú organizované do blokov a gridov. Blok predstavuje skupinu vlákien, ktoré môžu byť plánované na výpočtovej jednotke GPU a v prípade potreby spolupracovať prostredníctvom zdieľanej pamäte. Grid je množina blokov spustených pre jedno CUDA jadro. Takáto organizácia umožňuje mapovať dátovo paralelné úlohy na veľký počet vlákien, napríklad tak, že jedno vlákno spracuje jednu vstupnú IPv4 adresu pri LPM vyhľadávaní. Z hľadiska výkonu je dôležitá organizácia vlákien do warpov. Warp predstavuje základnú plánovaciu jednotku GPU a v architektúrach NVIDIA má zvyčajne 32 vlákien. Vlákna vo warpe vykonávajú rovnakú inštrukciu nad rôznymi údajmi. Ak sa však rozdelia do odlišných vetiev programu, vzniká divergencia vetvenia, pri ktorej sa jednotlivé vetvy

vykonávajú postupne. Tento jav znižuje efektívne využitie paralelizmu a je významný najmä pri algoritmoch s nerovnomerným priebehom výpočtu [11].

CUDA zároveň poskytuje viacúrovňový pamäťový model, ktorý zahŕňa najmä globálnu pamäť, zdieľanú pamäť a registre. Globálna pamäť má vyššiu latenciu, preto výkon výrazne ovplyvňuje spôsob, akým k nej vlákna pristupujú. Výhodné sú koaleskované prístupy, pri ktorých vlákna v rámci warpu pristupujú k susedným alebo vhodne usporiadaným adresám. Pri LPM algoritmoch preto nestačí posudzovať iba počet vyhľadávacích operácií, ale aj pravidelnosť pamäťových prístupov, mieru vetvenia a schopnosť udržať dostatočný počet aktívnych vlákien [11].

3.3 Typické úlohy vhodné pre GPU akceleráciu

GPU akcelerácia je vhodná najmä pre úlohy s vysokým stupňom dátového paralelizmu, nízkou mierou závislostí medzi operáciami a rovnomerným výpočtovým priebehom. Do tejto skupiny patrí aj dávkové LPM vyhľadávanie, pri ktorom možno jednotlivé cieľové IPv4 adresy spracúvať nezávisle. Každé CUDA vlákno môže vykonať jedno vyhľadanie nad pripravenou dátovou štruktúrou. Výsledný výkon však závisí od veľkosti dávky, pravidelnosti pamäťových prístupov, miery vetvenia a od réžie presunu údajov medzi CPU a GPU [4], [11].

Táto kapitola ukázala, že GPU architektúry založené na masívnom paralelizme a modeli SIMT poskytujú výkonnostný potenciál pre dátovo paralelné úlohy, medzi ktoré patrí aj vyhľadávanie v smerovacích tabuľkách. Zároveň však platí, že efektívne využitie GPU si vyžaduje špecifický návrh algoritmov a dôslednú prácu s pamäťovým modelom. Na tieto teoretické východiská nadväzuje nasledujúca kapitola, ktorá sa zameriava na existujúce GPU riešenia v sieťovom prostredí a ich identifikované obmedzenia.

4 Súčasný stav riešení GPU v sieťovom prostredí

Na základe teoretických východísk paralelizmu a GPGPU architektúr je možné pristúpiť k hodnoteniu ich reálneho využitia v oblasti sieťového spracovania. Výskum v tejto oblasti sa v posledných rokoch sústreďuje na akceleráciu vybraných sieťových funkcií pomocou GPU, pričom vznikli viaceré experimentálne aj prakticky orientované riešenia softvérových smerovačov a rámcov na spracovanie paketov. Táto kapitola poskytuje stručný prehľad relevantnej literatúry a existujúcich implementácií GPU akcelerovaného spracovania paketov, pričom zároveň identifikuje hlavné technické problémy a obmedzenia, ktoré tieto prístupy sprevádzajú.

4.1 Prehľad literatúry a výskumu

Výskum GPU akcelerácie sieťového spracovania sa zameriava najmä na úlohy dátovej roviny, ktoré možno rozdeliť na veľké množstvo nezávislých operácií a spracúvať paralelne po dávkach. Tento prístup je relevantný najmä pri vyhľadávaní v smerovacích tabuľkách, klasifikácii paketov a ďalších opakovaných operáciách nad veľkým počtom vstupov.

PacketShader [13] ukázal, že GPU môže výrazne zvýšiť priepustnosť softvérového smerovania, ak sa pakety nespracúvajú jednotlivo, ale v dávkach. Dávkové spracovanie umožňuje lepšie využiť veľký počet paralelných GPU vlákien, zároveň však prináša dodatočnú latenciu spojenú s vytvorením dávky a prenosom údajov.

Práca [4] zdôrazňuje, že hlavný prínos GPU nespočíva iba vo výkone jednotlivých jadier, ale najmä v schopnosti skrývať pamäťovú latenciu pomocou veľkého počtu súbežne vykonávaných vlákien. Autori zároveň poukazujú na to, že výsledný výkon závisí od pamäťového správania algoritmu a od toho, či CPU riešenie nie je možné optimalizovať efektívnejším využitím cache a paralelizmu.

APUNet [2] sa zameriava na problém prenosovej réžie medzi CPU a GPU. Namiesto samostatnej grafickej karty využíva integrovanú GPU architektúru so zdieľanou pamäťou, čím obmedzuje náklady na kopírovanie údajov medzi hostiteľskou časťou systému a akcelerátorom.

Blink [12] rieši problém nerovnomerného spracovania na GPU. Jeho cieľom je obmedziť neefektívne vykonávanie spôsobené rozdielnym priebehom výpočtu medzi vláknami, najmä v prípadoch, keď jednotlivé pakety alebo vyhľadávacie operácie nevyžadujú rovnaký počet krokov.

4.2 Existujúce implementácie spracovania paketov na GPU

Existujúce GPU riešenia zvyčajne využívajú CUDA alebo podobný výpočtový model, v ktorom GPU vystupuje ako akcelerátor vybraných častí dátovej roviny. GPU pritom nenahrádza celý sieťový zásobník, ale preberá najmä výpočtovo opakované alebo dátovo paralelné operácie, napríklad vyhľadávanie v tabuľkách, klasifikáciu paketov alebo kontrolu pravidiel.

Spoločným znakom týchto riešení je dávkové spracovanie. Namiesto samostatného spracovania jedného paketu sa vytvorí väčšia množina vstupov, nad ktorou sa paralelne spustí GPU výpočet. Pri LPM vyhľadávaní možno tento model aplikovať tak, že jedno GPU vlákno spracuje jednu cieľovú IPv4 adresu. Keďže jednotlivé vyhľadávacie operácie sú navzájom nezávislé, tento model prirodzene zodpovedá dátovo paralelnému spracovaniu. Výsledný výkon však závisí od organizácie dátovej štruktúry a charakteru prístupov do pamäte [3], [4].

4.3 Identifikované problémy a nevýhody GPU riešení

Hlavnými obmedzeniami GPU riešení sú prenosová réžia, potreba dávkového spracovania, nerovnomerný priebeh výpočtu a nepravidelné pamäťové prístupy. Pri diskrétnej GPU môže presun údajov cez PCIe znížiť praktický prínos akcelerácie, najmä pri menších dávkach. Dávkovanie zároveň zvyšuje latenciu jednotlivých paketov. Ďalším problémom je divergencia vetvenia a rozdielny počet operácií medzi vláknami, čo znižuje efektívne využitie paralelizmu [4], [12], [13]. Z týchto dôvodov je potrebné GPU akceleráciu hodnotiť experimentálne a rozlišovať medzi časom samotného CUDA jadra a celkovým časom spracovania vrátane prenosov.

5 Zdôvodnenie výberu riešenia

Predchádzajúce kapitoly ukázali, že výkonnosť softvérového smerovania je v dátovej rovine výrazne ovplyvnená efektivitou vyhľadávania v smerovacích tabuľkách. Operácia vyhľadania najdlhšieho zhodného prefixu predstavuje vhodný predmet analýzy, pretože sa vykonáva pre každú cieľovú IP adresu samostatne a umožňuje skúmať vplyv dátovej štruktúry, pamäťového správania a výpočtovej platformy.

Cieľom tejto kapitoly je zhrnúť dôvody výberu LPM vyhľadávania ako skúmanej operácie, vymedziť rozsah navrhovaného riešenia a formulovať predpoklady, ktoré je možné následne overiť experimentálnym meraním.

5.1 Formulácia problému pre CUDA implementáciu

Skúmaný problém možno formulovať ako dávkové spracovanie cieľových IPv4 adries nad vopred pripravenou FIB tabuľkou. Pre každú adresu sa vykoná LPM vyhľadávanie a výsledkom je príslušná next-hop hodnota. Takéto vymedzenie umožňuje izolovane hodnotiť výpočtovú časť smerovacieho rozhodnutia bez zahrnutia príjmu paketov, úprav hlavičiek alebo odosielania cez sieťové rozhranie.

Z pohľadu CUDA je podstatné, že jednotlivé vyhľadávacie operácie sú nezávislé a možno ich mapovať na samostatné GPU vlákna. Výhodnosť takéhoto prístupu však závisí od veľkosti dávky, použitej dátovej štruktúry, charakteru pamäťových prístupov a prenosovej réžie medzi CPU a GPU.

5.2 Hypotézy a očakávané prínosy riešenia

Na základe teoretických východísk a vlastností implementovaných algoritmov boli formulované tieto hypotézy:

- H1: GPU-akcelerované LPM vyhľadávanie dosiahne vyššiu priepustnosť ako jeho CPU ekvivalent pri spracovaní veľkých dávok vstupných adries, vďaka masívnemu paralelizmu GPU.
- H2: Výkon GPU riešenia bude závisieť od veľkosti dávky. Pri malých dávkach môže celkový prínos GPU znížiť réžia spojená s prenosom dát a spustením CUDA jadra.
- H3: Výhodnosť CPU a GPU spracovania závisí od prefixovej lokality vstupnej prevádzky. Pri vysokej lokalite môže CPU profitovať z opakovaného využívania

menšej aktívnej časti dátovej štruktúry v cache, a naopak pri nízkej lokalite a dostatočne veľkej dávke môže byť výhodnejšie GPU spracovanie.

- H4: Hybridný DIR24/8 model založený na prefixovej lokalite a veľkosti dávky dokáže pre testované scenáre určiť výhodnejšiu z CPU alebo GPU vetvy. Avšak jeho praktická efektívnosť bude závisieť od réžie samotného rozhodovania.

Očakávaným prínosom práce je rozsiahle porovnanie CPU, GPU a hybridného spracovania LPM vyhľadávania, identifikácia vplyvu prefixovej lokality na výkon a experimentálne overenie jednoduchého rozhodovacieho modelu pre výber výpočtovej platformy.

5.3 Riziká a obmedzenia navrhovaného prístupu

Navrhnuté riešenie sa sústreďuje iba na LPM vyhľadávanie nad dávkami IPv4 adries. Nezahŕňa príjem paketov zo sieťového rozhrania, úpravu hlavičiek, aktualizáciu TTL, výpočet kontrolných súčtov ani odosielanie paketov. Výsledky preto vyjadrujú výkon vyhľadávacej časti dátovej roviny, nie celkovú priepustnosť reálneho smerovača.

Pri GPU implementáciách je významným obmedzením prenos údajov medzi operačnou pamäťou a pamäťou diskrétnej GPU. Preto boli v práci oddelene sledované výsledky samotného CUDA jadra a výsledky vrátane prenosovej réžie. Ďalším obmedzením je charakter vstupnej prevádzky, keďže vyššia prefixová lokalita môže zvýhodniť CPU spracovanie vďaka opakovanému využívaniu menšej aktívnej časti dátovej štruktúry v cache.

Hybridný model používa jednoduché rozhodovanie založené na vzorke vstupnej dávky. Pri nereprezentatívnej vzorke alebo pri odlišnom type prevádzky môže zvoliť menej výhodnú výpočtovú vetvu.

6 Metodika experimentálneho overenia

Táto kapitola vymedzuje rozsah praktického experimentu, použité vstupné dátové súbory, testované konfigurácie, metodiku merania a hardvérovo-softvérové prostredie. Cieľom metodiky je zabezpečiť porovnateľné vyhodnotenie CPU, GPU a hybridného spracovania operácie IPv4 Longest Prefix Match.

6.1 Rozsah experimentu a testované varianty

Praktická časť práce sa zameriava na experimentálne vyhodnotenie operácie IPv4 Longest Prefix Match ako výpočtovo významnej časti prepájania paketov v dátovej rovine smerovača. Cieľom experimentov je porovnať výkon CPU a GPU spracovania pri použití optimalizovaných dátových štruktúr DIR24/8 a MultiBit Trie a následne overiť hybridný model založený na dynamickom výbere CPU alebo GPU vetvy.

V experimentoch boli hodnotené tieto hlavné varianty riešenia:

- viacvláknové CPU implementácie algoritmov DIR24/8 a MultiBit Trie s využitím OpenMP,
- CUDA implementácie algoritmov DIR24/8 a MultiBit Trie,
- hybridná implementácia algoritmu DIR24/8, ktorá podľa veľkosti dávky a prefixovej lokality vyberá CPU alebo GPU spracovanie.

Lineárne CPU a GPU implementácie boli použité iba v počiatočnej fáze riešenia ako referenčné varianty na overenie korektnosti LPM výpočtu a funkčnosti CUDA prostredia. Hlavné výkonové porovnania práce preto vychádzajú z optimalizovaných implementácií DIR24/8 a MultiBit Trie.

Experimentálne vyhodnotenie nezahŕňa kompletné spracovanie paketov v reálnom smerovači. Vstupom riešenia je pripravená FIB tabuľka a dávka cieľových IPv4 adries, pričom výstupom je pole výsledných next-hop hodnôt určených pomocou LPM vyhľadávania. Pojem next-hop v práci označuje hodnotu nasledujúceho smerovacieho kroku, napríklad identifikátor ďalšieho smerovača alebo výstupného rozhrania.

6.2 Testovacie vstupné dáta a typy vstupnej prevádzky

6.2.1 Testovacie vstupné dáta

Pre potreby experimentov boli vytvorené vlastné testovacie vstupné dáta pozostávajúce zo smerovacích tabuliek a množín vstupných IPv4 adries určených na vyhľadávanie. Smerovacie tabuľky reprezentovali záznamy FIB a obsahovali prefixy spolu s príslušnými výstupnými hodnotami. Vstupné množiny IP adries predstavovali cieľové adresy paketov, pre ktoré sa vykonávala operácia LPM vyhľadávanie.

Základné syntetické vstupné dáta spolu s FIB tabuľkami boli vygenerované vlastným programom `dataset_generator_synthetic_FIB.cpp`, pričom bol použitý pevne definovaný seed 12345. Tým bola zabezpečená reprodukovateľnosť syntetických meraní. Vstupné IPv4 adresy syntetickej prevádzky boli generované rovnomerným náhodným výberom 32-bitových hodnôt z celého IPv4 adresného priestoru. Vstupné dáta enterprise a hotspot prevádzky boli následne vytvorené zo syntetických vstupov pomocou samostatného skriptu v jazyku Python podľa podielov uvedených v tabuľke 1.

Vo všetkých implementáciách boli IPv4 adresy reprezentované hodnotami typu uint32_t. Syntetické vstupy boli uložené vo formáte CSV, zatiaľ čo vstupné súbory enterprise a hotspot prevádzky boli uložené ako binárne súbory obsahujúce postupnosť 32-bitových IPv4 adries.

6.2.2 Typy vstupnej prevádzky

V práci boli použité tri typy vstupnej prevádzky: syntetická, enterprise a hotspot prevádzka. Pri všetkých troch typoch sa nemenil princíp LPM vyhľadávania ani formát výsledkov. Cieľom bolo overiť, ako miera prefixovej lokality vstupných adries ovplyvňuje výkonnosť jednotlivých LPM implementácií.

Prefixovou lokalitou sa v tejto práci rozumie miera, s akou vstupné IP adresy pri LPM vyhľadávaní smerujú do rovnakých alebo blízkych /24 prefixových oblastí. Vyššia lokalita znamená, že vyhľadávacie operácie opakovane pristupujú k menšej aktívnej časti dátovej štruktúry.

Použitie troch úrovní prefixovej lokality umožnilo overiť, či je výhodnosť CPU a GPU spracovania stabilná, alebo či sa mení podľa charakteru vstupnej prevádzky.

Tab. 1 - Opis troch použitých typov prevádzky

Typ prevádzky	Spôsob vytvorenia
Syntetická	Rovnomerne náhodne generované IPv4 adresy z celého adresného priestoru
Enterprise	50 % adries z vybraných /24 prefixov, 35 % z vybraných /16 oblastí a 15 % náhodná zložka
Hotspot	95 % adries z ôsmich vybraných /24 prefixov a 5 % náhodná zložka

6.2.3 Merané konfigurácie

V experimentoch bolo pre každý typ prevádzky testovaných sedem rovnakých kombinácií veľkosti smerovacej tabuľky a počtu vstupných IPv4 adries. Rovnaké konfigurácie boli použité pre syntetickú, enterprise aj hotspot prevádzku, aby bolo možné porovnať vplyv charakteru vstupných dát pri zachovaní rovnakej veľkosti FIB a rovnakej veľkosti spracovávanej dávky.

Tab. 2 - Zoznam testovaných konfigurácií vstupných dát

Veľkosť FIB	Počet IPv4 adries
10 000 záznamov	200 000 adries
20 000 záznamov	400 000 adries
40 000 záznamov	800 000 adries
50 000 záznamov	1 000 000 adries
50 000 záznamov	1 500 000 adries
50 000 záznamov	2 000 000 adries
50 000 záznamov	3 000 000 adries

Pri CPU implementáciách boli pôvodne testované viaceré počty vlákien:

- 1 vlákno
- 2 vlákna
- 4 vlákna
- 6 vlákien
- 12 vlákien

Vo finálnych porovnaniach bola hlavná pozornosť venovaná 12-vláknovej konfigurácii, keďže zodpovedala maximálnemu počtu logických procesorov použitého CPU a predstavovala najvýkonnejší testovaný viacvláknový CPU variant.

6.3 Metodika CPU meraní

CPU implementácie algoritmov DIR24/8 a MultiBit Trie boli testované jednotnou metodikou merania. Vo finálnych porovnaniach bolo vyhľadávanie paralelizované pomocou OpenMP s využitím 12 CPU vlákien, čo zodpovedá maximálnemu počtu logických procesorov použitého CPU. Pre každú kombináciu algoritmu, typu prevádzky a meranej konfigurácie bolo vykonaných 5 zahrievacích a 10 meraných behov.

Pri oboch algoritmoch bol samostatne sledovaný čas zostavenia dátovej štruktúry. Tento čas predstavoval inicializačný alebo rekonfiguračný náklad smerovacej tabuľky a nebol zahrnutý do hlavnej metriky priepustnosti, pretože v ustálenom stave smerovača sa už zostavená dátová štruktúra opakovane používa na spracovanie vstupných dávok.

Po zostavení dátovej štruktúry bolo pri algoritmoch DIR24/8 aj MultiBit Trie merané izolované steady-state vyhľadávanie nad pripravenou dávkou IPv4 adries. Zároveň bol pri oboch implementáciách použitý model dávkového CPU spracovania, ktorý do meraného času zahŕňal skopírovanie vstupnej dávky do vopred alokovaného vstupného poľa, paralelné vykonanie LPM vyhľadávania a zápis výsledných next-hop hodnôt do výstupného zásobníka. Tento model bol navrhnutý tak, aby okrem samotného vyhľadávania zohľadňoval aj základnú režijnú prácu spojenú so spracovaním pripravenej dávky adries, nepredstavuje však meranie kompletného reťazca spracovania paketov v reálnom smerovači.

Ako hlavná metrika CPU výkonu bola pri oboch algoritmoch použitá priepustnosť odvodená z času dávkového CPU spracovania. Vo výstupných súboroch CSV je táto metrika z technických dôvodov označená pôvodným názvom `router_batch_throughput_mpps`. Izolovaný čas vyhľadávania a čas zostavenia dátovej štruktúry boli zachované ako doplnkové metriky umožňujúce analyzovať samotné vlastnosti algoritmov.

Z meraných behov boli vypočítané priemerné aj mediánové hodnoty, pričom v hlavných výkonových porovnaniach práce bola použitá priepustnosť odvodená z mediánového času spracovania dávky. Kontrolný súčet výsledných next-hop hodnôt bol použitý na overenie korektnosti a konzistentnosti výpočtu medzi opakovanými behmi a jednotlivými implementáciami.

Pri vyhodnotení hybridného riešenia bola navyše použitá CPU referenčná vetva algoritmu DIR24/8 implementovaná priamo v rovnakom CUDA meracom programe ako hybridný model. Táto referenčná hodnota bola použitá na metodicky korektnejšie porovnanie

samotného CPU spracovania s CPU vetvou hybridnej implementácie, keďže obe merania využívali rovnakú zostavenú dátovú štruktúru a rovnaké vykonávacie prostredie.

6.4 Metodika GPU meraní

GPU implementácie algoritmov DIR24/8 a MultiBit Trie boli testované jednotnou metodikou. Pre každú kombináciu algoritmu, typu prevádzky a meranej konfigurácie sa najskôr jednorazovo zostavila príslušná dátová štruktúra, transformovala sa do reprezentácie vhodnej pre GPU a nahrala sa do pamäte grafickej karty. Tieto inicializačné operácie boli sledované samostatne a neboli zahrnuté do hlavnej metriky priepustnosti vyhľadávania.

Pre každú konfiguráciu bolo následne vykonaných 5 zahrievacích behov a 10 meraných behov. V rámci jedného behu sa vstupná dávka IPv4 adries preniesla do pamäte GPU jedenkrát, CUDA jadro sa nad rovnakou dávkou spustilo 20-krát a výsledné next-hop hodnoty sa následne preniesli späť do operačnej pamäte jedenkrát. Zahrievacie behy mali rovnakú štruktúru, ale neboli zahrnuté do výsledných štatistík.

Jedno spustenie CUDA jadra spracovalo celú vstupnú dávku, pričom každá vstupná IPv4 adresa predstavovala jednu samostatnú LPM operáciu. Priemerný čas vykonania jedného CUDA jadra nad jednou vstupnou dávkou bol v každom meranom behu vypočítaný ako celkový čas dvadsiatich spustení CUDA jadra vydelený hodnotou 20. Takéto opakované spúšťanie umožnilo stabilnejšie merať veľmi krátke časy vykonania jadier CUDA.

Pri GPU meraniach boli rozlišované dve hlavné výkonové metriky. Prvou bol čas samotného CUDA jadra, ktorý vyjadroval samotný čas vykonania LPM výpočtu na GPU. Druhou bol čas spracovania vrátane prenosov dát, ktorý zahŕňal jeden prenos vstupnej dávky do GPU pamäte, priemerný čas jedného spracovania dávky CUDA jadrom a jeden prenos výsledkov späť do operačnej pamäte. Táto metrika preto lepšie reprezentovala praktické použitie GPU vetvy pri spracovaní jednej vstupnej dávky.

Z desiatich meraných behov boli vypočítané priemerné aj mediánové hodnoty. V hlavných porovnaniach CPU a GPU implementácií bola použitá priepustnosť odvodená z mediánového času spracovania dávky. Priepustnosť bola vyjadrená v miliónoch LPM operácií za sekundu, pričom počet spracovaných IP adries v dávke bol vydelený časom jedného spracovania celej dávky. Kontrolný súčet výsledných next-hop hodnôt slúžil na overenie korektnosti a konzistentnosti výpočtu.

6.5 Metodika hybridných meraní

Hybridná implementácia bola vytvorená pre algoritmus DIR24/8 ako samostatný merací program, ktorý pred spracovaním celej vstupnej dávky rozhoduje, či sa LPM vyhľadávanie vykoná na CPU alebo na GPU.

Základom rozhodovania bola /24 prefixová lokalita vypočítaná zo vzorky vstupných IP adries. Zo vstupnej dávky sa analyzovalo prvých 25 % vstupnej dávky, najviac však prvých 32 768 (2^{15}) adries. Takto zvolená veľkosť vzorky predstavovala kompromis medzi dostatočne spoľahlivým odhadom prefixovej lokality a režijným časom rozhodovania. Percentuálne obmedzenie zabezpečilo, že pri menších dávkach nebola analyzovaná neprimerane veľká časť vstupu, zatiaľ čo horný limit 32 768 adries obmedzil čas analýzy pri veľkých dávkach. Z každej analyzovanej IP adresy sa získal /24 prefix operáciou posunu o 8 bitov. Následne sa určil počet unikátnych /24 prefixov a vypočítala sa metrika:

$$\boldsymbol{avg_ips_per_unique24 = \frac{počet\ analyzovaných\ IP\ adries}{počet\ unikátnych\ /24\ prefixov}}$$

Vyššia hodnota tejto metriky znamenala vyššiu prefixovú lokalitu, teda väčšiu koncentráciu vyhľadávacích požiadaviek do menšieho počtu /24 oblastí.

Vo finálnej implementácii boli použité empiricky zvolené prahové hodnoty prefixovej lokality a hranice veľkosti dávky. Tieto hranice boli určené na základe priebežných meraní v použitom experimentálnom prostredí a slúžili ako jednoduché rozhodovacie pravidlo. Cieľom hybridného modelu nebolo navrhnúť všeobecne optimálny klasifikátor prevádzky, ale overiť, či kombinácia veľkosti dávky a prefixovej lokality dokáže rozlíšiť prípady, v ktorých je výhodnejšie CPU alebo GPU spracovanie. Zvolené prahy preto treba chápať ako experimentálne nastavenie platné pre použitý hardvér, implementácie a testované typy vstupných dát.

Pri hodnote menšej ako 2,0 bola prevádzka považovaná za nízko lokálnu. V tomto prípade sa dávka s najviac 200 000 IP adresami spracovala na CPU, pretože pri tejto veľkosti dávky nebola dávka dostatočne veľká na prekrytie režijných nákladov GPU spracovania. Väčšie nízko lokálne dávky boli spracované na GPU.

Pri hodnote lokality väčšej alebo rovnej 15,0 bola dávka považovaná za vysoko lokálnu a spracovala sa na CPU bez ohľadu na svoju veľkosť. Pri lokalite v intervale od 2,0 do 15,0 bola dávka považovaná za stredne lokálnu. V tomto prípade sa dávky s najviac 800 000 IP adresami spracovali na CPU a dávky od 800 001 IP adries na GPU.

V experimentoch boli vyhodnocované tri skupiny výstupov. Prvou bol hybrid bez započítania rozhodovacej réžie, pri ktorom sa meral iba čas vykonania už zvolenej CPU alebo

GPU vetvy. Druhou bol hybrid so započítaním oneskorenia vplyvom rozhodovacej réžie, pri ktorom sa do výsledného času zahrnula aj analýza prefixovej lokality a výber platformy. Treťou bola CPU referenčná vetva spustená v rovnakom hybridnom meracom programe, ktorá umožnila korektnejšie porovnanie samostatného CPU spracovania s CPU vetvou zvolenou hybridným rozhodovaním. Pri vyhodnotení hybridného modelu bola priepustnosť jednotlivých režimov rovnako odvodená z mediánového času spracovania dávky.

6.6 Sledované metriky

Pri meraniach boli sledované metriky umožňujúce oddeliť výkon samotného LPM vyhľadávania od režijných nákladov spojených so zostavením dátových štruktúr, prenosom dát a hybridným rozhodovaním. Pri CPU implementáciách sa hodnotil čas zostavenia dátovej štruktúry, izolovaný čas vyhľadávania v ustálenom stave, čas dávkového CPU spracovania a z neho odvodená priepustnosť.

Pri GPU implementáciách sa samostatne sledoval čas prípravy a prenosu dátovej štruktúry do pamäte GPU, čas prenosu vstupnej dávky, čas prenosu výsledkov späť do operačnej pamäte, čas samotného CUDA jadra a celkový čas GPU spracovania vrátane prenosovej réžie. Z týchto hodnôt bola odvodená priepustnosť samotného CUDA jadra aj priepustnosť realistickejšieho GPU režimu vrátane prenosov.

Pri hybridnom modeli DIR24/8 sa navyše sledoval čas analýzy prefixovej lokality a výberu CPU alebo GPU vetvy. Vyhodnocovaná bola priepustnosť hybridu bez započítania rozhodovacej réžie, priepustnosť hybridu s touto réžiou a priepustnosť referenčnej CPU vetvy meranej v rovnakom programe ako hybridná implementácia.

Priepustnosť bola vyjadrená v miliónoch LPM operácií za sekundu (Mpps), pričom jedna LPM operácia zodpovedala spracovaniu jednej vstupnej IPv4 adresy. Kontrolný súčet výsledných next-hop hodnôt slúžil na overenie korektnosti výsledkov medzi CPU, GPU a hybridnými variantmi. Výsledky boli exportované do súborov CSV a následne spracované v jazyku Python do tabuľkovej a grafickej podoby.

6.7 Použitý hardvér a softvér

Všetky experimenty boli realizované na notebooku Lenovo Legion Slim 7 (model S7 15ACH6) s operačným systémom Windows 11 Home. Zvolené zariadenie sa dá považovať za bežne dostupný výkonný používateľský hardvér, čo umožňuje hodnotiť možnosti GPU akcelerácie aj mimo serverových alebo dátových centier.

Použitá konfigurácia umožnila porovnať viacvláknové CPU spracovanie s CUDA akceleráciou na bežne dostupnom hardvéri. Pri interpretácii výsledkov je potrebné zohľadniť, že

použitá grafická karta je diskrétna GPU, a preto sa pri praktickom GPU spracovaní uplatňuje aj réžia prenosu dát medzi operačnou pamäťou systému a pamäťou GPU.

Tab. 3 - Špecifikácia použitého hardvéru

Komponent	Použitá konfigurácia
Testovacie zariadenie	Lenovo Legion Slim 7, model S7 15ACH6
Operačný systém	Windows 11 Home
CPU	AMD Ryzen 5 5600H, 6 jadier / 12 logických procesorov, L2 cache 3 MB, L3 cache 16 MB
GPU	NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4 GB GDDR6
Operačná pamäť	16 GB RAM
CPU implementácie	C++17, Microsoft Visual C++, optimalizácia /O2, OpenMP
GPU implementácie	CUDA C++, NVIDIA CUDA Toolkit 12.4, nvcc
Spracovanie výsledkov	Python 3.11.3, Matplotlib
Profilovanie GPU	NVIDIA Nsight Compute

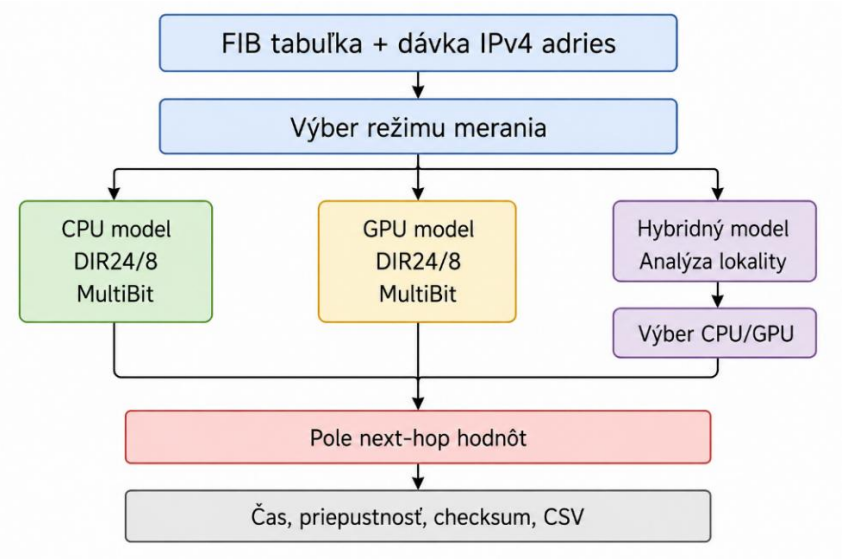
Použitý procesor AMD Ryzen 5 5600H je mobilný procesor so 6 fyzickými jadrami a 12 logickými vláknami. Disponuje 3 MB vyrovnávacej pamäte L2 a 16 MB vyrovnávacej pamäte L3. V experimentoch bola pri CPU implementáciách použitá 12-vláknová konfigurácia, ktorá zodpovedá maximálnemu počtu logických procesorov použitého CPU.

Použitá grafická karta NVIDIA GeForce RTX 3050 Ti Laptop GPU je diskrétna mobilná GPU architektúry Ampere s 4 GB pamäte GDDR6. Použitý model obsahuje 2560 hardvérových CUDA výpočtových jadier, 20 RT jadier a 80 Tensor jadier. V práci je potrebné tieto hardvérové CUDA výpočtové jadrá odlišovať od pojmu CUDA jadro, ktorý označuje výpočtovú funkciu spúšťanú na GPU (v anglickom jazyku CUDA kernel). Pri interpretácii výsledkov je zároveň potrebné zohľadniť, že ide o samostatnú diskrétnu GPU, a preto celkový čas GPU spracovania ovplyvňuje aj prenosová réžia medzi operačnou pamäťou systému a pamäťou grafickej karty.

7 Návrh a implementácia riešenia

Táto kapitola opisuje architektúru a implementačnú realizáciu riešenia použitého na experimentálne porovnanie LPM vyhľadávania na CPU a GPU. Implementované riešenie nepredstavuje kompletný softvérový smerovač, ale model rozhodovacej operácie dátovej roviny, pri ktorej sa pre dávku cieľových IPv4 adries určujú príslušné next-hop hodnoty podľa najdlhšej zhodnej prefixovej položky vo FIB tabuľke.

Riešenie bolo vytvorené vo forme samostatných meracích programov pre CPU, GPU a hybridné spracovanie. Hlavné výkonové porovnania boli založené na optimalizovaných dátových štruktúrach DIR24/8 a MultiBit Trie. Hybridný model bol implementovaný iba pre algoritmus DIR24/8, pretože pri priebežných meraniach vykazoval výrazne odlišné správanie na CPU a GPU v závislosti od prefixovej lokality vstupnej prevádzky.



Obr. 3 - Blokový diagram celého implementovaného riešenia

7.1 Architektúra a pracovný tok riešenia

Implementované riešenie pozostáva zo spracovania dávky cieľových IPv4 adries nad pripravenou FIB tabuľkou. Každá adresa predstavuje jedno nezávislé LPM vyhľadávanie a výsledkom je pole next-hop hodnôt uložených v rovnakom poradí ako vstupné adresy. Riešenie pozostáva z troch modelov: viacvláknového CPU spracovania pomocou OpenMP, GPU spracovania pomocou CUDA jadier a hybridného DIR24/8 modelu, ktorý pred spracovaním dávky vyberá CPU alebo GPU vetvu. Vo všetkých prípadoch sa najskôr zostaví príslušná dátová štruktúra a následne sa nad vstupnou dávkou vykoná LPM vyhľadávanie.

7.2 Implementácia DIR24/8

DIR24/8 využíva rozdelenie IPv4 adresy na horných 24 bitov a dolných 8 bitov. Horných 24 bitov slúži ako index do primárnej tabuľky veľkosti 2^{24} položiek. Prefixy dĺžky najviac /24 sa ukladajú priamo do primárnej tabuľky. Ak sa vo FIB nachádza prefix dlhší ako /24, príslušná položka primárnej tabuľky odkazuje na sekundárny blok so 256 položkami indexovanými dolnými ôsmimi bitmi adresy.

Výhodou tejto štruktúry je predvídateľný počet pamäťových prístupov a veľmi rýchle vyhľadávanie. Pre jednu vstupnú IPv4 adresu je potrebný jeden prístup do primárnej tabuľky a najviac jeden doplnkový prístup do sekundárneho bloku. Táto vlastnosť je vhodná pre CPU aj GPU implementáciu, pretože obmedzuje počet vykonávaných krokov pri každom vyhľadávaní.

Nevýhodou je vyššia pamäťová náročnosť spôsobená veľkou primárnou tabuľkou. Výkon algoritmu preto môže byť výrazne ovplyvnený rozložením vstupných adries. Pri vysokej prefixovej lokalite sa opakovane pristupuje k menšej aktívnej časti tabuľky, zatiaľ čo pri rozptýlených vstupoch sú prístupy rozložené naprieč rozsiahlejšou pracovnou množinou.

7.3 MultiBit Trie

Druhou optimalizovanou dátovou štruktúrou bol MultiBit Trie so stride hodnotou 4. Každá úroveň stromu spracúva štyri bity IPv4 adresy, takže jeden uzol obsahuje 16 možných vetiev. Pri 32-bitovej IPv4 adrese môže vyhľadávanie prejsť najviac ôsmimi úrovňami stromu.

V implementácii boli uzly uložené v lineárnej pamäťovej reprezentácii a väzby medzi nimi boli realizované pomocou indexov. Takéto usporiadanie bolo vhodné pre CPU aj GPU variant, pretože nevyžadovalo použitie dynamických ukazovateľov pri samotnom spracovaní vyhľadávacej operácie.

V porovnaní s DIR24/8 má MultiBit Trie kompaktnejšiu reprezentáciu, vyhľadávanie však môže vyžadovať viac postupných pamäťových prístupov. Z tohto dôvodu bol algoritmus použitý na porovnanie výkonového správania odlišnej optimalizovanej LPM štruktúry, zatiaľ čo hybridný model bol realizovaný iba nad algoritmom DIR24/8.

7.4 CPU implementácie a paralelizácia pomocou OpenMP

CPU implementácie algoritmov DIR24/8 a MultiBit Trie boli vytvorené v jazyku C++17. Pri oboch algoritmoch sa dátová štruktúra zostavila zo vstupnej FIB tabuľky pred meraním a počas spracovania dávok sa už iba čítala. Paralelizácia sa preto týkala samotného vykonávania LPM vyhľadávania nad vstupnými IPv4 adresami.

Paralelizácia bola realizovaná pomocou OpenMP nad slučkou prechádzajúcou vstupné pole adries. Jedna iterácia slučky predstavovala jedno nezávislé LPM vyhľadanie a zápis jednej next-hop hodnoty do výstupného poľa. Princíp paralelizácie možno zjednodušene vyjadriť nasledovne:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < static_cast<int>(ips.size()); ++i) {
    out_next_hop[static_cast<std::size_t>(i)] =
        dir.lookup(ips[static_cast<std::size_t>(i)]);
}
```

Vo finálnych meraniach bolo použitých 12 OpenMP vlákien, čo zodpovedalo maximálnemu počtu logických procesorov použitého CPU. Statické plánovanie rozdelilo vstupné adresy medzi vlákna bez potreby dynamického prideľovania práce počas vykonávania. Keďže jednotlivé vyhľadávacie operácie nemenili spoločnú dátovú štruktúru a každá iterácia zapisovala výsledok do odlišnej pozície výstupného poľa, nebola potrebná dodatočná synchronizácia.

Obe CPU implementácie používali rovnaký spôsob paralelizácie a rovnaký model spracovania vstupnej dávky. Rozdiel medzi nimi preto vyplýval predovšetkým z použitej dátovej štruktúry a z charakteru pamäťových prístupov.

7.5 CUDA implementácie a konfigurácia CUDA jadier

GPU implementácie algoritmov DIR24/8 a MultiBit Trie boli vytvorené v jazyku CUDA C++. Pri oboch algoritmoch sa dátová štruktúra zostavila na CPU, previedla do lineárnej reprezentácie a pred meraním jednorazovo preniesla do globálnej pamäte GPU. Počas meraných behov sa už neprenášala samotná vyhľadávacia štruktúra, ale iba vstupná dávka IPv4 adries a výsledné pole next-hop hodnôt. Obe CUDA implementácie teda používali rovnaký pamäťový a prenosový model, čím bola zachovaná ich metodická porovnateľnosť.

Pri oboch CUDA implementáciách bol použitý dátovo-paralelný model, v ktorom jedno CUDA vlákno spracovalo jednu vstupnú IPv4 adresu. Každé vlákno načítalo adresu zo

vstupného poľa, vykonalo LPM vyhľadávanie v príslušnej dátovej štruktúre a zapísalo výslednú next-hop hodnotu na rovnakú pozíciu výstupného poľa. Vyhľadávacie operácie boli navzájom nezávislé, a preto medzi vláknami nebola potrebná synchronizácia.

Globálny index spracovávanej adresy bol v CUDA jadre určený spôsobom:

```
std::size_t idx =
static_cast<std::size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
if (idx >= ip_count) {
return;
}
const uint32_t ip = ips[idx];

// vykonanie LPM vyhľadania pre jednu IPv4 adresu // ...

out_next_hop[idx] = result;
```

V CUDA jadre DIR24/8 vlákno získalo výsledok prístupom do primárnej tabuľky a podľa potreby aj do sekundárneho bloku. V CUDA jadre MultiBit Trie vlákno postupne prechádzalo lineárne uloženými uzlami podľa štvorbitových častí IPv4 adresy. Rozdiel medzi GPU variantmi preto vyplýval najmä z počtu a charakteru pamäťových prístupov potrebných na jedno LPM vyhľadávanie.

Pri oboch CUDA algoritmoch bola použitá veľkosť bloku 256 vlákien. Počet blokov bol určený podľa veľkosti vstupnej dávky tak, aby bol pre každú vstupnú adresu vytvorený práve jeden potenciálny výpočtový prvok:

$$\textbf{počet blokov} = \left\lceil \frac{\textbf{počet vstupných IP adries}}{\mathbf{256}} \right\rceil$$

Ak počet vstupných adries nebol násobkom veľkosti bloku, posledný blok obsahoval aj vlákna bez priradenej adresy. Tieto vlákna boli ukončené kontrolou hranice vstupného poľa ešte pred vykonaním vyhľadávania.

Takto navrhnutá CUDA implementácia umožnila paralelne spracovať celú dávku vstupných adries, pričom rozdiel medzi algoritmami DIR24/8 a MultiBit Trie vyplýval najmä z ich pamäťovej reprezentácie a počtu pamäťových prístupov potrebných na jedno LPM vyhľadanie.

7.6 Hybridný DIR24/8 model

Hybridný model bol implementovaný ako samostatný merací program pre algoritmus DIR24/8. V jednom programe obsahoval 12-vláknovú CPU vetvu realizovanú pomocou OpenMP a GPU vetvu realizovanú pomocou CUDA. Obe vetvy spracúvali rovnaký typ vstupu a vytvárali pole next-hop hodnôt v rovnakom formáte, čo umožnilo vybrať jednu z nich pre spracovanie celej vstupnej dávky.

Po načítaní FIB tabuľky bola vytvorená CPU reprezentácia dátovej štruktúry DIR24/8 a zároveň reprezentácia potrebná pre GPU vetvu. Obe štruktúry boli pripravené pred opakovaným spracovaním dávok. CPU vetva následne vykonávala viacvláknové LPM vyhľadávanie nad pripravenou CPU štruktúrou. GPU vetva zabezpečila prenos vstupnej dávky do pamäte grafickej karty, vykonanie CUDA jadra a prenos výsledného poľa späť do operačnej pamäte systému.

Pred vykonaním vyhľadávania hybridný model analyzoval vzorku vstupných IPv4 adries a určil mieru ich /24 prefixovej lokality. Spolu s veľkosťou vstupnej dávky táto metrika určovala, či sa na spracovanie použije CPU alebo GPU vetva. Konkrétna definícia metriky, veľkosť analyzovanej vzorky a rozhodovacie hranice boli uvedené v metodike hybridných meraní v kapitole 6.

Hybridný model nepredstavoval súčasné paralelné spracovanie jednej dávky na CPU a GPU. Výsledkom rozhodovacej časti bola vždy jediná platforma, ktorá spracovala celú vstupnú dávku. Takéto riešenie umožnilo overiť, či je možné výberom platformy reagovať na zmeny prefixovej lokality.

Súčasťou hybridného programu bola aj samostatná referenčná CPU vetva. Dôvodom jej zahrnutia bolo, že pri počiatočnom porovnaní CPU vetvy hybridného režimu so samostatne meranou CPU implementáciou DIR24/8 sa napriek rovnakej metodike objavili výrazné výkonnostné rozdiely. Tieto rozdiely mohli súvisieť s odlišným kontextom vykonania merania, ktorý ovplyvňuje stav pamäťovej hierarchie CPU a tým aj namerané hodnoty. Referenčná CPU vetva preto umožnila porovnať CPU spracovanie a hybridný režim v rámci rovnakého programu a rovnakých podmienok vykonania. Samostatná referenčná GPU vetva do hybridného programu doplnená nebola, pretože po zahrnutí prenosovej réžie sa výkon GPU vetvy hybridného režimu prakticky zhodoval so samostatne meranou GPU implementáciou DIR24/8.

Z implementačného hľadiska teda hybridný program obsahoval tri hodnotené režimy. Hybrid bez rozhodovacej réžie, zahŕňajúci iba spracovanie už vybranou CPU alebo GPU vetvou. Hybrid so započítanou rozhodovacou vetvou, ktorá zahŕňala navyše analýzu prefixovej lokality vstupnej dávky a výber platformy. A nakoniec už vyššie spomínanú referenčnú CPU vetvu.

7.7 Organizácia meracích programov a spracovanie výsledkov

Implementované riešenie bolo rozdelené do samostatných programov a pomocných skriptov podľa ich úlohy v experimentálnom reťazci. Toto rozdelenie umožnilo oddeliť generovanie vstupných údajov, meranie výkonu jednotlivých implementácií a následné spracovanie výsledkov.

CPU meracie programy boli kompilované v prostredí Microsoft Visual C++ s podporou štandardu C++17, optimalizáciou a zapnutým OpenMP. GPU a hybridná implementácia boli kompilované pomocou nástroja `nvcc` z balíka CUDA Toolkit. Presné kompilačné príkazy a zoznam použitých zdrojových programov sú uvedené v Dodatku A.

Jednotlivé meracie programy exportovali namerané hodnoty a kontrolné údaje do CSV súborov. Rozsah sledovaných metrík bol definovaný v kapitole 6. Následné spracovanie výsledkov bolo vykonané pomocou skriptov v jazyku Python, ktoré načítali výstupné CSV súbory, vypočítali priepustnosti z vybraných časových metrík a vytvorili grafické porovnania CPU, GPU a hybridných variantov.

Pri finálnych grafoch boli použité metriky zodpovedajúce praktickému porovnaniu jednotlivých platforiem: pri CPU implementáciách priepustnosť odvodená z času dávkového CPU spracovania a pri GPU implementáciách priepustnosť zahŕňajúca vykonanie CUDA jadra aj prenos vstupných a výstupných dát. Pri hybridnom modeli bolo osobitne vyhodnotené spracovanie bez rozhodovacej réžie a spracovanie vrátane tejto réžie.

Pri tvorbe a iteratívnej úprave meracích programov a pomocných skriptov boli použité nástroje generatívnej umelej inteligencie ako podporný konzultačný a implementačný nástroj. Ich použitie sa týkalo najmä skriptov v jazykoch C++, CUDA C++ a Python, metodických konzultácií a spracovania výsledkov. Podrobné deklarovanie použitia umelej inteligencie je uvedené v samostatnej časti za zoznamom literatúry.

8 Záverečné vyhodnotenie výsledkov

Táto kapitola vyhodnocuje výsledky experimentov zameraných na dávkové spracovanie IPv4 Longest Prefix Match operácií. Hodnotené sú optimalizované implementácie algoritmov DIR24/8 a MultiBit Trie realizované na CPU a GPU, pričom osobitná pozornosť je venovaná vplyvu veľkosti spracovávanej dávky a charakteru vstupnej prevádzky. Výsledky výkonového porovnania sú následne doplnené profilovacou analýzou CUDA implementácií a vyhodnotením hybridného modelu DIR24/8.

Pri hlavnom porovnaní CPU a GPU variantov boli použité metriky reprezentujúce spracovanie jednej pripravenej dávky IPv4 adries. Pri CPU implementáciách ide o priepustnosť odvodenú z času dávkového CPU spracovania, ktorý zahŕňa prípravu vstupnej dávky, paralelné LPM vyhľadávanie a zápis výsledkov. Pri GPU implementáciách ide o priepustnosť odvodenú z času spracovania zahŕňajúceho vykonanie CUDA jadra aj prenos vstupných a výstupných dát. Vo všetkých hlavných porovnaniach bola priepustnosť odvodená z mediánového času spracovania dávky.

Čas samotného CUDA jadra bol sledovaný ako doplnková metrika na analýzu výkonu výpočtu vykonávaného priamo na GPU. Pri praktickom porovnaní s CPU variantmi však nebol použitý ako hlavná metrika, pretože nezahŕňa prenosovú réžiu diskrétnej grafickej karty.

8.1 Porovnanie CPU a GPU implementácií

Výsledky ukázali, že najvýkonnejšia implementácia sa menila podľa charakteru vstupnej prevádzky a veľkosti spracovávanej dávky. Na prehľadné porovnanie boli vybrané tri reprezentatívne konfigurácie: najmenšia testovaná dávka, dávka s 1 000 000 adries a najväčšia dávka s 3 000 000 adries. Výsledky sú uvedené v Tab. 4.

Pri syntetickej prevádzke s nízkou prefixovou lokalitou bola vo všetkých reprezentatívnych konfiguráciách najvýkonnejšia CUDA implementácia MultiBit Trie. Pri enterprise prevádzke sa poradie zmenilo s veľkosťou dávky: pri najmenšej konfigurácii dosiahla najlepší výsledok CPU implementácia DIR24/8, zatiaľ čo pri väčších dávkach bola výhodnejšia CUDA implementácia DIR24/8. Pri hotspot prevádzke s vysokou prefixovou lokalitou dosahovala najvyššiu priepustnosť CPU implementácia DIR24/8.

Výsledky teda nepotvrdili jednoznačnú prevahu CPU ani GPU spracovania. GPU bolo výhodné najmä pri rozptýlených vstupoch a väčších dávkach, zatiaľ čo CPU DIR24/8 výrazne profitovalo z lokálnej prevádzky. Úplné grafické porovnania všetkých siedmich konfigurácií sú uvedené v Dodatku B.

Tab. 4 - Porovnanie priepustnosti implementácií pri reprezentatívnych konfiguráciách a typoch prevádzky

Typ	Konfigurácia	CPU DIR24/8 [Mpps]	CPU MultiBit Trie [Mpps]	CUDA DIR24/8 [Mpps]	CUDA MultiBit Trie [Mpps]
Syntetická	10K FIB / 200K IP	260,49	532,48	398,08	**575,11**
Syntetická	50K FIB / 1M IP	181,79	314,78	451,65	**522,17**
Syntetická	50K FIB / 3M IP	181,07	262,28	448,07	**532,15**
Enterprise	10K FIB / 200K IP	**1435,75**	733,81	540,80	611,55
Enterprise	50K FIB / 1M IP	581,01	445,21	**710,17**	690,86
Enterprise	50K FIB / 3M IP	413,14	410,04	**729,00**	721,98
Hotspot	10K FIB / 200K IP	**2056,56**	851,25	542,87	617,09
Hotspot	50K FIB / 1M IP	**1235,64**	607,55	727,52	715,30
Hotspot	50K FIB / 3M IP	**861,41**	594,66	749,00	750,34

Hodnoty priepustnosti sú odvodené z mediánového času spracovania dávky a zaokrúhlené na dve desatinné miesta.

Kompletné grafické porovnanie CPU a GPU implementácií pre syntetickú, enterprise a hotspot prevádzku je uvedené v Dodatku B, najmä na Obr. 19 až Obr. 29. Grafy so samostatným meraním CUDA jadra sú uvedené ako doplnkové, pretože nezahŕňajú prenosovú réžiu medzi CPU a GPU.

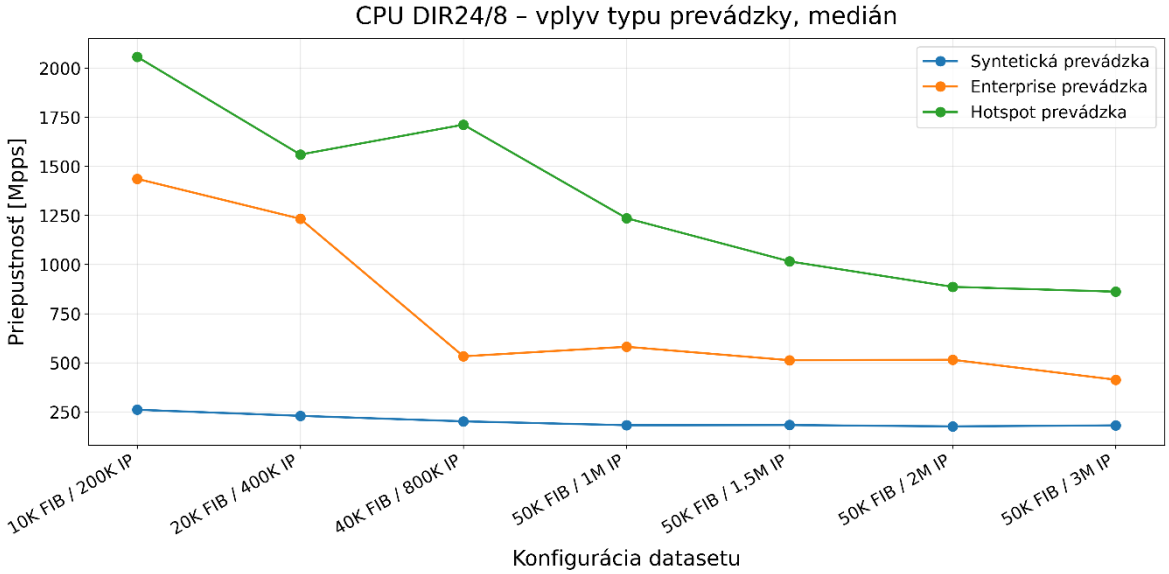
8.2 Vplyv typu prevádzky a prefixovej lokality

Rozdiely medzi implementáciami súviseli najmä s prefixovou lokalitou vstupných adries. Tá bola vyjadrená metrikou `avg_ips_per_unique24`, ktorá udáva priemerný počet analyzovaných adries pripadajúcich na jeden unikátny /24 prefix. Vyššia hodnota znamená, že vyhľadávania smerujú do menšej aktívnej časti dátovej štruktúry.

Najvýraznejší vplyv lokality sa prejavil pri CPU implementácii DIR24/8. Pri syntetickej prevádzke boli prístupy do rozsiahlej primárnej tabuľky rozptýlené, zatiaľ čo pri hotspot prevádzke sa opakovane využívala menšia množina /24 položiek. To vytvorilo priaznivejšie podmienky pre využitie pamäťovej hierarchie CPU a viedlo k výraznému zvýšeniu priepustnosti.

Tab. 5 - Nameraná prefixová lokalita testovaných typov prevádzky

Typ prevádzky	Rozsah hodnoty avg_ips_per_unique24	Charakter prefixovej lokality
Syntetická	1,001 – 1,001	Nízka
Enterprise	3,686 – 3,729	Stredná
Hotspot	18,930 – 20,583	Vysoká



Obr. 4 - Vplyv typu prevádzky na priepustnosť CPU implementácie DIR24/8

Toto správanie vysvetľuje aj výber algoritmu DIR24/8 pre hybridný model. Pri nízkej lokalite a väčších dávkach bola výhodnejšia GPU vetva, pri vysokej lokalite dosahovala výrazne lepšie výsledky CPU vetva. Enterprise prevádzka predstavovala prechodový scenár, v ktorom sa vhodnejšia platforma menila podľa veľkosti dávky. Tieto zistenia vytvorili základ pre rozhodovanie hybridného modelu, v ktorom nízka lokalita a väčšia dávka zvýhodňovali GPU vetvu, zatiaľ čo vysoká lokalita zvýhodňovala CPU vetvu algoritmu DIR24/8.

Doplnkové grafy zobrazujúce vplyv typu prevádzky na CPU a GPU implementácie sú uvedené v Dodatku B. Tieto grafy rozširujú hlavné vyhodnotenie o porovnania časov a priepustnosti jednotlivých algoritmov.

8.3 Profilovacia analýza pomocou Nsight Compute

Na vysvetlenie rozdielov medzi CUDA implementáciami bolo vykonané profilovanie pomocou nástroja NVIDIA Nsight Compute. Analyzované boli CUDA jadrá algoritmov DIR24/8 a MultiBit Trie pri syntetickej a hotspot prevádzke pre tri reprezentatívne veľkosti dávky: 200 000, 1 000 000 a 3 000 000 IPv4 adries. Profilovanie sa týkalo samotného vykonania CUDA jadra, preto dopĺňa hlavné výkonové porovnanie, v ktorom bola pri GPU zohľadnená aj prenosová réžia.

Tab. 6 - Súhrn profilovania CUDA jadier pomocou Nsight Compute

Algoritmus a typ prevádzky	Čas CUDA jadra: 200K / 3M	Miera zásahov L2 cache [%]	Využitie DRAM [%]	Long Scoreboard Stall [%]
DIR24/8 – syntetická	185,89 µs / 2,84 ms	11,97 – 14,44	36,70 – 39,20	89,63 – 93,81
DIR24/8 – hotspot	13,02 µs / 149,73 µs	50,89 – 52,26	64,24 – 85,97	87,59 – 100,64
MultiBit Trie – syntetická	52,42 µs / 1,83 ms	64,71 – 78,70	49,18 – 53,14	82,58 – 97,25
MultiBit Trie – hotspot	18,21 µs / 226,75 µs	53,48 – 55,27	46,42 – 60,21	65,12 – 67,04

Najvýraznejšia zmena sa prejavila pri algoritme DIR24/8. Pri syntetickej prevádzke boli prístupy do primárnej tabuľky rozptýlené, čo sa prejavilo nízkou mierou zásahov L2 cache a vysokým podielom čakania warpu na dáta z pamäťovej hierarchie. Pri hotspot prevádzke sa vstupné adresy sústreďovali do menšieho počtu /24 oblastí, vďaka čomu sa miera zásahov L2 cache zvýšila približne na polovicu všetkých prístupov. Súčasne sa výrazne skrátil čas vykonania CUDA jadra.

Pri algoritme MultiBit Trie bola závislosť od typu prevádzky odlišná. Hotspot prevádzka síce znížila čas vykonania CUDA jadra a obmedzila čakanie na pamäťové operácie, ale neprejavila sa zvýšením miery zásahov L2 cache oproti syntetickej prevádzke. Výkon tejto štruktúry bol preto ovplyvnený aj charakterom postupného prechodu uzlami, nielen koncentráciou vstupných adries do rovnakých /24 oblastí.

Profilovanie potvrdilo, že výkon CUDA implementácií nezávisí iba od veľkosti dávky, ale aj od toho, či vstupné adresy vedú k opakovanému využívaniu rovnakej časti dátovej štruktúry alebo k rozptýleným prístupom do pamäte. Pri DIR24/8 sa vplyv prefixovej lokality prejavil najjednoznačnejšie, čo zodpovedá aj výsledkom CPU meraní, pri ktorých vysoko lokálna hotspot prevádzka výrazne zvýhodnila CPU variant tohto algoritmu.

Z uvedeného dôvodu bol pre experimentálne hybridné riešenie zvolený algoritmus DIR24/8. Pri nízkej lokalite a väčších dávkach bolo možné využiť GPU spracovanie, zatiaľ čo pri vysokej lokalite dosahovala výrazne výhodnejšie výsledky CPU vetva. Hybridný model preto overuje, či možno tieto rozdiely prakticky využiť rozhodovaním podľa prefixovej lokality a veľkosti vstupnej dávky.

Výsledky samotného CUDA jadra ukazujú, že CUDA implementácie majú vysoký výpočtový potenciál. Pri praktickom použití diskrétnej GPU však tento potenciál znižujú prenosy medzi operačnou pamäťou systému a pamäťou GPU.

Úplné výstupy profilovania CUDA jadier z nástroja Nsight Compute sú uvedené v Dodatku D. Doplnkové grafy meraní ČJ sú uvedené v Dodatku B.

8.4 Vyhodnotenie hybridného DIR24/8 modelu

Hybridný model bol vyhodnotený pre algoritmus DIR24/8, pri ktorom sa v predchádzajúcich meraniach najvýraznejšie prejavil vplyv prefixovej lokality na vhodnosť CPU alebo GPU spracovania. Rozhodovací mechanizmus pri syntetickej prevádzke zvolil CPU iba pre najmenšiu dávku a pri väčších dávkach GPU. Pri enterprise prevádzke boli dávky do 800 000 adries spracované na CPU a väčšie dávky na GPU. Pri hotspot prevádzke bola vo všetkých prípadoch zvolená CPU vetva.

Vyhodnotené boli dva režimy hybridu. Režim bez rozhodovacej réžie predstavuje výkon už zvolenej CPU alebo GPU vetvy bez času potrebného na analýzu vstupu. Režim s rozhodovacou réžiou zahŕňa aj výpočet prefixovej lokality a výber platformy, preto zodpovedá prakticky použiteľnému variantu hybridného riešenia. Dopad tejto réžie je uvedený v tabuľke pre tri reprezentatívne veľkosti dávky.

Tab. 7 - Dopad rozhodovacej réžie na priepustnosť hybridného DIR24/8 modelu

Prevádzka	Dávka [IP]	Zvolená vetva	Hybrid bez rozhodovania [Mpps]	Hybrid s rozhodovaním [Mpps]	Rozdiel [%]
Syntetická	200K	CPU	542,59	57,4	−89,42
Syntetická	1M	GPU	433,7	228,26	−47,37
Syntetická	3M	GPU	426,42	345,54	−18,97
Enterprise	200K	CPU	1 771,48	180,91	−89,79
Enterprise	1M	GPU	710,52	487,01	−31,46
Enterprise	3M	GPU	729,37	631,67	−13,40
Hotspot	200K	CPU	1 495,89	400,96	−73,20
Hotspot	1M	CPU	954,88	919,29	−3,73
Hotspot	3M	CPU	796,37	695,82	−12,63

Priepustnosť v tabuľke bola odvodená z mediánového času spracovania. Z výsledkov vyplýva, že rozhodovacia réžia mala najväčší relatívny dopad pri malých dávkach. Pri dávke 200K IP znížila priepustnosť pri všetkých typoch prevádzky o viac než 70 %, pretože čas samotného spracovania bol krátky a analýza vstupu tvorila významnú časť celkového času.

Pri väčších dávkach sa dopad rozhodovania znižoval, najmä pri syntetickej a enterprise prevádzke, kde bola zvolená GPU vetva. Pri dávke 3M IP predstavoval pokles oproti režimu bez rozhodovania približne 19 % pri syntetickej a 13 % pri enterprise prevádzke. Pri hotspot prevádzke hybrid správne volil CPU vetvu vo všetkých konfiguráciách, keďže vysoká prefixová lokalita zvýhodňovala CPU spracovanie algoritmu DIR24/8.

Výsledky ukazujú, že zvolená rozhodovacia logika dokázala reagovať na charakter vstupnej prevádzky a veľkosť dávky. Praktický prínos hybridného modelu však obmedzila rozhodovacia réžia, najmä pri malých dávkach, kde bol samotný čas spracovania krátky a analýza vstupu tvorila veľkú časť celkového času. Hybridný model preto možno hodnotiť ako experimentálne overenie adaptívneho výberu CPU alebo GPU vetvy, nie ako finálne optimalizované riešenie. Jeho praktické nasadenie by vyžadovalo zníženie nákladov analýzy vstupu, napríklad použitím menšej vzorky alebo opätovným použitím rozhodnutia pre viac po sebe nasledujúcich dávok. Testovaný model zároveň vyberal jednu výpočtovú platformu pre celú dávku, preto nevyužíval súčasné spracovanie rozdielnych častí dávky na CPU a GPU.

Podrobné grafy hybridného DIR24/8 modelu pre syntetickú, enterprise a hotspot prevádzku sú uvedené v Dodatku B, najmä na Obr. 30 až Obr. 40.

8.5 Diskusia výsledkov a obmedzenia merania

Dosiahnuté výsledky ukazujú, že prínos GPU akcelerácie pri IPv4 LPM vyhľadávaní nemožno hodnotiť iba podľa času samotného CUDA jadra. Výsledky merania samotného CUDA jadra potvrdzujú výpočtový potenciál GPU pri veľkých dávkach, avšak po započítaní prenosov medzi CPU a GPU sa praktický prínos znižuje, najmä pri menších vstupoch. Tento jav zodpovedá aj poznatkom štúdie APUNet, ktorá poukazuje na význam prenosovej réžie pri GPU spracovaní sieťových úloh [2].

Významným faktorom bola prefixová lokalita vstupnej prevádzky. Pri hotspot prevádzke sa opakovane pristupovalo k menšej aktívnej časti dátovej štruktúry, čo zvýhodnilo CPU spracovanie. Pri syntetickej prevádzke boli prístupy rozptýlenejšie, a preto sa vo väčších dávkach viac prejavil paralelizmus GPU. Výhodnosť CPU alebo GPU vetvy závisí nielen od veľkosti dávky, ale aj od charakteru vstupných adries.

Rozdiely medzi algoritmami DIR24/8 a MultiBit Trie ukazujú, že výkon nemožno vysvetliť iba počtom paralelných vlákien. Pri DIR24/8 sa prefixová lokalita prejavuje priamo cez opakované prístupy do menšej časti primárnej tabuľky, čo zvýhodňuje najmä cache správanie CPU. Pri MultiBit Trie je vyhľadávanie ovplyvnené aj postupným prechodom cez viacero úrovní stromovej štruktúry, preto samotná koncentrácia adries do /24 oblastí nemusí viesť k rovnako výraznému zrýchleniu. Výsledky tak ukazujú, že výber medzi CPU a GPU spracovaním závisí nielen od veľkosti dávky, ale aj od dátovej štruktúry a jej pamäťového správania.

Hybridný DIR24/8 model ukázal, že jednoduché rozhodovanie podľa veľkosti dávky a prefixovej lokality dokáže vybrať vhodnú výpočtovú vetvu. Jeho praktický prínos však obmedzila réžia samotnej analýzy vstupu. Model preto slúži skôr ako experimentálne overenie adaptívneho výberu platformy než ako finálne optimalizované riešenie.

Obmedzením práce je, že merania hodnotia iba LPM vyhľadávanie nad pripravenými dávkami IPv4 adries. Nezahŕňajú celý proces spracovania paketov v smerovači, teda príjem paketov zo sieťového rozhrania, úpravu hlavičiek ani odosielanie paketov. Výsledky preto treba chápať ako hodnotenie výpočtovej časti smerovacieho rozhodnutia, nie ako priepustnosť kompletného smerovača. Ďalším obmedzením je použitie synteticky generovaných a modelovaných vstupných dát, ktoré nemusia presne zodpovedať reálnej prevádzke konkrétnej siete.

Vo vzťahu k hypotézam formulovaným v kapitole 5.2 možno konštatovať, že H1 bola potvrdená iba čiastočne. GPU implementácie dosahovali vyššiu priepustnosť najmä pri rozptýlenej prevádzke a väčších dávkach, nie však univerzálne pri všetkých typoch vstupov. H2 bola potvrdená, keďže pri menších dávkach mala prenosová a spúšťacia réžia výrazný vplyv

na praktický prínos GPU spracovania. H3 bola potvrdená, pretože prefixová lokalita významne ovplyvnila výhodnosť CPU a GPU spracovania, najmä pri algoritme DIR24/8. H4 bola potvrdená čiastočne: hybridný model dokázal na základe veľkosti dávky a prefixovej lokality zvoliť vhodnú vetvu, ale jeho praktický prínos bol obmedzený rozhodovacou réžiou.

Zhrnutím možno konštatovať, že CUDA implementácie majú pri dávkovom LPM vyhľadávaní vysoký výpočtový potenciál, no ich praktický prínos ovplyvňuje prenosová réžia, veľkosť dávky, dátová štruktúra a typ vstupnej prevádzky. CPU implementácie zostávajú konkurencieschopné najmä pri prevádzke s vysokou prefixovou lokalitou. Ďalší výskum by sa mohol zamerať na platformy so spoločnou pamäťou CPU a GPU, reálne vzorky prevádzky a pokročilejší hybridný model so súčasným využitím CPU aj GPU.

Záver

Cieľom bakalárskej práce bolo preskúmať možnosti urýchlenia smerovania pomocou technológie CUDA. Práca sa nezameriavala na implementáciu kompletného smerovača, ale na výpočtovú časť smerovacieho rozhodnutia, konkrétne na operáciu vyhľadania najdlhšieho zhodného prefixu pre IPv4 adresy. V rámci riešenia boli implementované a experimentálne porovnané dátové štruktúry DIR24/8 a MultiBit Trie vo viacvláknovej CPU verzii a v CUDA verzii. Súčasťou práce bol aj hybridný model pre algoritmus DIR24/8, ktorý overoval možnosť výberu vhodnejšej výpočtovej platformy podľa vlastností vstupnej dávky.

Experimentálne merania potvrdili, že LPM vyhľadávanie je vhodné na paralelné spracovanie, keďže jednotlivé vstupné IPv4 adresy možno vyhodnocovať nezávisle. CUDA implementácie dosahovali vysoký výpočtový potenciál najmä pri meraní samotného jadra CUDA. Pri praktickom porovnaní však bolo potrebné zohľadniť aj prenos vstupných a výstupných dát medzi operačnou pamäťou systému a pamäťou grafickej karty. Výsledky preto ukázali, že GPU akcelerácia nie je univerzálne výhodná, ale jej prínos závisí od veľkosti dávky, použitej dátovej štruktúry a charakteru vstupnej prevádzky.

Dôležitým zistením práce bol vplyv prefixovej lokality. Pri syntetickej prevádzke s rozptýlenejšími adresami sa viac prejavil paralelný výkon GPU, zatiaľ čo pri hotspot prevádzke bola výhodnejšia CPU implementácia DIR24/8, ktorá dokázala lepšie využiť lokalitu pamäťových prístupov. Enterprise prevádzka predstavovala prechodový scenár, v ktorom sa výhodnosť CPU a GPU spracovania menila podľa veľkosti vstupnej dávky. Tieto výsledky potvrdili, že výber vhodnej platformy nemožno určiť iba podľa veľkosti dát, ale aj podľa charakteru samotnej prevádzky.

Hybridný model DIR24/8 ukázal, že adaptívny výber medzi CPU a GPU vetvou môže byť zmysluplný, ale jeho praktická efektívnosť závisí od nákladov rozhodovania. Pri menších dávkach tvorila rozhodovacia réžia významnú časť celkového času, čím sa znižoval prínos hybridného prístupu. Výsledky však naznačili, že ďalší vývoj hybridných riešení môže byť prínosný, najmä pri znížení režijných nákladov alebo pri opätovnom použití rozhodnutia pre viac po sebe nasledujúcich dávok.

Práca splnila stanovený cieľ tým, že navrhla, implementovala a experimentálne vyhodnotila CPU, GPU a hybridné varianty IPv4 LPM vyhľadávania. Dosiahnuté výsledky ukazujú, že technológia CUDA môže byť pre tento typ úlohy prínosná najmä v scenároch s dostatočne veľkou dávkou vstupných adries a s priaznivým pomerom medzi výpočtovým ziskom a prenosovou réžiou.

Ďalšie pokračovanie práce by sa mohlo zamerať na pokročilejší hybridný model, ktorý by rozdeľoval vstupné adresy podľa prefixovej lokality medzi CPU a GPU. Tým by sa mohli

vstupné adresy spracovávať na oboch výpočtových platformách súčasne. Vhodným rozšírením by bolo overenie rovnakých algoritmov na platforme so spoločnou pamäťou CPU a GPU, kde by sa mohol znížiť vplyv prenosovej réžie. Praktickú hodnotu výsledkov by zároveň zvýšilo doplnenie experimentov o reálne vzorky sieťovej prevádzky a smerovacie tabuľky odvodené z reálnych dát.

Literatúra

[1] TANENBAUM, Andrew S.; WETHERALL, David J. Computer Networks. 5th ed. Pearson, 2011. ISBN 978-0-13-212695-3.

[2] GO, Younghwan; JAMSHED, Muhammad Asim; MOON, YoungGyoun; HWANG, Changho; PARK, KyoungSoo. APUNet: Revitalizing GPU as Packet Processing Accelerator. In: Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17). Boston, MA, USA, 27–29 March 2017. USENIX Association, 2017, pp. 83–96. ISBN 978-1-931971-37-9. Dostupné na: https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/go. [cit. 2025-11-23].

[3] S. Mu, X. Zhang, N. Zhang, J. Lu, Y. S. Deng and S. Zhang, „IP routing processing with graphic processors," 2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010), Dresden, Germany, 2010, pp. 93-98, doi: 10.1109/DATE.2010.5457229. Dostupné na: https://ieeexplore.ieee.org/document/5457229 [cit. 2025-11-23]

[4] KALIA, Anuj; ZHOU, Dong; KAMINSKY, Michael; ANDERSEN, David G. Raising the Bar for Using GPUs in Software Packet Processing. In: Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI '15). Oakland, CA, USA, 4–6 May 2015. USENIX Association, 2015. ISBN 978-1-931971-218. Dostupné na: https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/kalia. [cit. 2025-11-23].

[5] ZHAO, Jin; ZHANG, Xinya; WANG, Xin; XUE, Xiangyang. Achieving O(1) IP Lookup on GPU-Based Software Routers. ACM SIGCOMM Computer Communication Review, 2010, vol. 40, no. 4, pp. 429–430. DOI: 10.1145/1851275.1851246.

[6] WALDVOGEL, Marcel. Fast Longest Prefix Matching: Algorithms, Analysis, and Applications [online]. 2000. Dostupné na: https://kops.uni-konstanz.de/server/api/core/bitstreams/8aff16d6-bcdf-44b7-93e9-8104145e5f9a/content. [cit. 2025-12-13].

[7] ZYLA, K.; LIESS, M.; WILD, T.; HERKERSDORF, A. FlexCross: High-Speed and Flexible Packet Processing via a Crosspoint-Queued Crossbar. In: 2024 27th Euromicro Conference on Digital System Design (DSD). Paris, France, 2024, pp. 98–105. DOI: 10.1109/DSD64264.2024.00022.

[8] BARBETTE, Tom; SOLDANI, Cyril; MATHY, Laurent. Fast Userspace Packet Processing. In: Proceedings of the ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS 2015). Oakland, CA, USA, 7–8 May 2015. ACM, 2015, pp. 5–16.

[9] RIZZO, Luigi. Netmap: A Novel Framework for Fast Packet I/O. In: Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC ’12). Boston, MA, USA, 13–15 June 2012. USENIX Association, 2012. Dostupné na: https://www.usenix.org/conference/atc12/technical-sessions/presentation/rizzo. [cit. 2026-01-11].

[10] JAYAKUMAR, Muthurajan. Data Plane Development Kit: Performance Optimization Guidelines [online]. Intel, 2016. Dostupné na: https://www.intel.com/content/www/us/en/developer/articles/guide/dpdk-performance-optimization-guidelines-white-paper.html . [cit. 2026-01-11].

[11] NVIDIA Corporation. CUDA C++ Programming Guide. Release 13.0 [online]. NVIDIA, 2025. Dostupné na: https://docs.nvidia.com/cuda/cuda-c-programming-guide/. [cit. 2026-04-10].

[12] LIN, Huanxin; WANG, Cho-Li. Efficient low-latency packet processing using On-GPU Thread-Data Remapping. Journal of Parallel and Distributed Computing, 2019, vol. 132, pp. 208–220.

[13] HAN, Sangjin; JANG, Keon; PARK, KyoungSoo; MOON, Sue. PacketShader: A GPU-Accelerated Software Router. In: Proceedings of the ACM SIGCOMM 2010 Conference. New Delhi, India, 30 August – 3 September 2010. ACM, 2010, pp. 195–206. DOI: 10.1145/1851182.1851207.

[14] DPDK PROJECT. Longest Prefix Match (LPM) Library. DPDK Programmer’s Guide [online]. Dostupné na: https://doc.dpdk.org/guides/prog_guide/lpm_lib.html. [cit. 2026-04-12].

[15] SRINIVASAN, V.; VARGHESE, G. Fast Address Lookups Using Controlled Prefix Expansion. ACM Transactions on Computer Systems, 1999, vol. 17, no. 1, pp. 1–40. DOI: 10.1145/296502.296503.

Použitie nástrojov umelej inteligencie

Pri tvorbe tejto bakalárskej práce boli použité nástroje generatívnej umelej inteligencie. Ich použitie malo podporný a konzultačný charakter. Nástroje neboli použité ako náhrada vlastnej odbornej práce autora. Autor práce bol zodpovedný za návrh riešenia, metodiku merania, vykonanie experimentov, kontrolu výstupov, interpretáciu výsledkov a konečné znenie práce.

Použité nástroje umelej inteligencie:

OpenAI (2025 – 2026), ChatGPT, služba ChatGPT Plus – použitie pri návrhu, úpravách a ladení meracích a pomocných skriptov v jazykoch C++, CUDA C++ a Python. Nástroj bol použitý najmä pri implementácii CPU, GPU a hybridných variantov LPM vyhľadávania v kapitole 7, pri kontrole chybových hlásení, úprave CSV výstupov a tvorbe skriptov na spracovanie výsledkov a grafov v dodatkoch A až D.

OpenAI (2025 – 2026), ChatGPT, služba ChatGPT Plus – použitie pri konzultácii metodiky merania, štruktúry vyhodnotenia a interpretácie výsledkov v kapitolách 6 až 8. Nástroj bol použitý najmä na kontrolu metodickej konzistentnosti porovnania CPU, GPU a hybridného riešenia.

OpenAI (2025 – 2026), ChatGPT, služba ChatGPT Plus – použitie pri jazykovej a štylistickej úprave textu práce. Nástroj bol použitý pri skracovaní textu, kontrole preklepov, zjednocovaní terminológie a úprave formulácií vybraných odsekov. Nástroj bol použitý aj pri preklade a vysvetľovaní vybraných anglických odborných textov a pri hľadaní súvisiacich zdrojov k analytickej časti práce.

OpenAI (2025 – 2026), ChatGPT, služba ChatGPT Plus – návrh a generovanie obrázku Obr. 3 v kapitole 7 Návrh a implementácia riešenia.

Všetky výstupy vytvorené alebo upravené pomocou uvedeného nástroja boli kontrolované autorom. Za odbornú správnosť, pôvodnosť, výsledky, interpretácie a finálne znenie bakalárskej práce nesie zodpovednosť autor.

Zoznam elektronických príloh

Súčasťou odovzdania práce je ZIP archív s elektronickými prílohami. Archív obsahuje zdrojové kódy použitých CPU, CUDA a hybridných implementácií, skripty na generovanie vstupných dát, vstupné FIB tabuľky a dávky IPv4 adries, CSV a textové výstupy meraní, skripty na spracovanie výsledkov a vybrané výstupy profilovania pomocou nástroja NVIDIA Nsight Compute.

Tab. 8 - Obsah elektronických príloh odovzdávaných spolu s prácou

Označenie	Umiestnenie v archíve	Obsah
EP1	1_Generovanie_vstupnych_dat/	Skripty použité na vytvorenie syntetických vstupných dát a dát modelujúcich enterprise a hotspot prevádzku.
EP2	2_Implementacie_a_vystupy_merani/	Zdrojové kódy CPU, CUDA a hybridných implementácií LPM vyhľadávania spolu so súhrnnými a raw CSV výstupmi meraní.
EP3	3_Spracovanie_vysledkov_a_tvorba_grafov/	Python skripty použité na spracovanie CSV výstupov a tvorbu grafov použitých v práci.
EP4	4_Vstupne_data/	Použité FIB tabuľky a dávky IPv4 adries pre syntetickú, enterprise a hotspot prevádzku.
EP5	5_Nsight_Compute/	PDF a CSV výstupy profilovania CUDA jadier pomocou nástroja NVIDIA Nsight Compute.
EP6	README.txt	Stručný opis štruktúry elektronických príloh a spôsobu orientácie v archíve.

Dodatok A: Použité skripty a spôsob spustenia

Tento dodatok uvádza prehľad zdrojových programov a pomocných skriptov použitých pri príprave vstupných dát, meraní LPM implementácií a spracovaní nameraných výsledkov do grafickej podoby. Uvedené príkazy predstavujú všeobecný spôsob kompilácie a spustenia použitých programov v prostredí Windows pomocou príkazového riadku alebo PowerShellu.

A.1 Skripty na generovanie vstupných dát

Na vytvorenie vstupných FIB tabuliek a testovacích dávok IPv4 adries boli použité nasledujúce skripty:

Tab. 9 - Skripty na generovanie vstupných dát

Úloha	Použitý súbor
Generovanie syntetických FIB tabuliek a syntetických vstupných IPv4 adries	`dataset_generator_synthetic_FIB.cpp`
Vytvorenie enterprise a hotspot vstupných dávok zo syntetických dát	`dataset_generator_enterprise_hotspot.py`

Skript `dataset_generator_synthetic_FIB.cpp` bol použitý na vytvorenie základných syntetických vstupných dát. Python skript `dataset_generator_enterprise_hotspot.py` následne vytváral upravené vstupné dávky reprezentujúce enterprise a hotspot prevádzku so zvýšenou prefixovou lokalitou.

A.2 Meracie programy pre LPM implementácie

Na výkonnostné merania CPU, GPU a hybridných implementácií boli použité meracie programy z Tab. 10. CPU implementácie boli kompilované ako programy v jazyku C++ s podporou OpenMP. GPU implementácie boli kompilované pomocou nástroja `nvcc` z CUDA Toolkit. Hybridná implementácia bola kompilovaná ako CUDA program s povolenou podporou OpenMP pre CPU vetvu.

Tab. 10 - Meracie programy pre LPM implementácie

Typ implementácie	Použitý súbor
CPU implementácia algoritmu DIR24/8	`cpu_dir24_8.cpp`
CPU implementácia algoritmu Multi-Bit Trie	`cpu_multibit_trie.cpp`
CUDA implementácia algoritmu DIR24/8	`gpu_dir24_8.cu`
CUDA implementácia algoritmu MultiBit Trie	`gpu_multibit_trie.cu`

A.3 Skripty na spracovanie výsledkov a tvorbu grafov

Na spracovanie CSV výstupov z meracích programov a vytvorenie grafov boli použité nasledujúce Python skripty:

Tab. 11 - Skripty na spracovanie výsledkov a tvorbu grafov

Úloha	Použitý súbor
Porovnanie CPU implementácií podľa typu prevádzky	`CPU_Traffic_Type_comparison.py`
Porovnanie GPU implementácií podľa typu prevádzky	`GPU_Traffic_Type_comparison.py`
Spracovanie CPU/GPU výsledkov pre enterprise prevádzku	`process_cpu_gpu_all_lpm_enterprise.py`
Spracovanie CPU/GPU výsledkov pre hotspot prevádzku	`process_cpu_gpu_all_lpm_hotspot.py`
Spracovanie CPU/GPU výsledkov pre syntetickú prevádzku	`process_cpu_gpu_all_lpm_synthetic.py`
Spracovanie výsledkov CPU, GPU a hybridného DIR24/8 modelu	`process_dir24_8_cpu_gpu_cuda_hybrid.py`

Skript `process_cpu_gpu_all_lpm.py` bol použitý iba ako pracovná alebo všeobecná verzia spracovania výsledkov a nepredstavoval hlavný zdroj finálnych grafov uvedených v práci.

A.4 Spôsob spustenia Python skriptov

Python skripty boli spúšťané z príkazového riadku alebo PowerShellu pomocou príkazu:

```
py nazov_skriptu.py
```

Príklad:

```
py CPU_Traffic_Type_comparison.py
```

Skripty načítavali príslušné CSV súbory, spracovali namerané hodnoty a vytvorili grafické výstupy použité v experimentálnom vyhodnotení a dodatkoch.

A.5 Kompilácia a spustenie C++ programov

C++ programy boli kompilované v prostredí x64 Native Tools Command Prompt for Visual Studio 2022, ktoré inicializuje premenné prostredia potrebné pre kompilátor Microsoft Visual C++:

```
cl /O2 /std:c++17 /EHsc /openmp nazov_skriptu.cpp
```

Spustenie programu:

```
nazov_skriptu.exe
```

Príklad:

```
cl /O2 /std:c++17 /EHsc /openmp cpu_dir24_8.cpp
cpu_dir24_8.exe
```

A.6 Kompilácia a spustenie CUDA programov

CUDA programy boli kompilované pomocou nástroja `nvcc` s optimalizáciou a podporou štandardu C++17:

```
nvcc -O2 -std=c++17 nazov_skriptu.cu -o nazov_skriptu.exe
```

Spustenie programu:

```
nazov_skriptu.exe
```

Príklad:

```
nvcc -O2 -std=c++17 gpu_dir24_8.cu -o gpu_dir24_8.exe
gpu_dir24_8.exe
```

A.7 Kompilácia a spustenie hybridného CUDA programu

Hybridný program obsahoval CUDA GPU vetvu aj CPU vetvu paralelizovanú pomocou OpenMP. Z tohto dôvodu bol kompilovaný pomocou `nvcc` s odovzdaním prepínača `/openmp` hostiteľskému kompilátoru:

```
nvcc -O3 -std=c++17 -Xcompiler "/openmp" nazov_skriptu.cu -o
nazov_skriptu.exe
```

Spustenie programu:

```
nazov_skriptu.exe
```

Príklad:

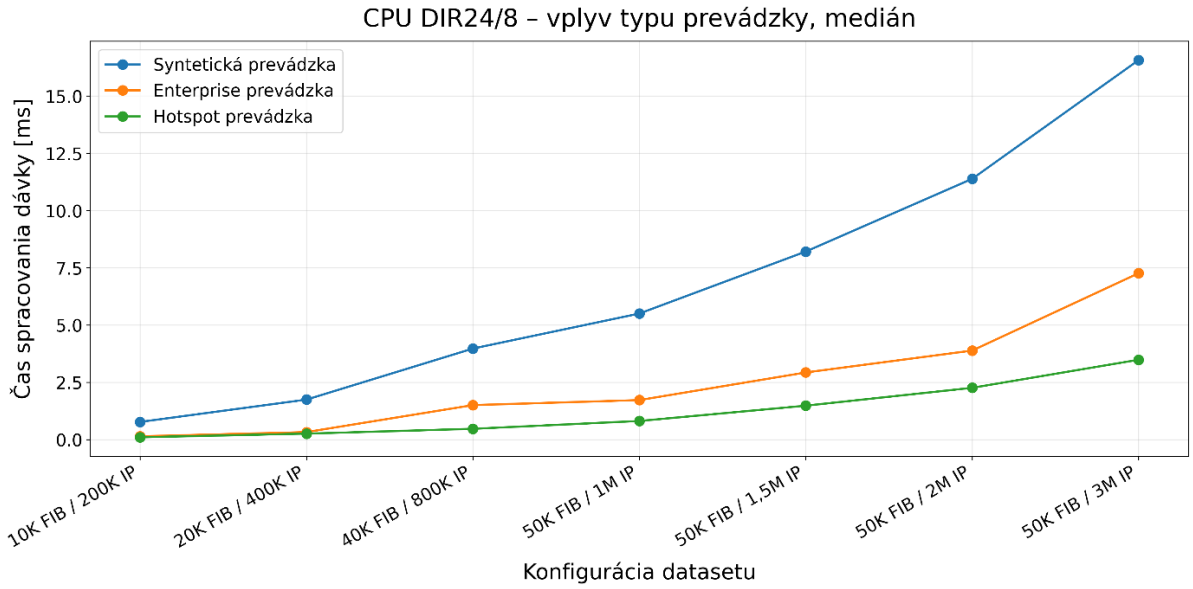
```
nvcc -O3 -std=c++17 -Xcompiler "/openmp" hyb-
rid_cpu_dir24_8.cu -o hybrid_cpu_dir24_8.exe
hybrid_cpu_dir24_8.exe
```

A.8 Poznámka k reprodukovateľnosti

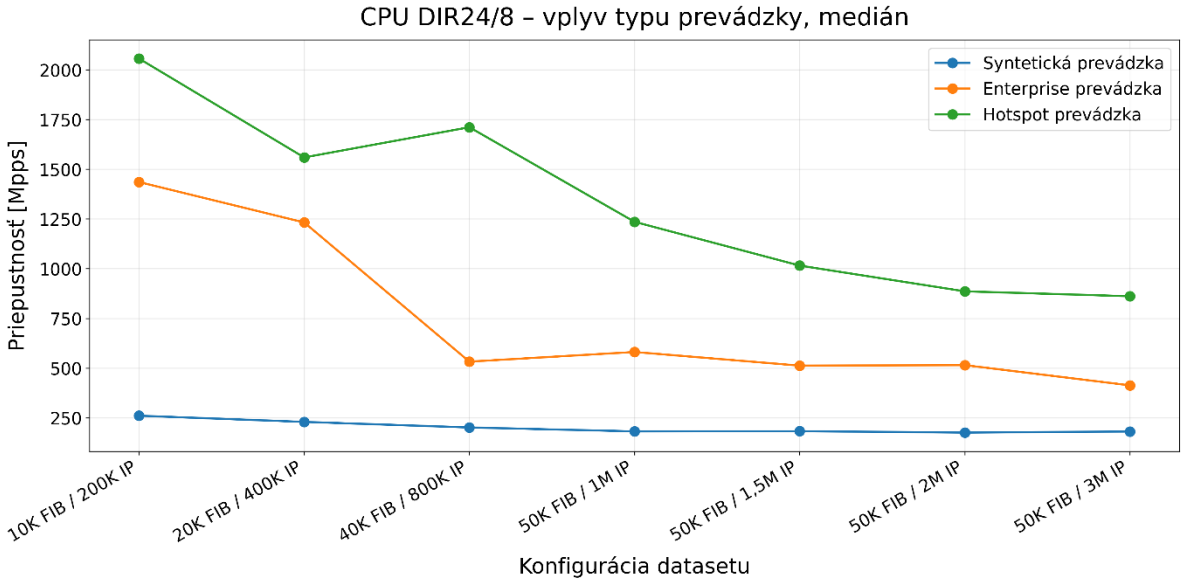
Meracie programy vytvárali výstupné CSV súbory obsahujúce namerané časy, priepustnosti a kontrolné hodnoty. Tieto výstupy boli následne spracované Python skriptmi do grafov a tabuliek použitých v experimentálnom vyhodnotení. Pri interpretácii výsledkov je potrebné zohľadniť, že merania boli vykonané na konkrétnej hardvérovej konfigurácii uvedenej v kapitole metodiky experimentálneho overenia.

Dodatok B: Doplnkové grafy experimentálneho vyhodnotenia

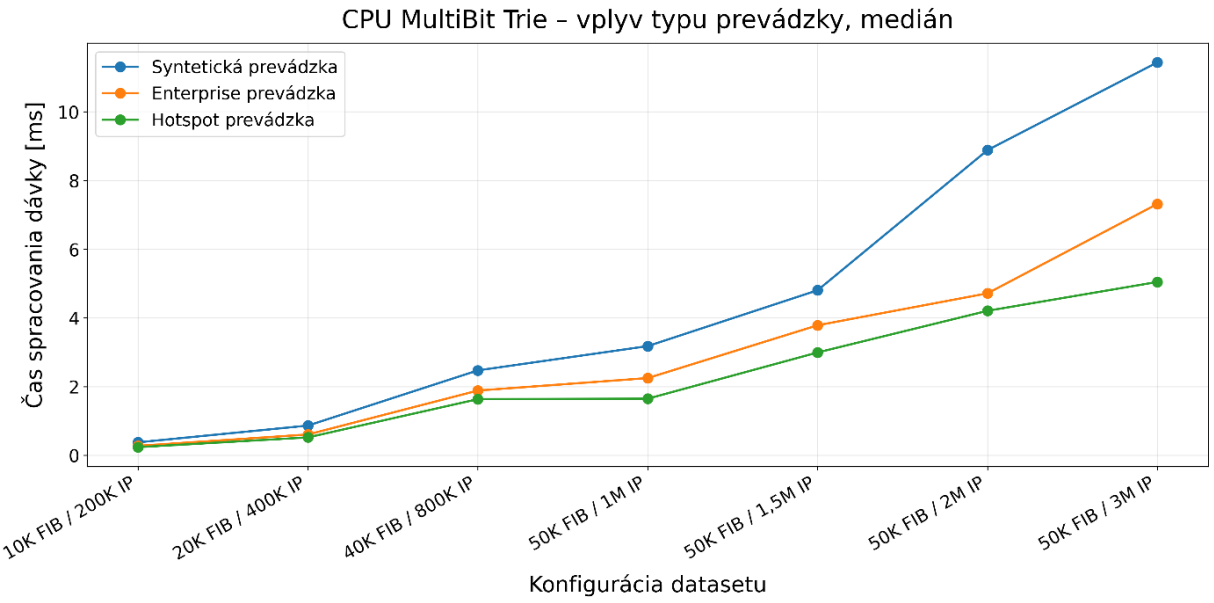
Dodatok B obsahuje doplnkové grafy experimentálneho vyhodnotenia. Grafy rozširujú výsledky uvedené v kapitole 8 a zobrazujú úplné porovnanie časov a priepustností pre jednotlivé typy prevádzky, implementácie a merané konfigurácie.



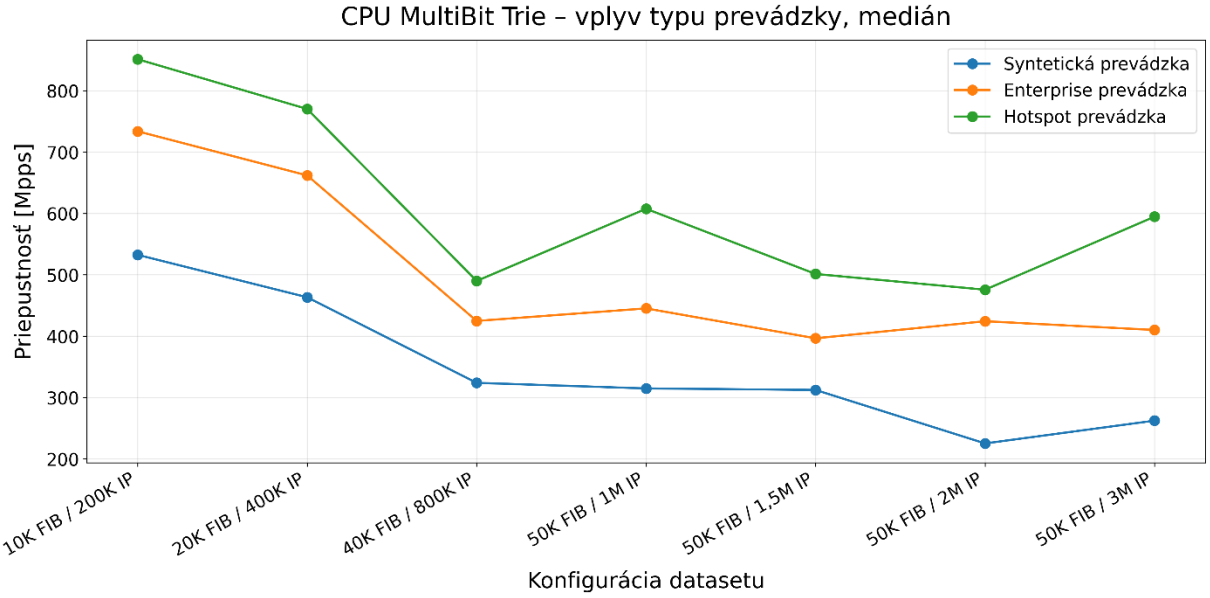
Obr. 5 - Vplyv typu prevádzky na čas spracovania dávky pri CPU implementácii DIR24/8



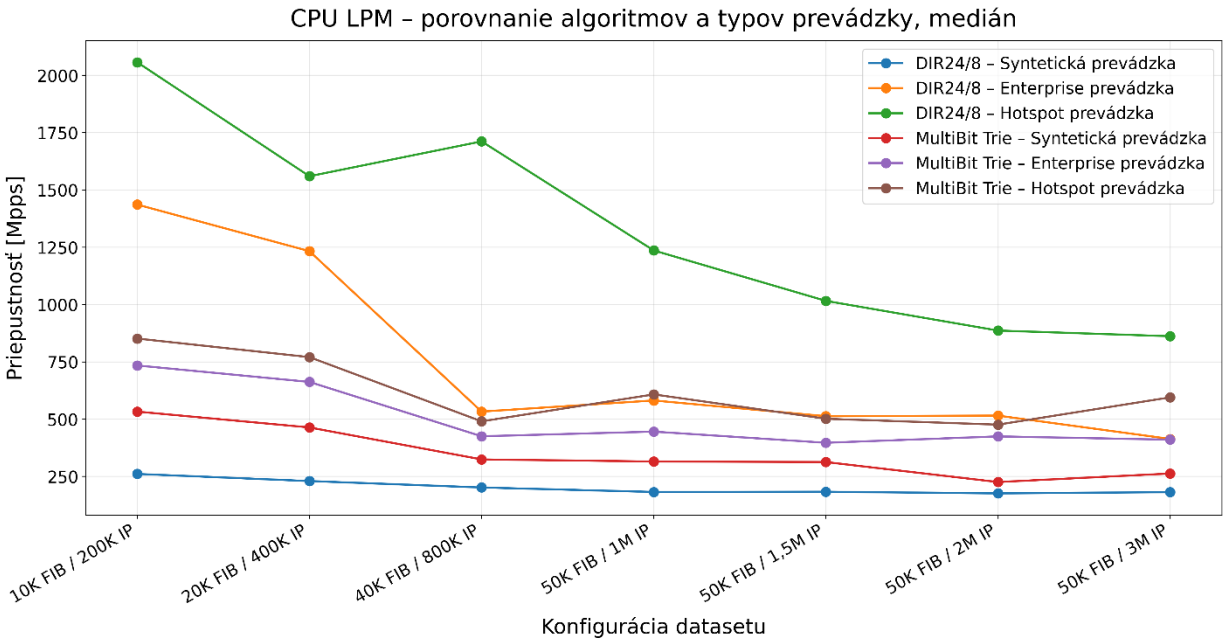
Obr. 6 - Vplyv typu prevádzky na priepustnosť CPU implementácie DIR24/8



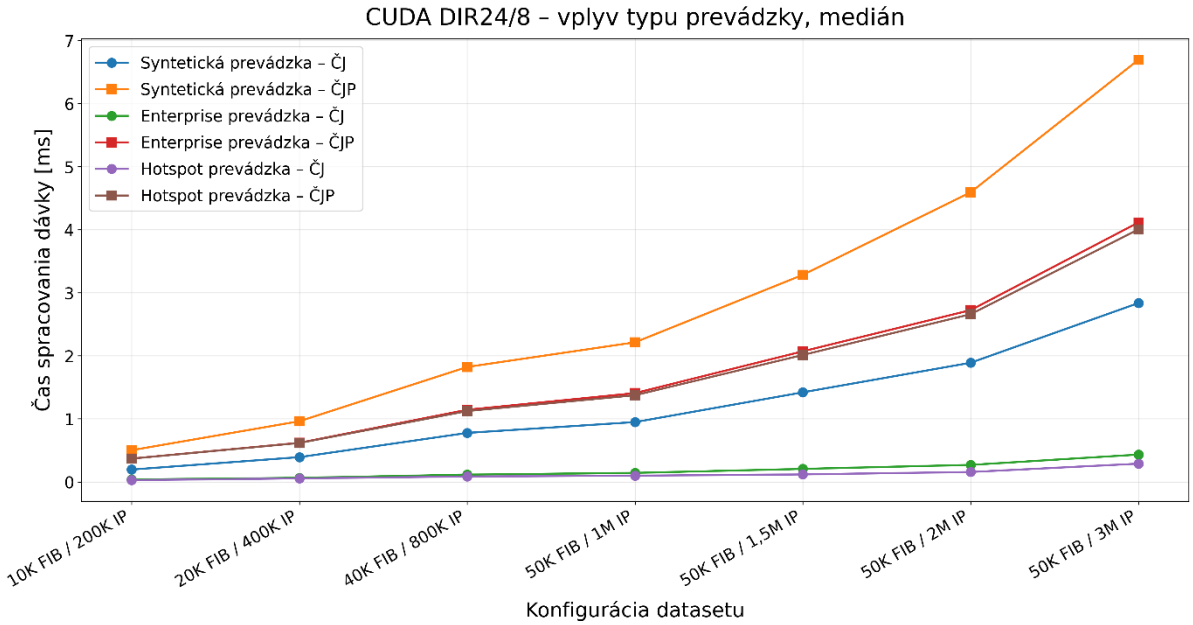
Obr. 7 - Vplyv typu prevádzky na čas spracovania dávky pri CPU implementácii MultiBit Trie



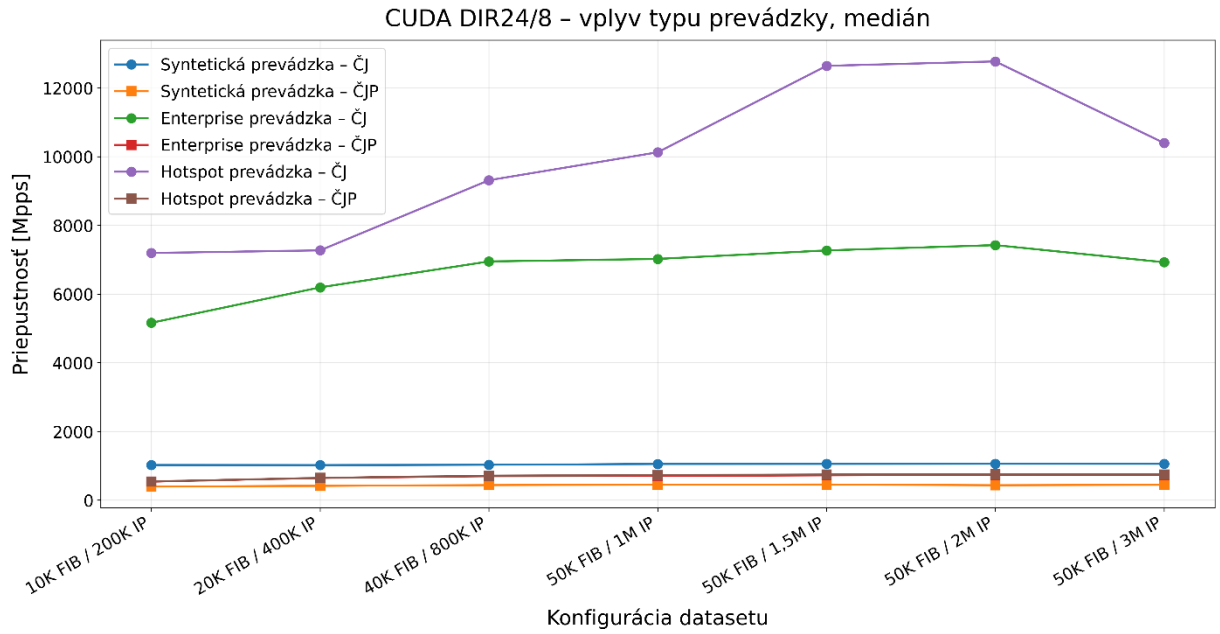
Obr. 8 - Vplyv typu prevádzky na priepustnosť CPU implementácie MultiBit Trie



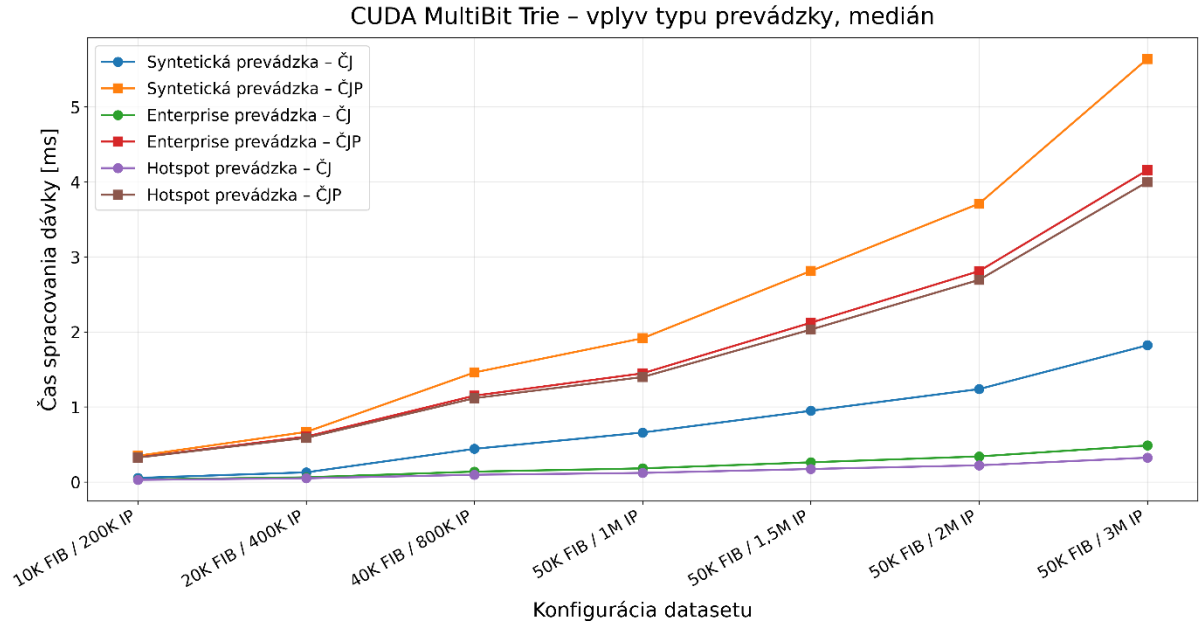
Obr. 9 - Porovnanie priepustnosti CPU implementácií DIR24/8 a MultiBit Trie pri rôznych typoch prevádzky



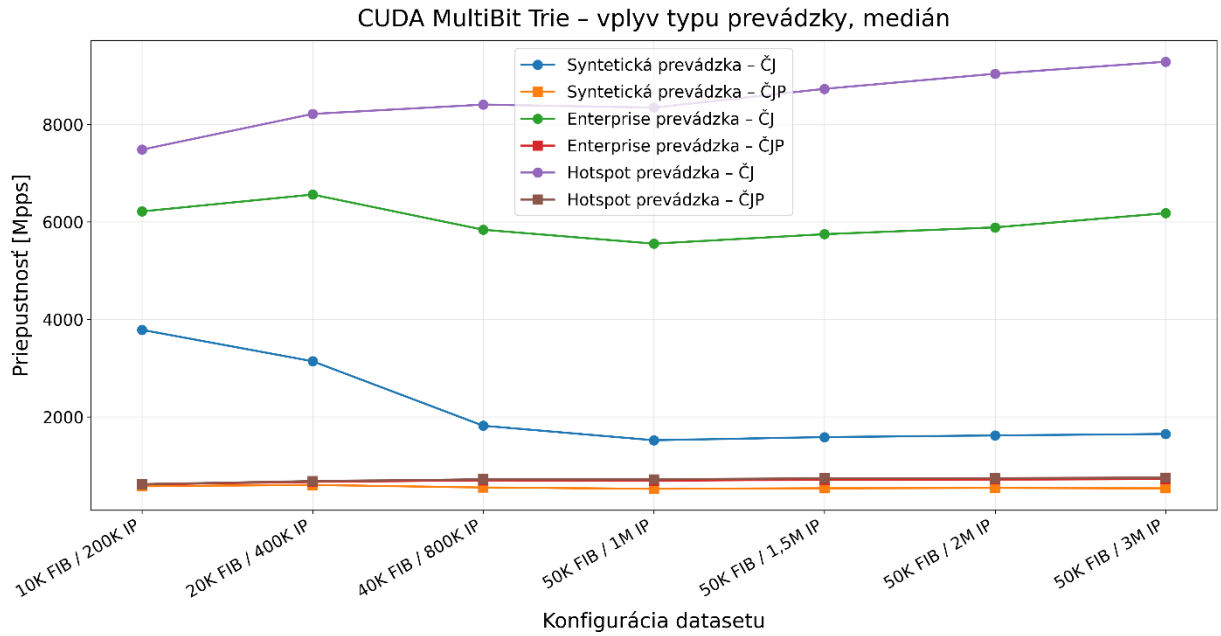
Obr. 10 - Vplyv typu prevádzky na čas spracovania dávky pri GPU implementácii DIR24/8



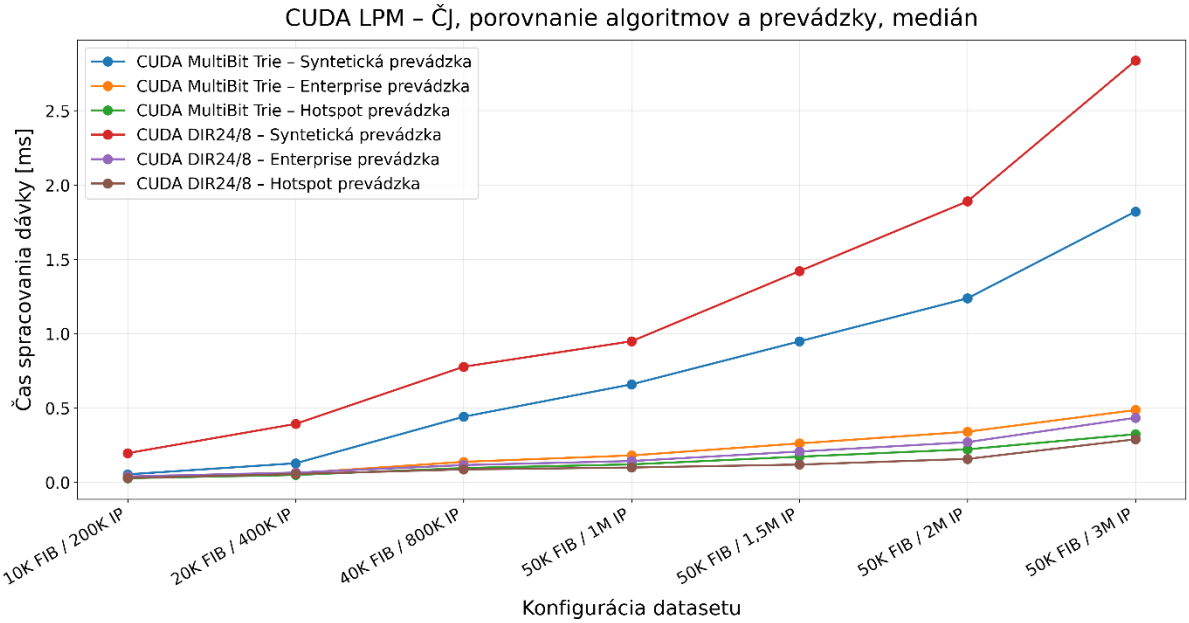
Obr. 11 - Vplyv typu prevádzky na priepustnosť GPU implementácie DIR24/8



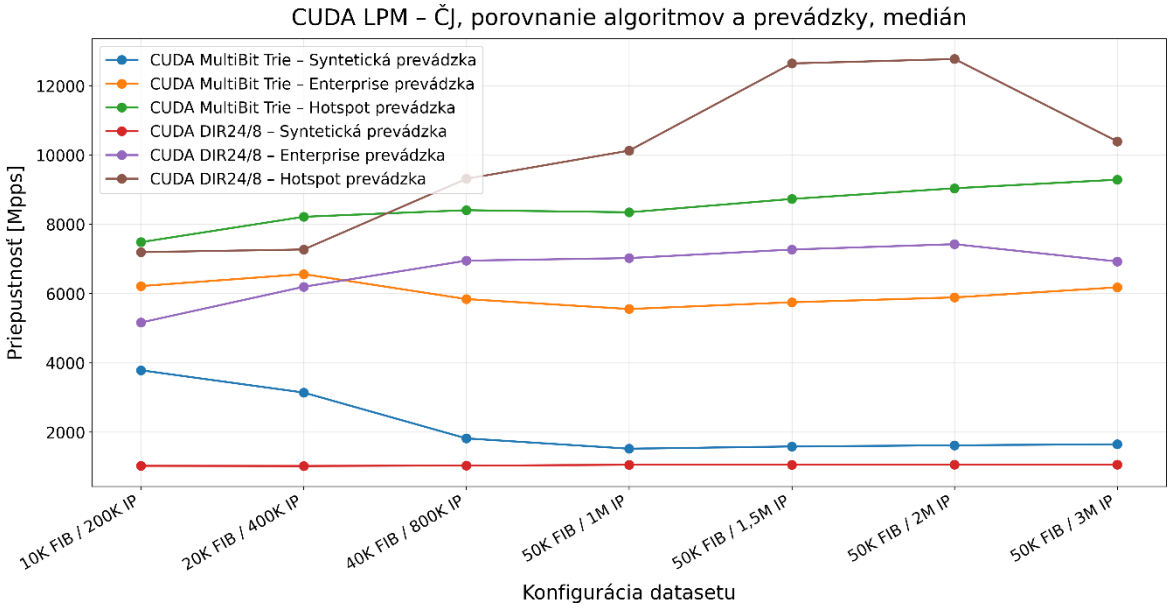
Obr. 12 - Vplyv typu prevádzky na čas spracovania dávky pri GPU implementácii MultiBit Trie



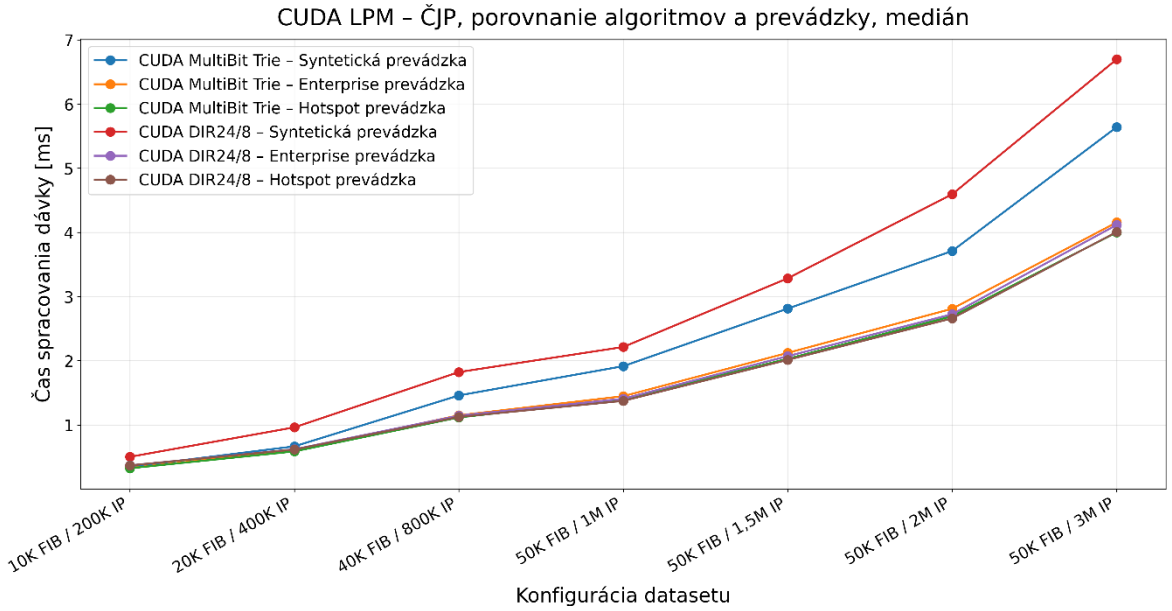
Obr. 13 - Vplyv typu prevádzky na priepustnosť GPU implementácie MultiBit Trie



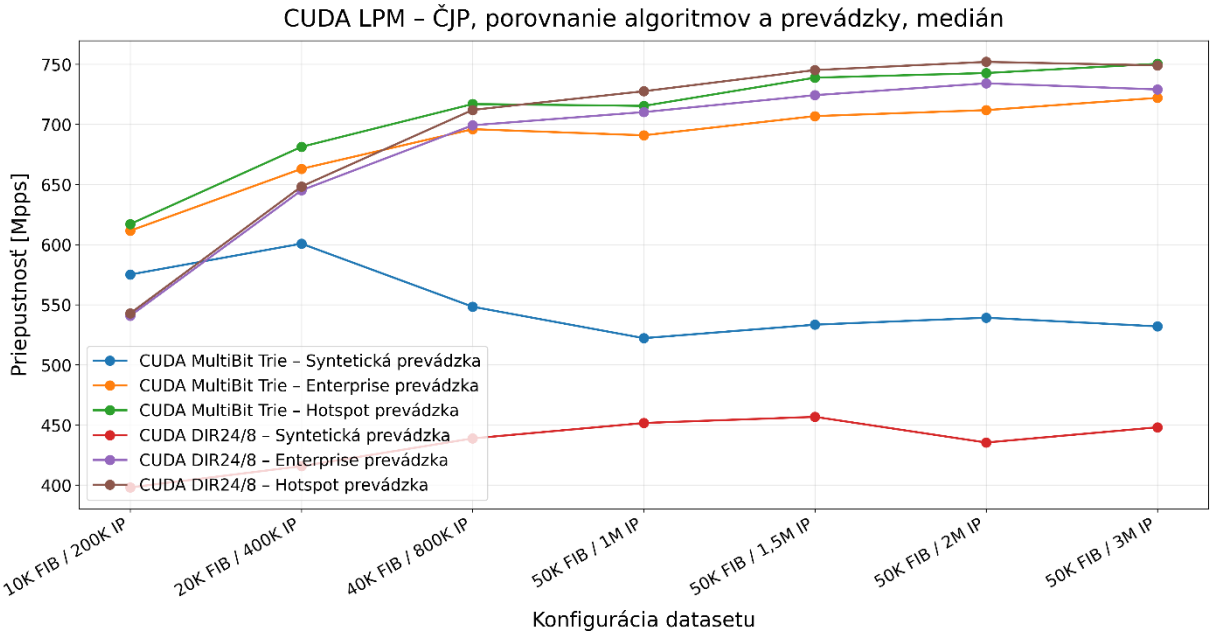
Obr. 14 - Porovnanie času spracovania dávky pri GPU implementáciách DIR24/8 a MultiBit Trie bez prenosovej réžie



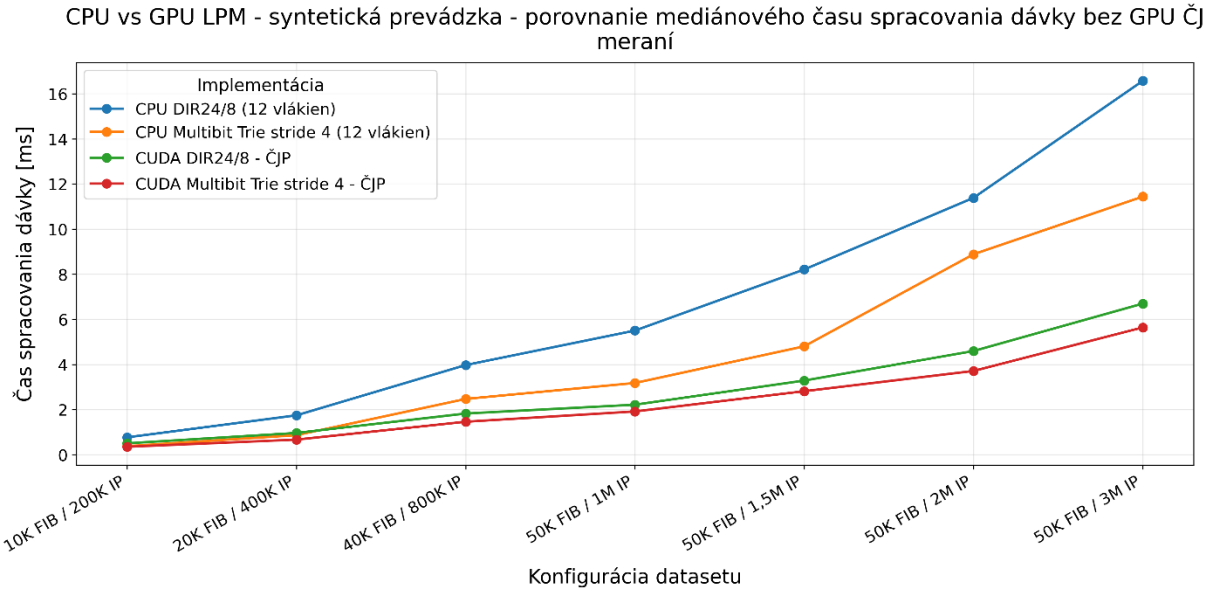
Obr. 15 - Porovnanie priepustnosti GPU implementácií DIR24/8 a MultiBit Trie bez prenosovej réžie



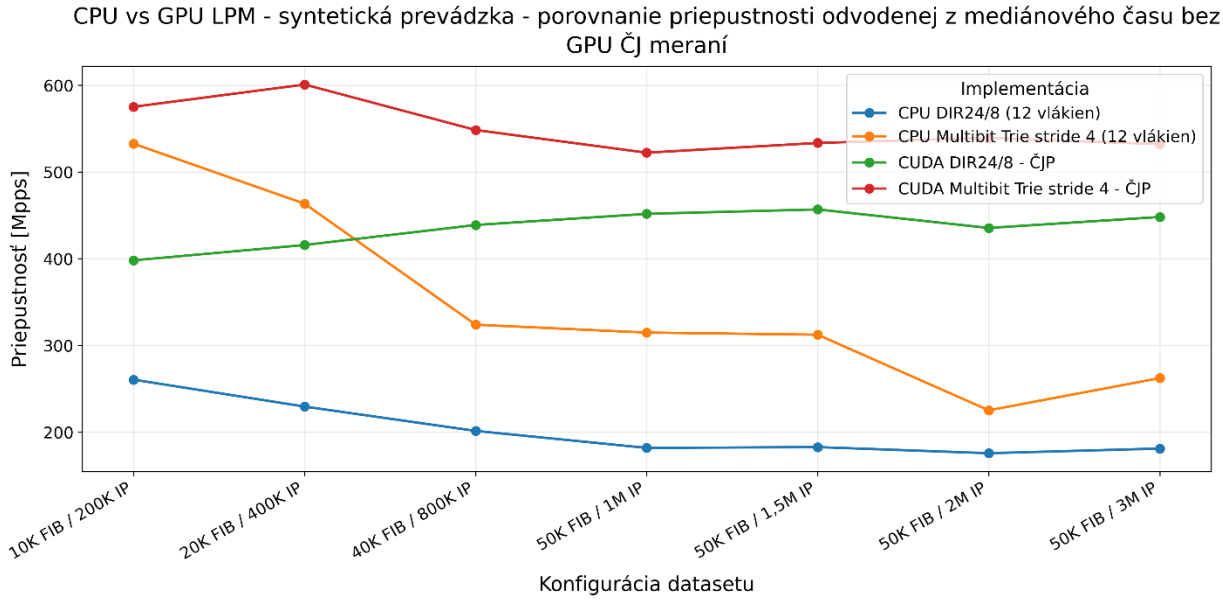
Obr. 16 - Porovnanie času spracovania dávky pri GPU implementáciách DIR24/8 a MultiBit Trie vrátane prenosovej réžie



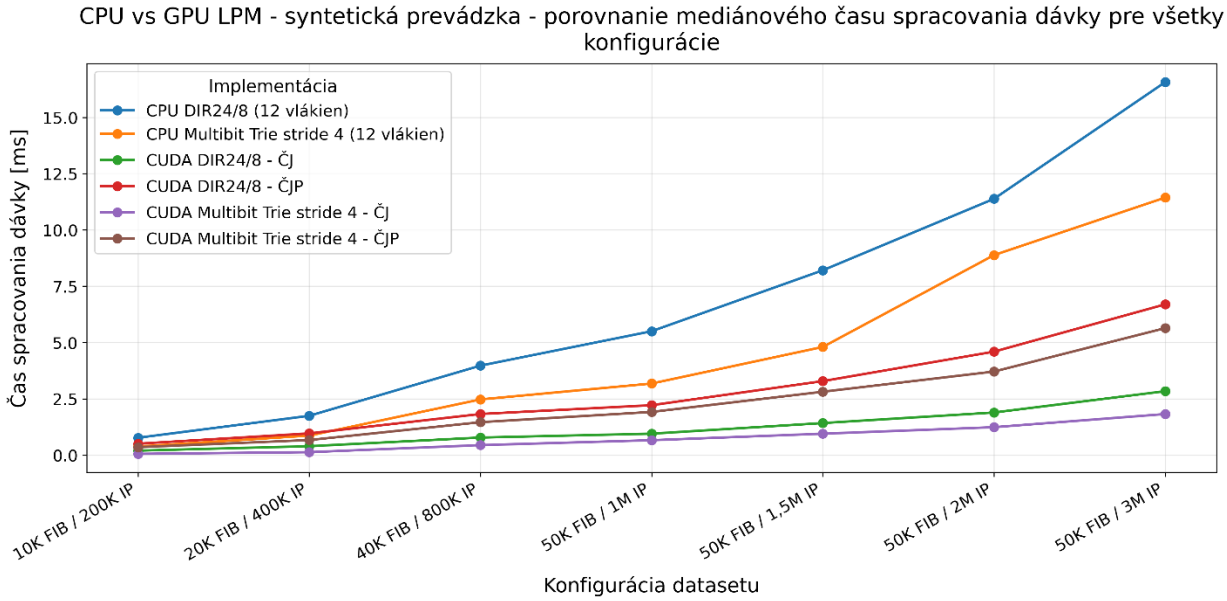
Obr. 17 - Porovnanie priepustnosti GPU implementácií DIR24/8 a MultiBit Trie vrátane prenosovej réžie



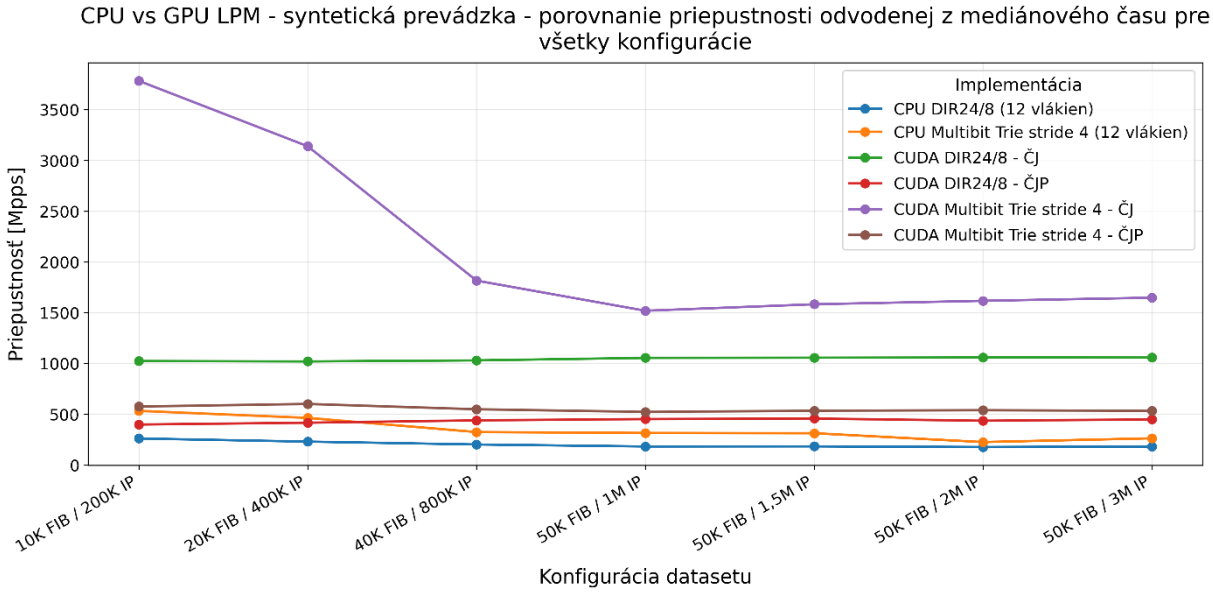
Obr. 18 - Porovnanie času spracovania dávky CPU a GPU implementácií pri syntetickej prevádzke bez samostatných meraní ČJ



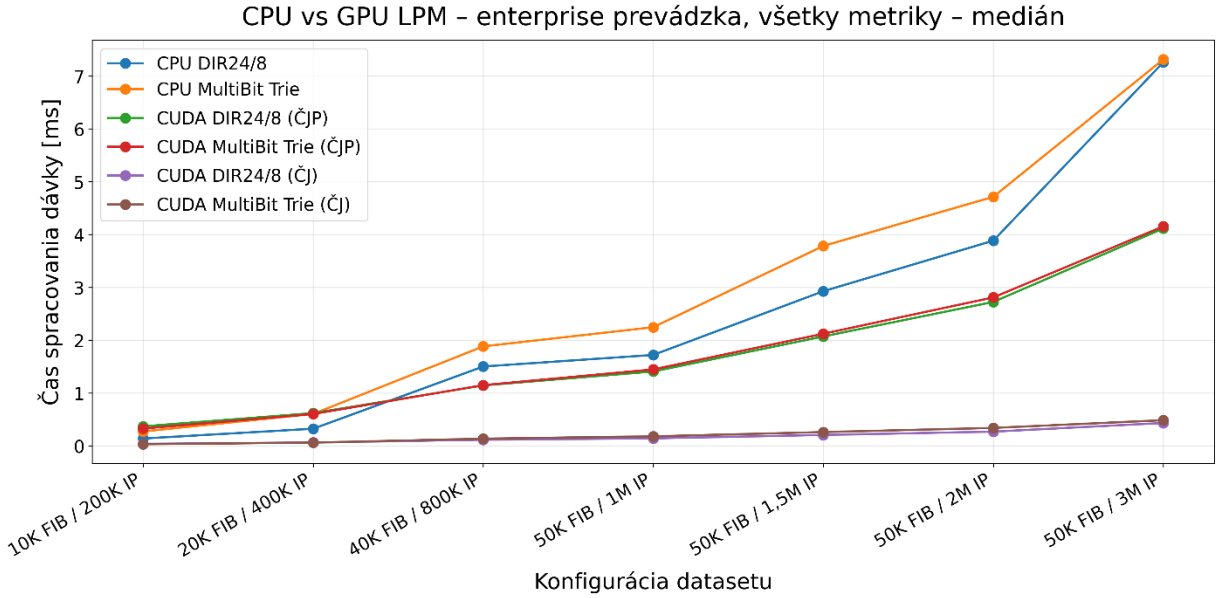
Obr. 19 - Porovnanie priepustnosti CPU a GPU implementácií pri syntetickej prevádzke bez samostatných meraní ČJ



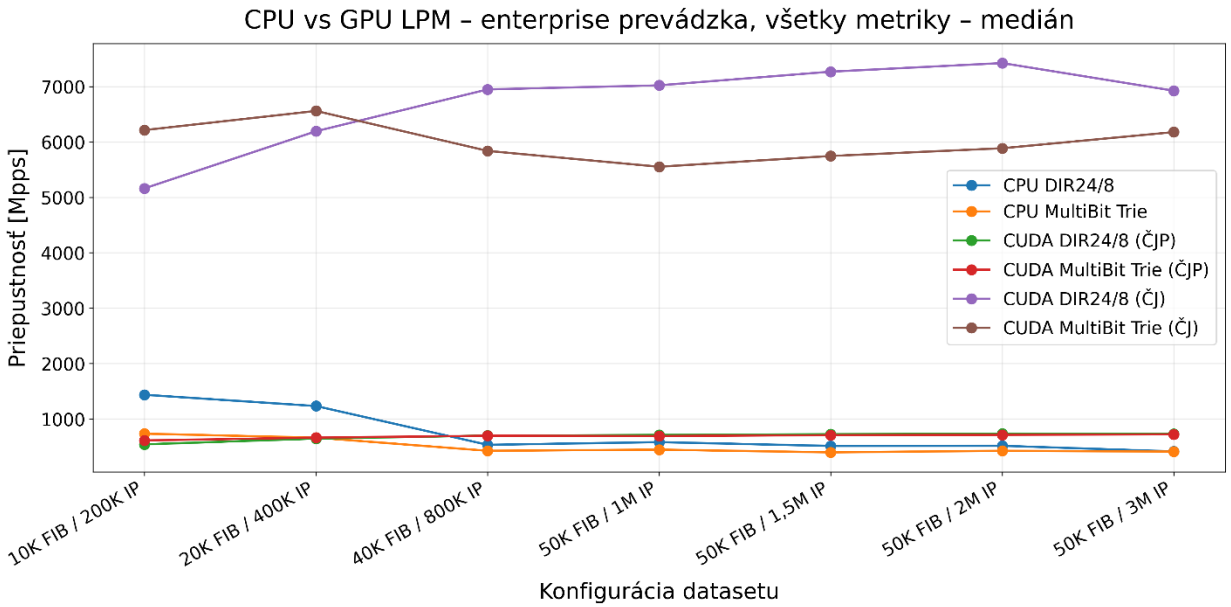
Obr. 20 - Porovnanie času spracovania dávky CPU a GPU implementácií pri syntetickej prevádzke vrátane samostatných meraní ČJ



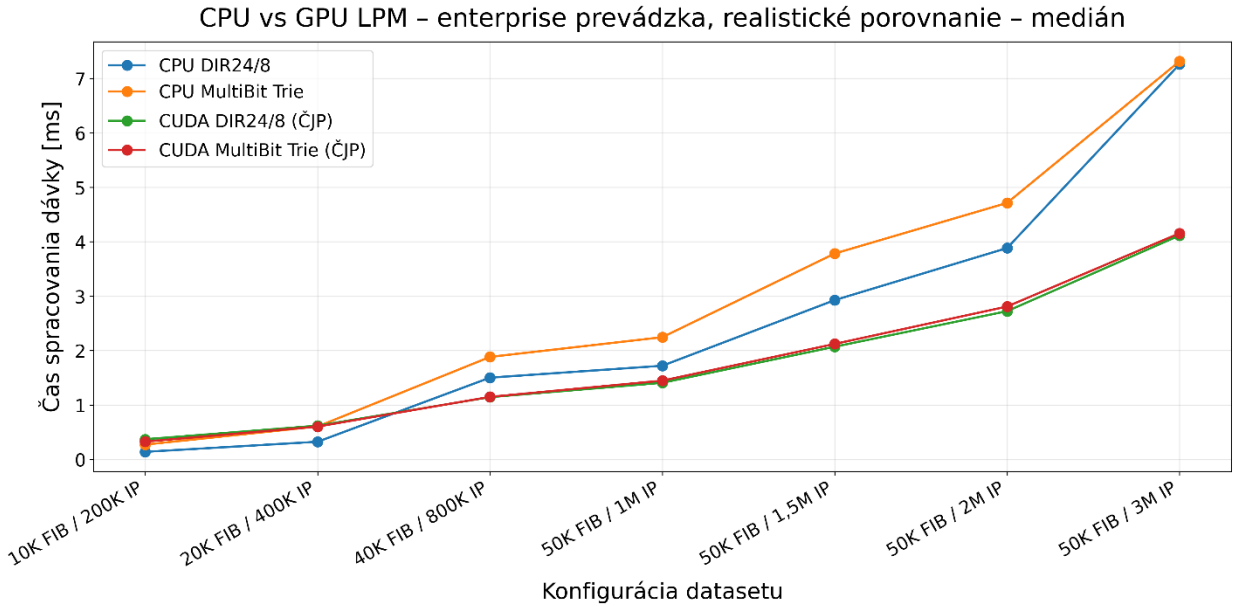
Obr. 21 - Porovnanie priepustnosti CPU a GPU implementácií pri syntetickej prevádzke vrátane samostatných meraní ČJ



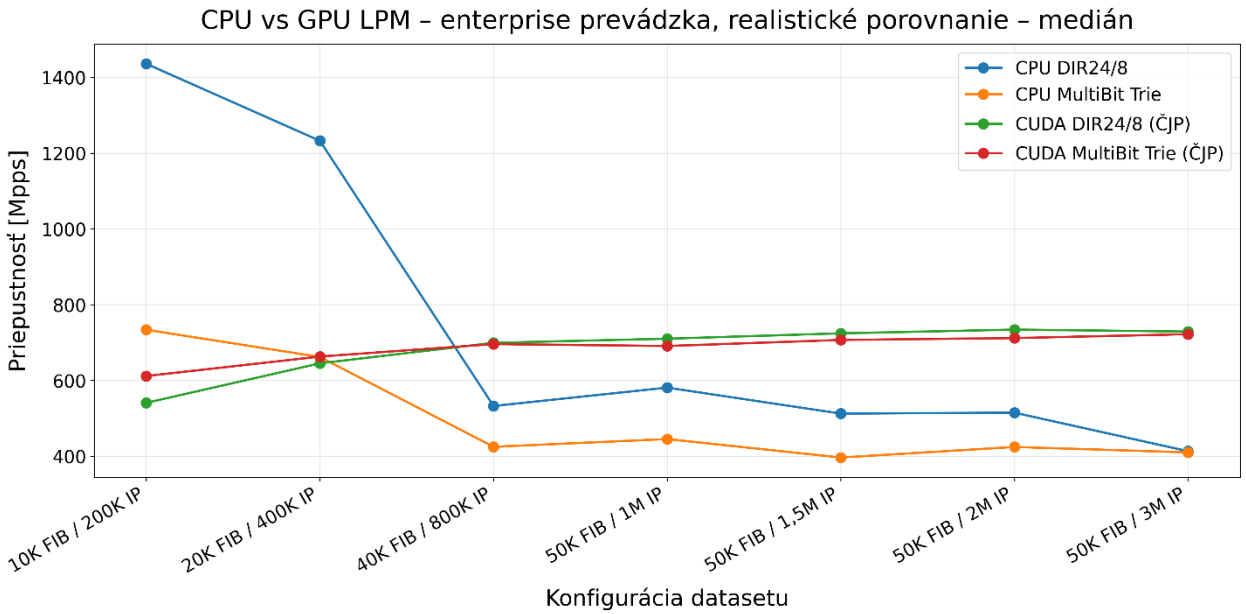
Obr. 22 - Porovnanie času spracovania dávky CPU a GPU implementácií pri enterprise prevádzke vrátane samostatných meraní ČJ



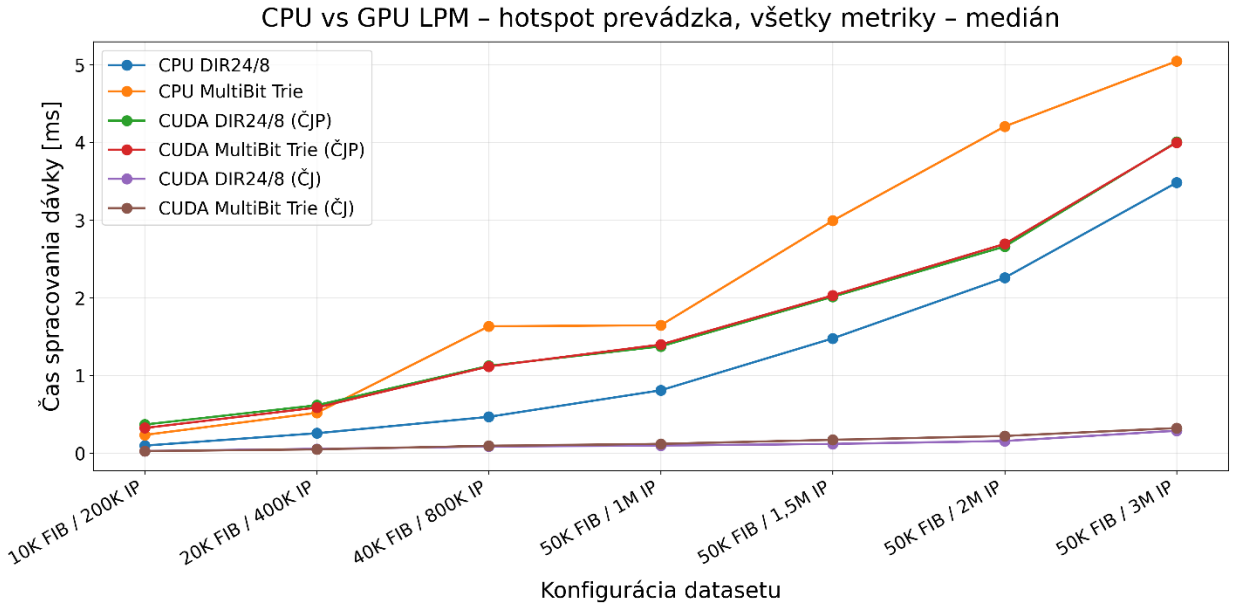
Obr. 23 - Porovnanie priepustnosti CPU a GPU implementácií pri enterprise prevádzke vrátane samostatných meraní ČJ



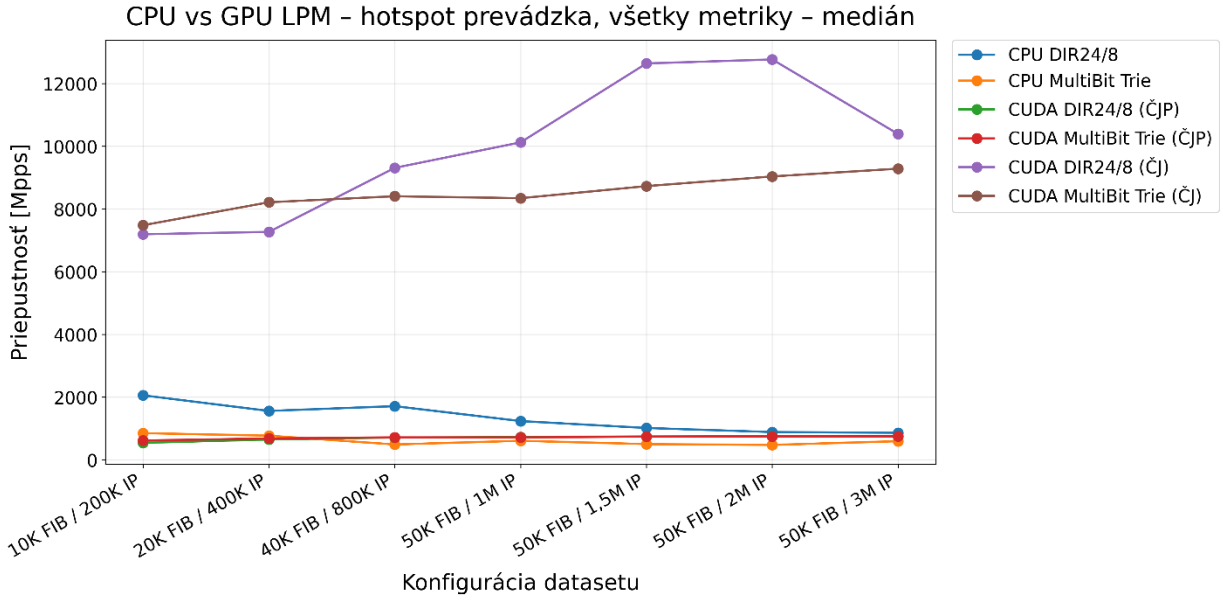
Obr. 24 - Realistické porovnanie času spracovania dávky CPU a GPU implementácií pri enterprise prevádzke



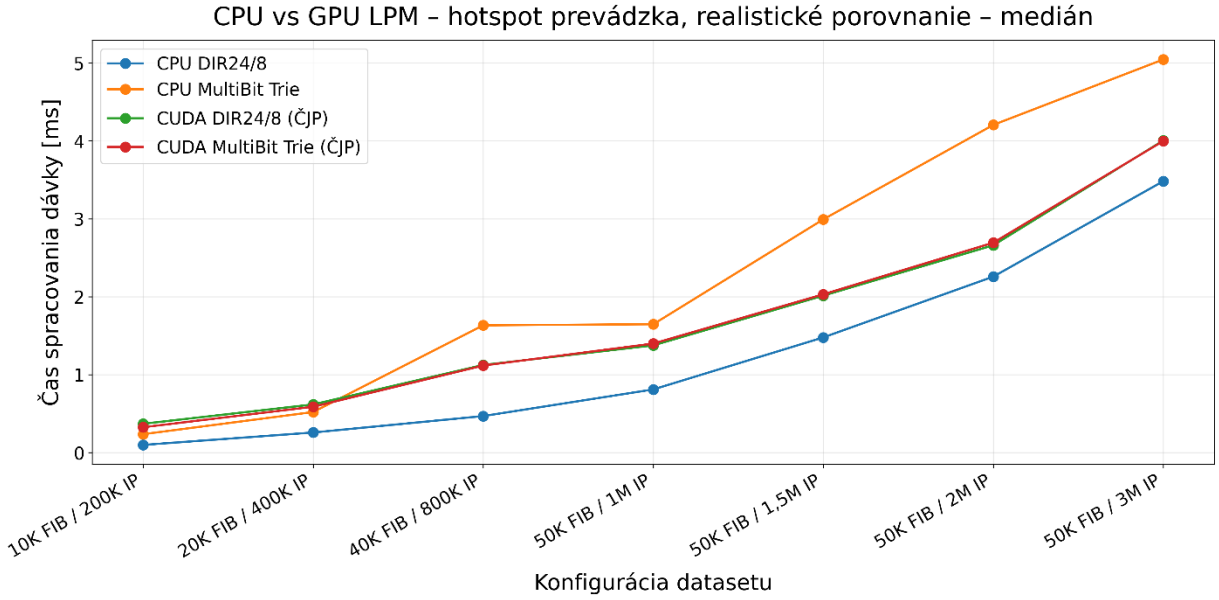
Obr. 25 - Realistické porovnanie priepustnosti CPU a GPU implementácií pri enterprise prevádzke



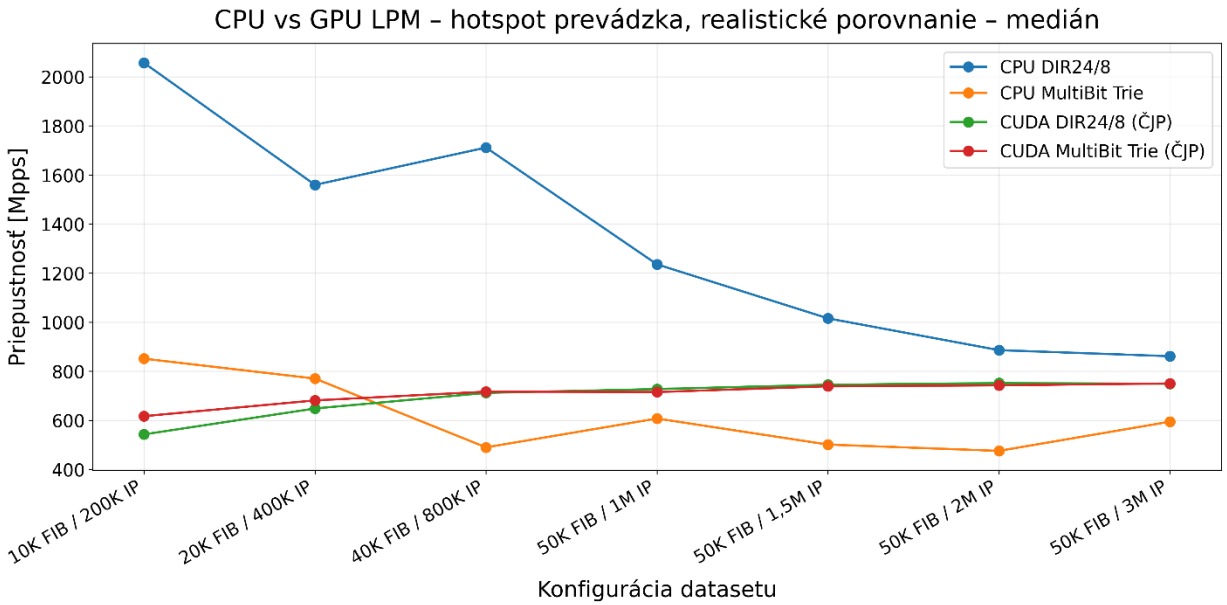
Obr. 26 - Porovnanie času spracovania dávky CPU a GPU implementácií pri hotspot prevádzke vrátane samostatných meraní ČJ



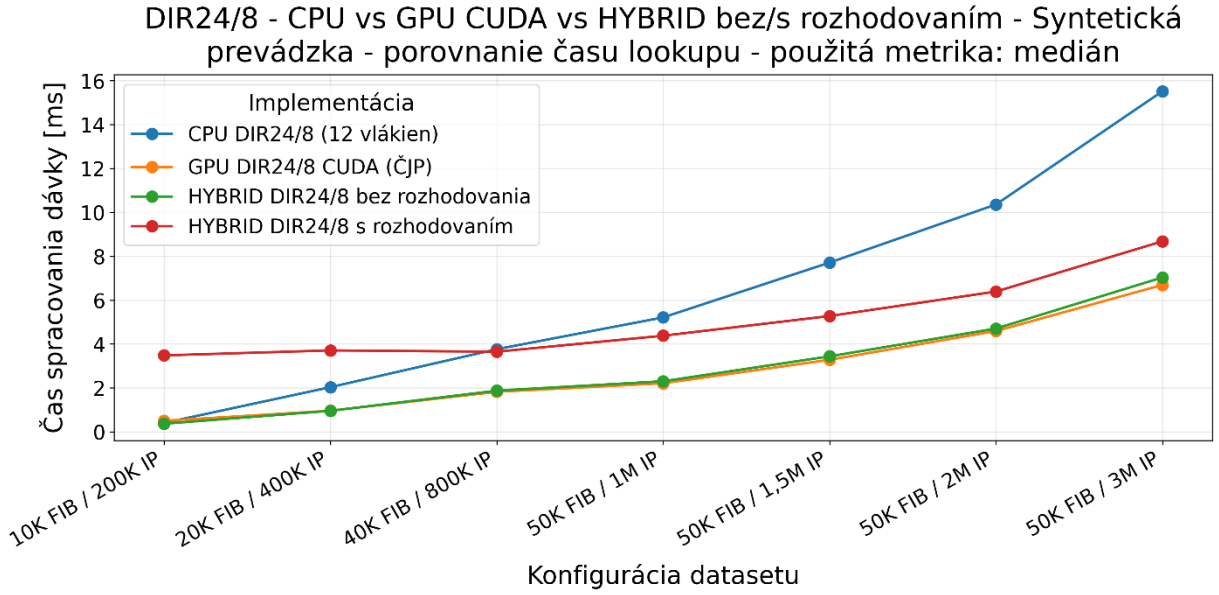
Obr. 27 - Porovnanie priepustnosti CPU a GPU implementácií pri hotspot prevádzke vrátane samostatných meraní ČJ



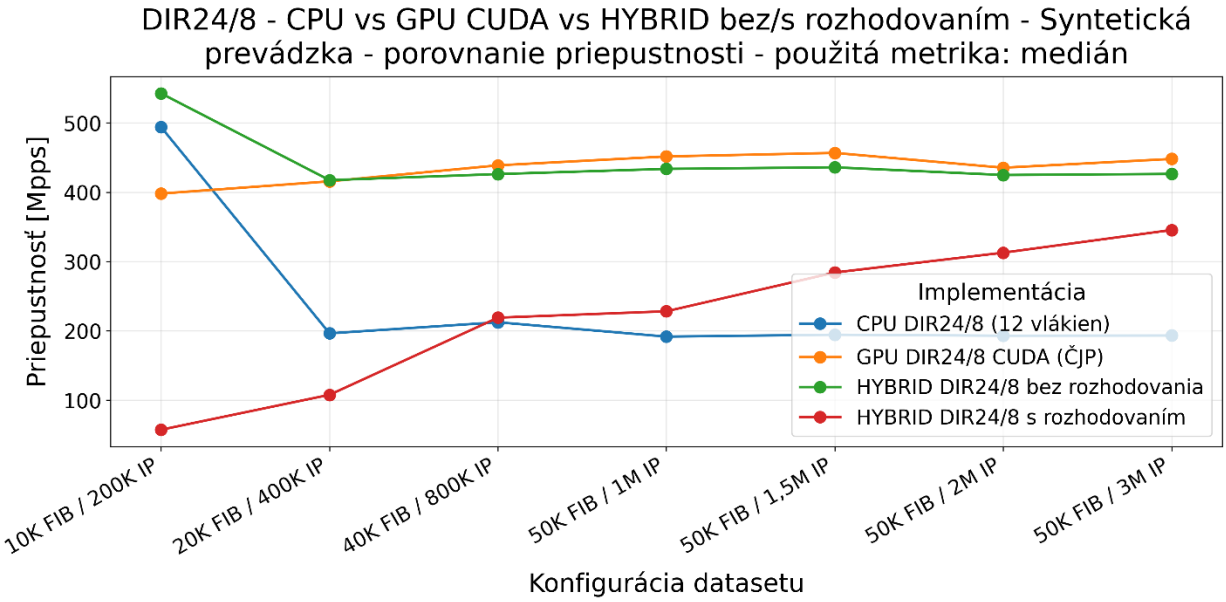
Obr. 28 - Realistické porovnanie času spracovania dávky CPU a GPU implementácií pri hotspot prevádzke



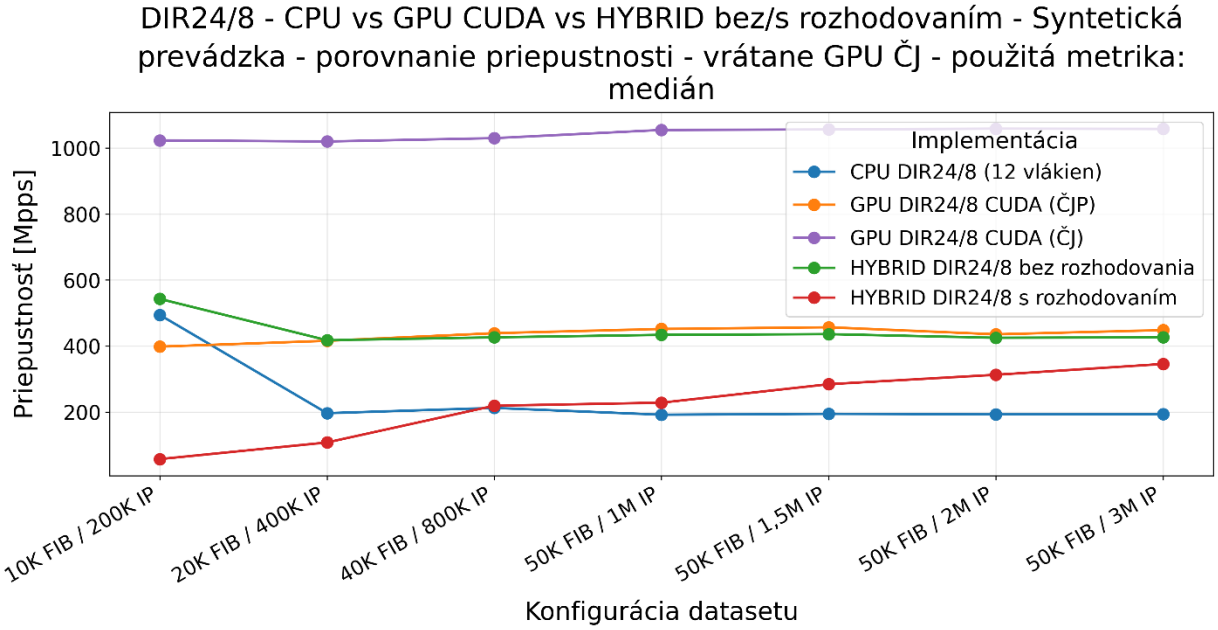
Obr. 29 - Realistické porovnanie priepustnosti CPU a GPU implementácií pri hotspot prevádzke



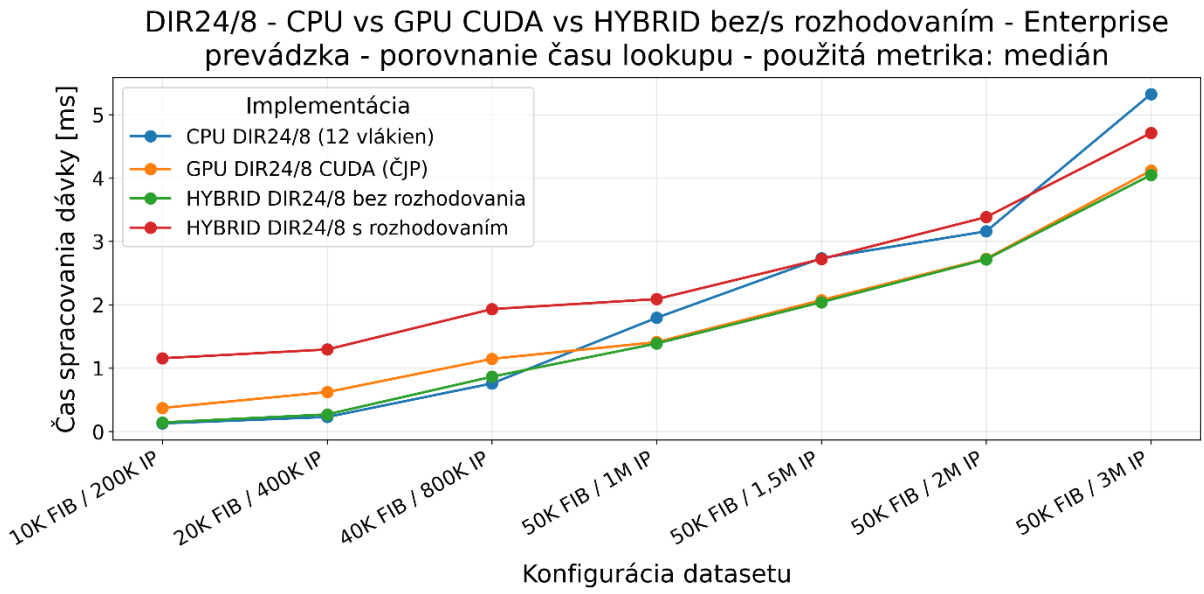
Obr. 30 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke



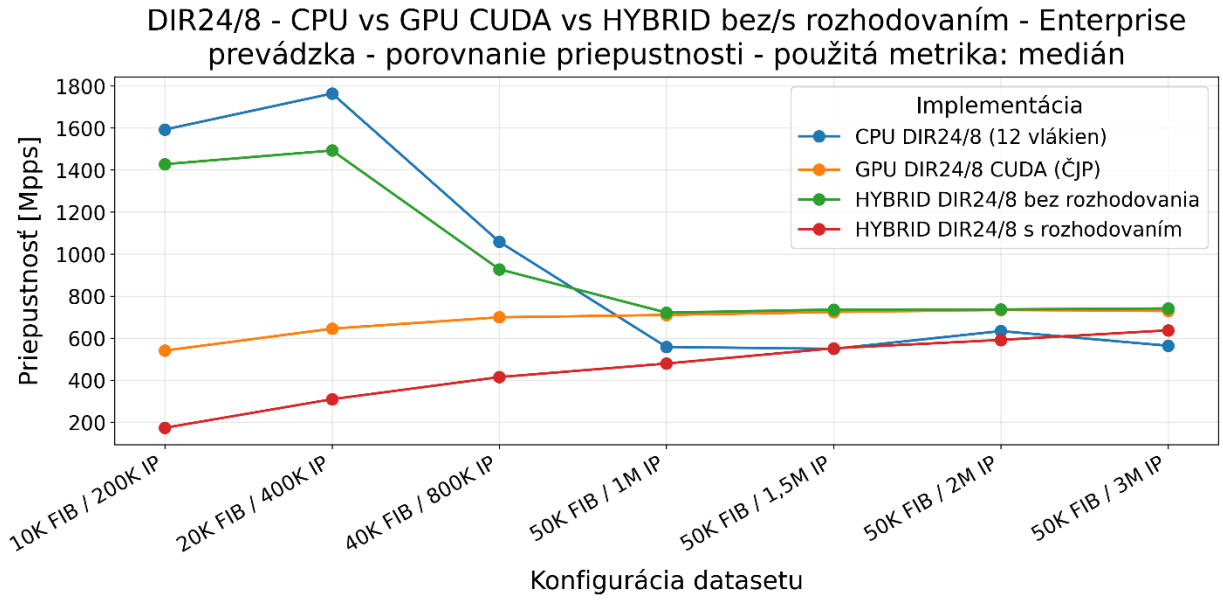
Obr. 31 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke



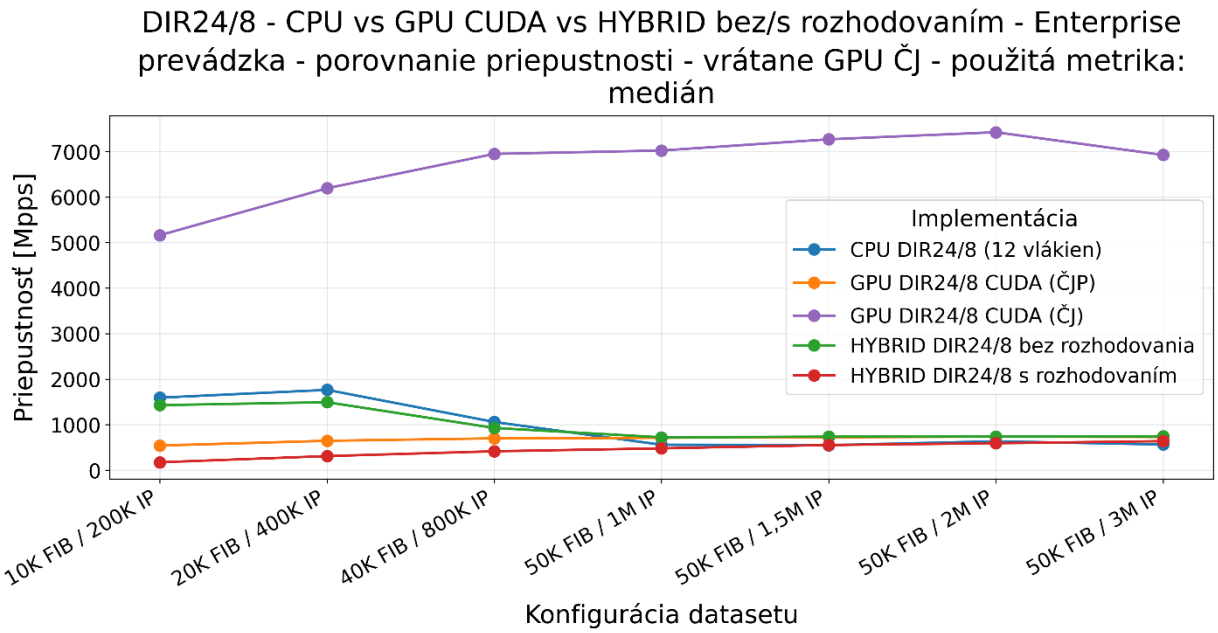
Obr. 32 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri syntetickej prevádzke vrátane merania samotného CUDA jadra



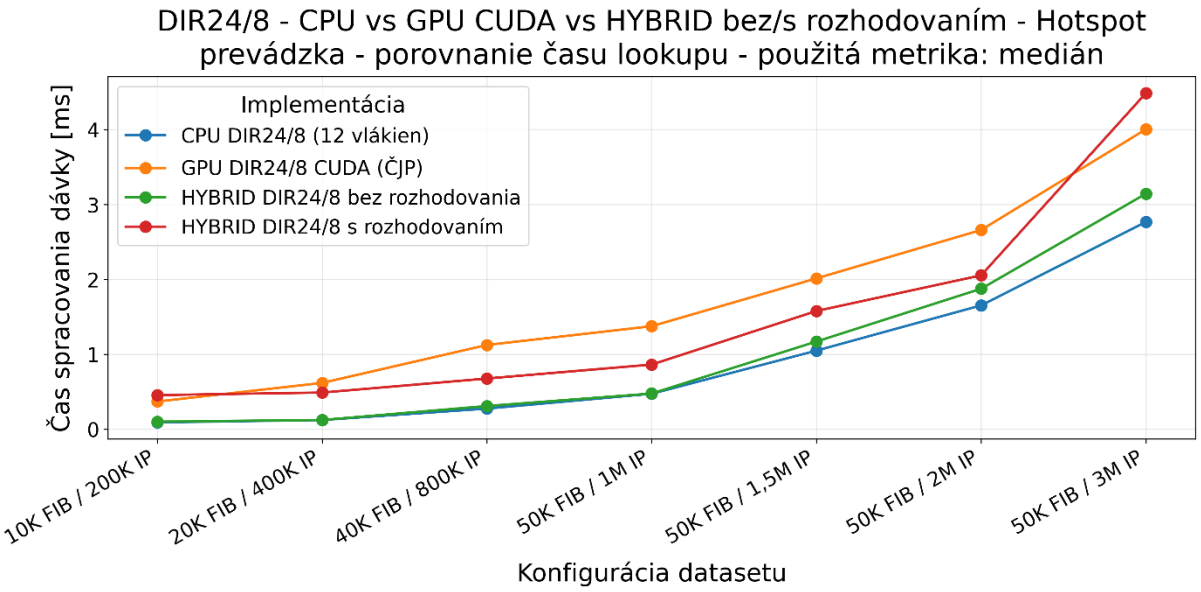
Obr. 33 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke



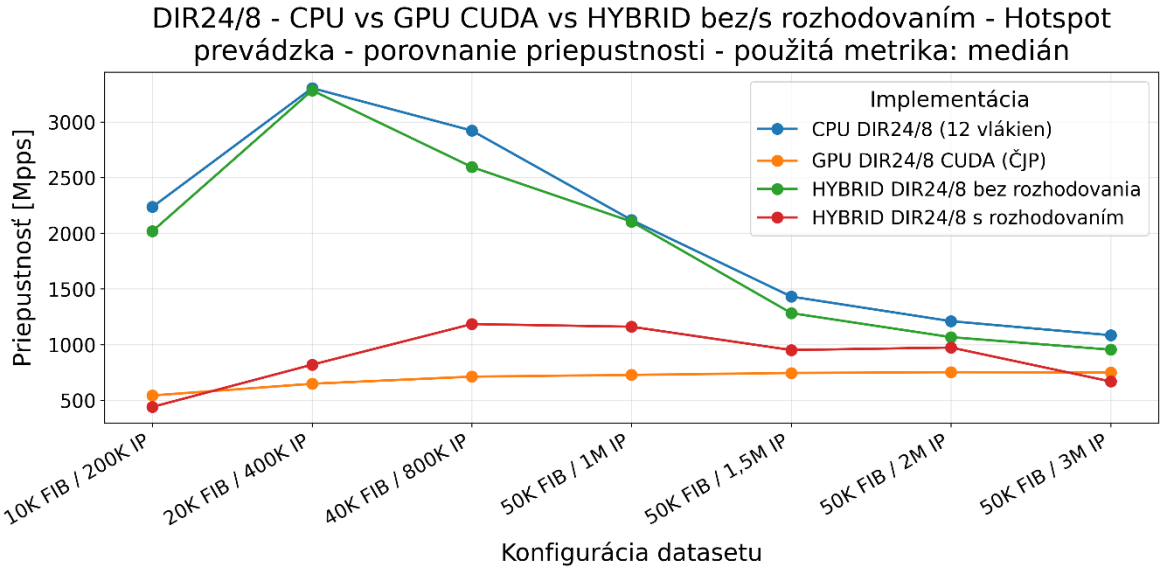
Obr. 34 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke



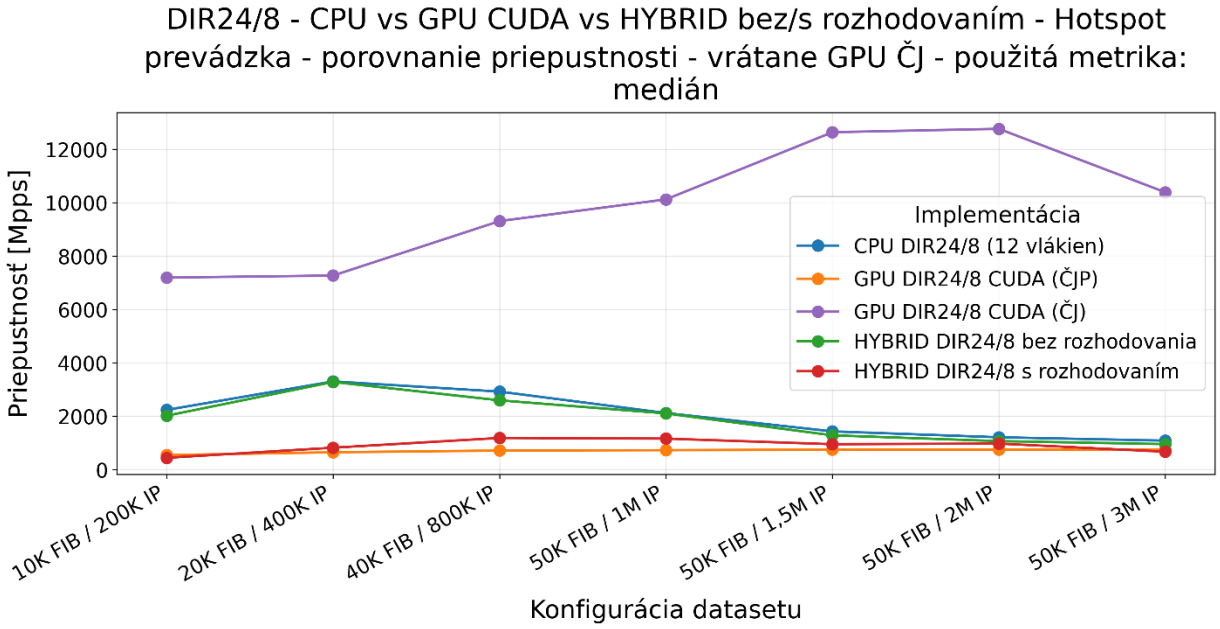
Obr. 35 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri enterprise prevádzke vrátane merania samotného CUDA jadra



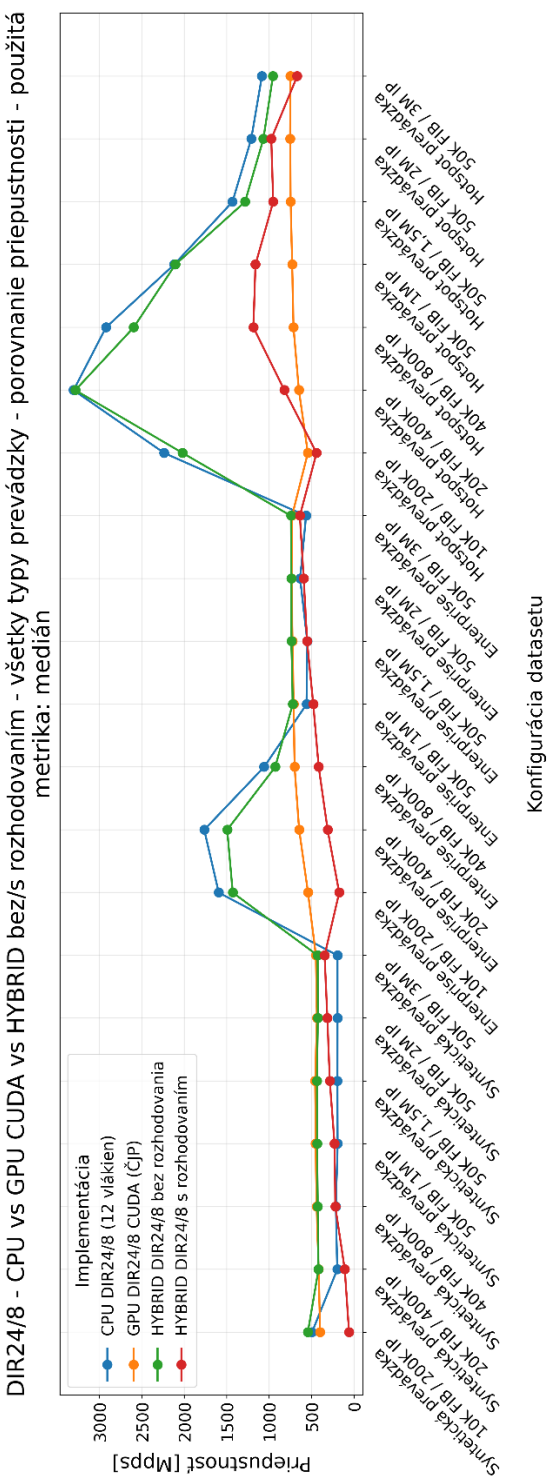
Obr. 36 - Porovnanie času spracovania dávky CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke

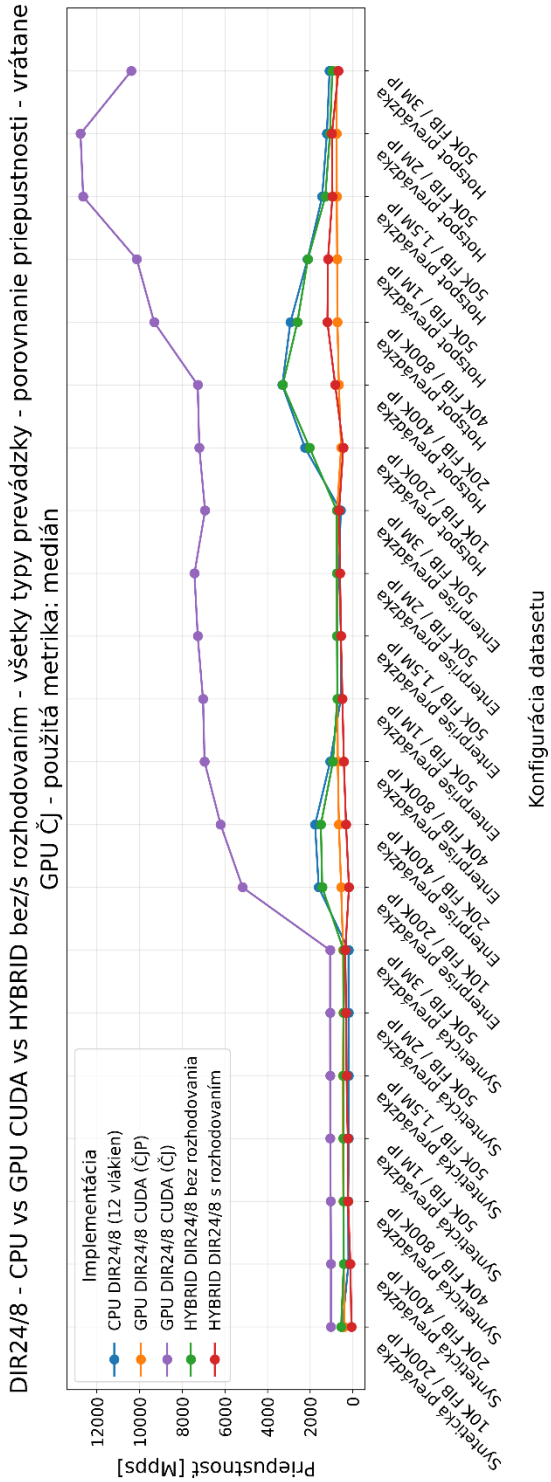


Obr. 37 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke



Obr. 38 - Porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pri hotspot prevádzke vrátane merania samotného CUDA jadra





Obr. 39 (vľavo) - Súhrnné porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pre všetky typy prevádzky

Obr. 40 (vpravo) - Súhrnné porovnanie priepustnosti CPU, GPU a hybridného DIR24/8 modelu pre všetky typy prevádzky vrátane merania samotného CUDA jadra

Dodatok C: Súhrnné CSV výstupy meracích skriptov

Tento dodatok uvádza prehľad výstupných súborov vytvorených meracími programami. Hlavným podkladom pre tabuľky a grafy v práci boli súbory typu summary.csv, ktoré obsahovali agregované výsledky pre jednotlivé implementácie, typy prevádzky a merané konfigurácie. Súbory typu raw_runs.csv obsahovali hodnoty jednotlivých meraných behov a slúžili ako technický podklad na výpočet súhrnných štatistík. Pri hybridných meraniach boli vytvorené aj textové logy s doplnkovým výpisom priebehu merania.

C.1 Súhrnné CSV výstupy CPU meraní

Tab. 12 - Súhrnné CSV výstupy CPU meraní algoritmu DIR24/8

Súbor	Obsah
cpu_dir24_8_lpm_synthetic_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU DIR24/8 pre syntetickú prevádzku pri 12 vláknach
cpu_dir24_8_lpm_enterprise_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU DIR24/8 pre enterprise prevádzku pri 12 vláknach
cpu_dir24_8_lpm_hotspot_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU DIR24/8 pre hotspot prevádzku pri 12 vláknach

Tab. 13 - Súhrnné CSV výstupy CPU meraní algoritmu MultiBit Trie

Súbor	Obsah
cpu_multibit_trie_lpm_synthetic_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU MultiBit Trie pre syntetickú prevádzku pri 12 vláknach
cpu_multibit_trie_lpm_enterprise_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU MultiBit Trie pre enterprise prevádzku pri 12 vláknach
cpu_multibit_trie_lpm_hotspot_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky CPU MultiBit Trie pre hotspot prevádzku pri 12 vláknach

C.2 Súhrnné CSV výstupy GPU meraní

Tab. 14 - Súhrnné CSV výstupy GPU meraní algoritmu DIR24/8

Súbor	Obsah
gpu_dir24_8_lpm_synthetic_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA DIR24/8 pre syntetickú prevádzku
gpu_dir24_8_lpm_enterprise_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA DIR24/8 pre enterprise prevádzku
gpu_dir24_8_lpm_hotspot_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA DIR24/8 pre hotspot prevádzku

Tab. 15 - Súhrnné CSV výstupy GPU meraní algoritmu MultiBit Trie

Súbor	Obsah
gpu_multibit_trie_lpm_synthetic_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA MultiBit Trie pre syntetickú prevádzku
gpu_multibit_trie_lpm_enterprise_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA MultiBit Trie pre enterprise prevádzku
gpu_multibit_trie_lpm_hotspot_ids1_2_3_4_5_6_7_summary.csv	Súhrnné výsledky CUDA MultiBit Trie pre hotspot prevádzku

C.3 Súhrnné CSV výstupy hybridného DIR24/8 modelu

Tab. 16 - Súhrnné CSV výstupy hybridného DIR24/8 modelu

Súbor	Obsah
hybrid_dir24_8_lpm_synthetic_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky hybridného DIR24/8 modelu pre syntetickú prevádzku
hybrid_dir24_8_lpm_enterprise_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky hybridného DIR24/8 modelu pre enterprise prevádzku
hybrid_dir24_8_lpm_hotspot_ids1_2_3_4_5_6_7_threads12_summary.csv	Súhrnné výsledky hybridného DIR24/8 modelu pre hotspot prevádzku

C.4 Raw výstupy a logy meraní

Tab. 17 - Doplnkové CSV výstupy a logy meraní

Súbor	Obsah
gpu_dir24_8_lpm_synthetic_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA DIR24/8 pre syntetickú prevádzku
gpu_dir24_8_lpm_enterprise_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA DIR24/8 pre enterprise prevádzku
gpu_dir24_8_lpm_hotspot_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA DIR24/8 pre hotspot prevádzku
gpu_multibit_trie_lpm_synthetic_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA MultiBit Trie pre syntetickú prevádzku
gpu_multibit_trie_lpm_enterprise_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA MultiBit Trie pre enterprise prevádzku
gpu_multibit_trie_lpm_hotspot_ids1_2_3_4_5_6_7_raw_runs.csv	Jednotlivé merané behy CUDA MultiBit Trie pre hotspot prevádzku
hybrid_dir24_8_lpm_enterprise_ids1_2_3_4_5_6_7_threads12_raw_runs.csv	Jednotlivé merané behy hybridného DIR24/8 modelu pre enterprise prevádzku
hybrid_dir24_8_lpm_hotspot_ids1_2_3_4_5_6_7_threads12_raw_runs.csv	Jednotlivé merané behy hybridného DIR24/8 modelu pre hotspot prevádzku
hybrid_dir24_8_lpm_enterprise_log.txt	Textový log hybridného merania pre enterprise prevádzku
hybrid_dir24_8_lpm_hotspot_log.txt	Textový log hybridného merania pre hotspot prevádzku

CSV výstupy s jednotlivými behmi neboli priamo použité ako hlavné tabuľky v práci, ale predstavovali podklad na overenie jednotlivých meraných behov. Finálne grafy a reprezentatívne tabuľky boli vytvorené zo súhrnných summary.csv výstupov.

C.5 Hlavné typy údajov v CSV výstupoch

Tab. 18 - Hlavné typy údajov v CSV výstupoch

Typ údaja	Význam
Veľkosť FIB	Počet prefixových záznamov v testovanej FIB tabuľke
Počet IP adries	Veľkosť spracovávanej vstupnej dávky
Čas spracovania dávky	Čas potrebný na spracovanie celej vstupnej dávky
Priepustnosť	Počet vykonaných LPM operácií za sekundu vyjadrený v Mpps
ČJ	Čas samotného CUDA jadra bez prenosovej réžie
ČJP	Čas GPU spracovania vrátane prenosu vstupov do GPU a výsledkov späť do CPU
Rozhodovacia réžia	Čas potrebný na analýzu vstupnej dávky a výber CPU alebo GPU vetvy v hybridnom modeli
Kontrolný súčet	Hodnota použitá na overenie konzistentnosti výsledkov medzi opakovanými behmi a implementáciami

Dodatok D: Výstupy profilovania Nsight Compute

Tento dodatok uvádza prehľad výstupov profilovania CUDA jadier pomocou nástroja NVIDIA Nsight Compute. Profilovanie bolo vykonané pre algoritmy DIR24/8 a MultiBit Trie pri syntetickej a hotspot prevádzke. Pre každý algoritmus a typ prevádzky boli vybrané tri reprezentatívne veľkosti vstupnej dávky: 200 000, 1 000 000 a 3 000 000 IPv4 adries.

Výstupy boli uložené vo formáte PDF a CSV. PDF súbory predstavujú vizuálne reporty z nástroja Nsight Compute, zatiaľ čo CSV súbory obsahujú exportované metriky v tabuľkovej podobe. Hlavná interpretácia profilovania je uvedená v kapitole 8.3.

D.1 PDF výstupy profilovania

Tab. 19 - PDF výstupy profilovania CUDA jadier pomocou Nsight Compute

Súbor	Obsah
CUDA_DIR24_8_SYNTHETIC_200k.pdf	Profilovanie CUDA DIR24/8 pri syntetickej prevádzke pre dávku 200 000 IPv4 adries
CUDA_DIR24_8_SYNTHETIC_1M.pdf	Profilovanie CUDA DIR24/8 pri syntetickej prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_DIR24_8_SYNTHETIC_3M.pdf	Profilovanie CUDA DIR24/8 pri syntetickej prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_200k.pdf	Profilovanie CUDA DIR24/8 pri hotspot prevádzke pre dávku 200 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_1M.pdf	Profilovanie CUDA DIR24/8 pri hotspot prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_3M.pdf	Profilovanie CUDA DIR24/8 pri hotspot prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_200k.pdf	Profilovanie CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 200 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_1M.pdf	Profilovanie CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_3M.pdf	Profilovanie CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_MultiBit_Trie_HOTSPOT_200k.pdf	Profilovanie CUDA MultiBit Trie pri hotspot prevádzke pre dávku 200 000 IPv4 adries

CUDA_MultiBit_Trie_HOTSPOT_1M.pdf	Profilovanie CUDA MultiBit Trie pri hotspot prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_MultiBit_Trie_HOTSPOT_3M.pdf	Profilovanie CUDA MultiBit Trie pri hotspot prevádzke pre dávku 3 000 000 IPv4 adries

D.2 CSV výstupy profilovania

Tab. 20 - CSV výstupy profilovania CUDA jadier pomocou Nsight Compute

Súbor	Obsah
CUDA_DIR24_8_SYNTHETIC_200k.csv	Export metrík CUDA DIR24/8 pri syntetickej prevádzke pre dávku 200 000 IPv4 adries
CUDA_DIR24_8_SYNTHETIC_1M.csv	Export metrík CUDA DIR24/8 pri syntetickej prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_DIR24_8_SYNTHETIC_3M.csv	Export metrík CUDA DIR24/8 pri syntetickej prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_200k.csv	Export metrík CUDA DIR24/8 pri hotspot prevádzke pre dávku 200 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_1M.csv	Export metrík CUDA DIR24/8 pri hotspot prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_DIR24_8_HOTSPOT_3M.csv	Export metrík CUDA DIR24/8 pri hotspot prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_200k.csv	Export metrík CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 200 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_1M.csv	Export metrík CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_MultiBit_Trie_SYNTHETIC_3M.csv	Export metrík CUDA MultiBit Trie pri syntetickej prevádzke pre dávku 3 000 000 IPv4 adries
CUDA_MultiBit_Trie_HOTSPOT_200k.csv	Export metrík CUDA MultiBit Trie pri hotspot prevádzke pre dávku 200 000 IPv4 adries
CUDA_MultiBit_Trie_HOTSPOT_1M.csv	Export metrík CUDA MultiBit Trie pri hotspot prevádzke pre dávku 1 000 000 IPv4 adries
CUDA_MultiBit_Trie_HOTSPOT_3M.csv	Export metrík CUDA MultiBit Trie pri hotspot prevádzke pre dávku 3 000 000 IPv4 adries

D.3 Sledované metriky

Tab. 21 - Vybrané metriky použité pri interpretácii profilovania

Metrika	Význam
gpu__time_duration.sum	Čas trvania profilovaného CUDA jadra
sm__throughput.avg.pct_of_peak_sustained_elapsed	Využitie výpočtových prostriedkov SM voči teoretickému maximu
gpu__dram_throughput.avg.pct_of_peak_sustained_elapsed	Využitie priepustnosti globálnej DRAM pamäte GPU
lts__t_sector_hit_rate.pct	Miera zásahov v L2 cache
smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct	Podiel čakania warpov na dáta z pamäťovej hierarchie
sm__warps_active.avg.pct_of_peak_sustained_active	Dosiahnutá obsadenosť, teda miera aktívnych warpov voči teoretickému maximu

Uvedené metriky boli použité na posúdenie, či výkon CUDA jadier viac obmedzovali výpočtové prostriedky alebo pamäťové správanie. Súhrn týchto hodnôt pre vybrané konfigurácie je uvedený v kapitole 8.3.